

POSIX Threads: POSIX stands for Portable Operating System Interface. The *POSIX.4* standard deals with the Real-Time extensions and *POSIX.4a* standard deals with thread extensions. The POSIX standard library for thread creation and management is '*Pthreads*'. '*Pthreads*' library defines the set of POSIX thread creation and management functions in 'C' language.

The primitive

```
int pthread_create(pthread_t *new_thread_ID, const pthread_attr_t *attribute,
void * (*start_function)(void *), void *arguments);
```

creates a new thread for running the function *start_function*. Here *pthread_t* is the handle to the newly created thread and *pthread_attr_t* is the data type for holding the thread attributes. '*start_function*' is the function the thread is going to execute and *arguments* is the arguments for '*start_function*' (It is a void * in the above example). On successful creation of a *Pthread*, *pthread_create()* associates the Thread Control Block (TCB) corresponding to the newly created thread to the variable of type *pthread_t* (*new_thread_ID* in our example).

The primitive

```
int pthread_join(pthread_t new_thread, void * *thread_status);
```

blocks the current thread and waits until the completion of the thread pointed by it (In this example *new_thread*)

All the POSIX 'thread calls' returns an integer. A return value of zero indicates the success of the call. It is always good to check the return value of each call.

Example 1

Write a multithreaded application to print "Hello I'm in main thread" from the main thread and "Hello I'm in new thread" 5 times each, using the *pthread_create()* and *pthread_join()* POSIX primitives.

```
//Assumes the application is running on an OS where POSIX library is
//available
#include <pthread.h>
#include <stdlib.h>
#include <stdio.h>
//*****
//New thread function for printing "Hello I'm in new thread"
void *new_thread( void *thread_args )
{
    int i, j;
    for( j= 0; j < 5; j++ )
    {
        printf("Hello I'm in new thread\n" );
        //Wait for some time. Do nothing
        //The following line of code can be replaced with
        //OS supported delay function like sleep(), delay () etc...
        for( i= 0; i < 10000; i++ );
    }
    return NULL;
}
```



```

//*****
//Start of main thread
int main( void )
{
    int i, j;
    pthread_t tcb;
    //Create the new thread for executing new_thread function
    if (pthread_create( &tcb, NULL, new_thread, NULL ))
    {
        //New thread creation failed
        printf("Error in creating new thread\n" );
        return -1;
    }
    for( j= 0; j < 5; j++ )
    {
        printf("Hello I'm in main thread\n" );
        //Wait for some time. Do nothing
        //The following line of code can be replaced with
        //OS supported delay function like sleep(), delay etc...
        for( i= 0; i < 10000; i++ );
    }
    if (pthread_join(tcb, NULL ))
    {
        //Thread join failed
        printf("Error in Thread join\n" );
        return -1;
    }
    return 1;
}

```

You can compile this application using the *gcc* compiler. Examine the output to figure out the thread execution switching. The lines printed will give an idea of the order in which the thread execution is switched between. The *pthread_join* call forces the main thread to wait until the completion of the thread *tcb*, if the main thread finishes the execution first.

The termination of a thread can happen in different ways. The thread can terminate either by completing its execution (natural termination) or by a forced termination. In a natural termination, the thread completes its execution and returns back to the main thread through a simple *return* or by executing the *pthread_exit()* call. Forced termination can be achieved by the call *pthread_cancel()* or through the termination of the main thread with *exit* or *exec* functions. *pthread_cancel()* call is used by a thread to terminate another thread.

pthread_exit() call is used by a thread to explicitly exit after it completes its work and is no longer required to exist. If the main thread finishes before the threads it has created, and exits with *pthread_exit()*, the other threads continue to execute. If the main thread uses *exit* call to exit the thread, all threads created by the main thread is terminated forcefully. Exiting a thread with the call *pthread_exit()* will not perform a cleanup. It will not close any files opened by the thread and files will remain in the open status even after the thread terminates. Calling *pthread_join* at the end of the main thread is the best way to achieve synchronisation and proper cleanup. The main thread, after finishing its task waits for the completion of other threads, which were joined to it using the *pthread_join* call. With a *pthread_join* call, the main thread waits other threads, which were joined to it, and finally merges to the single main thread. If a new thread spawned by the main thread is still not joined to the main thread, it will be counted against the system's maximum thread limit. Improper cleanup will lead to the failure of new thread creation.

Win32 Threads Win32 threads are the threads supported by various flavours of Windows Operating Systems. The Win32 Application Programming Interface (Win32 API) libraries provide the standard set of Win32 thread creation and management functions. Win32 threads are created with the API

```
HANDLE CreateThread(LPSECURITY_ATTRIBUTES lpThreadAttributes, DWORD dwStackSize, LPTHREAD_START_ROUTINE lpStartAddress, LPVOID lpParameter, DWORD dwCreationFlags, LPDWORD lpThreadId );
```

The parameter *lpThreadAttributes* defines the security attributes for the thread and *dwStackSize* defines the stack size for the thread. These two parameters are not supported by the Windows CE Real-Time Operating Systems and it should be kept as NULL and 0 respectively in a *CreateThread* API Call. The other parameters are

lpStartAddress: Pointer to the function which is to be executed by the thread.

lpParameter: Parameter specifying an application-defined value that is passed to the thread routine.

dwCreationFlags: Defines the state of the thread when it is created. Usually it is kept as 0 or *CREATE_SUSPENDED* implying the thread is created and kept at the suspended state.

lpThreadId: Pointer to a DWORD that receives the identifier for the thread.

On successful creation of the thread, *CreateThread* returns the handle to the thread and the thread identifier.

The *APIGetCurrentThread(void)* returns the handle of the current thread and *GetCurrentThreadId(void)* returns its ID. *GetThreadPriority (HANDLE hThread)* API returns an integer value representing the current priority of the thread whose handle is passed as *hThread*. Threads are always created with normal priority (*THREAD_PRIORITY_NORMAL*). Refer MSDN documentation for the different thread priorities and their meaning). *SetThreadPriority (HANDLE hThread, int nPriority)* API is used for setting the priority of a thread. The first parameter to this function represents the thread handle and the second one the thread priority.

For Win32 threads, the normal thread termination happens when an exception occurs in the thread, or when the thread's execution is completed or when the primary thread or the process to which the thread is associated is terminated. A thread can exit itself by calling the *ExitThread (DWORD dwExitCode)* API. The parameter *dwExitCode* sets the exit code for thread termination. Calling *ExitThread* API frees all the resources utilised by the thread. The exit code of a thread can be checked by other threads by calling the *GetExitCodeThread (HANDLE hThread, LPDWORD lpExitCode)*. *TerminateThread (HANDLE hThread, DWORD dwExitCode)* API is used for terminating a thread from another thread. The handle *hThread* indicates which thread is to be terminated and *dwExitCode* sets the exit code for the thread. This API will not execute the thread termination and clean up code and may not free the resources occupied by the thread. *TerminateThread* is a potentially dangerous call and it should not be used in normal conditions as a mechanism for terminating a thread. Use this call only as a final choice. *SuspendThread (HANDLE hThread)* API can be used for suspending a thread from execution provided the handle *hThread* possesses *THREAD_SUSPEND_RESUME* access right. If the *SuspendThread* API call succeeds, the thread stops executing and increments its internal suspend count. The thread becomes suspended if its suspend count is greater than zero. The *SuspendThread* function is primarily designed for use by debuggers. One must be cautious in using this API for the reason it may cause *deadlock* condition if the thread is suspended at a stage where it acquired a mutex or shared resource and another thread tries to access the same. The *ResumeThread (HANDLE hThread)* API is used for resuming a suspended thread. The *ResumeThread* API checks the suspend count of the specified thread. A suspend count of

zero indicates that the specified thread is not currently in the suspended mode. If the count is not zero, the count is decremented by one and if the resulting count value is zero, the thread is resumed. The API *Sleep* (*DWORD dwMilliseconds*) can be used for suspending a thread for the duration specified in milliseconds by the *Sleep* function. The *Sleep* call is initiated by the thread.

Example 2

Write a multithreaded application using Win32 APIs to set up a counter in the main thread and secondary thread to count from 0 to 10 and print the counts from both the threads. Put a delay of 500 ms in between the successive printing in both the threads.

```
#include "windows.h"
#include "stdio.h"
//*****
//Child thread
//*****
void ChildThread(void)
{
    char i;
    for(i=0;i<=10;++i)
    {
        printf("Executing Child Thread:Counter = %d\n",i);
        Sleep(500);
    }
}
//*****
//Primary thread
//*****
int main(int argc, char* argv[])
{
    HANDLE hThread;
    DWORD dwThreadID;
    char i;
    hThread=CreateThread(NULL,1000,(LPTHREAD_START_ROUTINE)
    ChildThread, NULL, 0, &dwThreadID);
    if(hThread==NULL)
    {
        printf("Thread Creation Failed\nError No:
        %d\n",GetLastError());
        return 1;
    }
    for(i=0;i<=10;++i)
    {
        printf("Executing Main Thread: Counter = %d\n",i);
        Sleep(500);
    }
    return 0;
}
```


To execute this program, create a new Win32 Console Application using Microsoft Visual C++ and add the above piece of code to it and compile. The output obtained on running this application on a machine with Windows XP operating system is given in Fig. 10.9.

If you examine the output, you can see the switching between main and child threads. The output need not be the same always. The output is purely dependent on the scheduling policies implemented by the windows operating system for thread scheduling. You may get the same output or a different output each time you run the application.

Java Threads Java threads are the threads supported by Java programming Language. The java thread class '*Thread*' is defined in the package '*java.lang*'. This package needs to be imported for using the thread creation functions supported by the Java thread class. There are two ways of creating threads in Java: Either by extending the base '*Thread*' class or by implementing an interface. Extending the thread class allows inheriting the methods and variables of the parent class (Thread class) only whereas interface allows a way to achieve the requirements for a set of classes. The following piece of code illustrates the implementation of Java threads with extending the thread base class '*Thread*'.

```
import java.lang.*;
public class MyThread extends Thread
{
    public void run()
    {
        System.out.println("Hello from MyThread!");
    }
    public static void main(String args[])
    {
        (new MyThread()).start();
    }
}
```

The above piece of code creates a new class *MyThread* by extending the base class *Thread*. It also overrides the *run()* method inherited from the base class with its own *run()* method. The *run()* method of *MyThread* implements all the task for the *MyThread* thread. The method *start()* moves the thread to a pool of threads waiting for their turn to be picked up for execution by the scheduler. The thread is said to be in the 'Ready' state at this stage. The scheduler picks the threads for execution from the pool based on the thread priorities.

E.g. *MyThread.start()*;

The output of the above piece of code when executed on Windows XP platform is given in Fig. 10.10.

```
D:\VC++\Thread\Debug\Thread.exe
Executing Main Thread: Counter = 0
Executing Child Thread: Counter = 0
Executing Main Thread: Counter = 1
Executing Child Thread: Counter = 1
Executing Main Thread: Counter = 2
Executing Child Thread: Counter = 2
Executing Main Thread: Counter = 3
Executing Child Thread: Counter = 3
Executing Main Thread: Counter = 4
Executing Child Thread: Counter = 4
Executing Main Thread: Counter = 5
Executing Child Thread: Counter = 5
Executing Main Thread: Counter = 6
Executing Child Thread: Counter = 6
Executing Main Thread: Counter = 7
Executing Child Thread: Counter = 7
Executing Main Thread: Counter = 8
Executing Child Thread: Counter = 8
Executing Main Thread: Counter = 9
Executing Child Thread: Counter = 9
Executing Main Thread: Counter = 10
Executing Child Thread: Counter = 10
Press any key to continue_
```

Fig. 10.9 Output of the Win32 Multithreaded application


```

C:\SYSROOT\system32\cmd.exe
D:\JavaHome\jre1.5.0_11\bin>java MyThread
Hello from MyThread!
D:\JavaHome\jre1.5.0_11\bin>

```

Fig. 10.10 Output of the Java Multithreaded application

Invoking the static method *yield()* voluntarily give up the execution of the thread and the thread is moved to the pool of threads waiting to get their turn for execution, i.e. the thread enters the 'Ready' state.

E.g. `MyThread.yield();`

The static method *sleep()* forces the thread to sleep for the duration mentioned by the sleep call, i.e. the thread enters the 'Suspend' mode. Once the sleep period is expired, the thread is moved to the pool of threads waiting to get their turn for execution, i.e. the thread enters the 'Ready' state. The method *sleep()* only guarantees that the thread will sleep for the minimum period mentioned by the argument to the call. It will not guarantee anything on the resume of the thread after the sleep period. It is dependent on the scheduler.

E.g. `MyThread.sleep(100);` Sleep for 100 milliseconds.

Calling a thread Object's *wait()* method causes the thread object to wait. The thread will remain in the 'Wait' state until another thread invokes the *notify()* or *notifyAll()* method of the thread object which is waiting. The thread enters the 'Blocked' state when waiting for input from I/O devices or waiting for object lock in case of accessing shared resources. The thread is moved to the 'Ready' state on receiving the I/O input or on acquiring the object lock. The thread enters the 'Finished/Dead' state on completion of the task assigned to it or when the *stop()* method is explicitly invoked. The thread may also enter this state if it is terminated by an unrecoverable error condition.

For more information on Java threads, visit Sun Micro System's tutorial on Threads, available at <http://java.sun.com/tutorial/applet/overview/threads.html>

Summary So far we discussed about the various thread classes available for creation and management of threads in a multithreaded system in a General Purpose Operating System's perspective. From an RTOS perspective, POSIX threads and Win32 threads are the most commonly used thread class libraries for thread creation and management. Many non-standard, proprietary thread classes are also used by some proprietary RTOS. Portable threads (*Pth*), a very portable POSIX/ANSI-C based library from GNU, may be the "next generation" threads library. *Pth* provides non-preemptive priority based scheduling for multiple threads inside event driven applications. Visit <http://www.gnu.org/software/pth/> for more details on GNU Portable threads.

10.3.2.3 Thread Pre-emption Thread pre-emption is the act of pre-empting the currently running thread (stopping the currently running thread temporarily). Thread pre-emption ability is solely dependent on the Operating System. Thread pre-emption is performed for sharing the CPU time among all the threads. The execution switching among threads are known as 'Thread context switching'. Thread context switching is dependent on the Operating system's scheduler and the type of the thread. When we say 'Thread', it falls into any one of the following types.

User Level Thread User level threads do not have kernel/Operating System support and they exist solely in the running process. Even if a process contains multiple user level threads, the OS treats it as single thread and will not switch the execution among the different threads of it. It is the responsibility of the process to schedule each thread as and when required. In summary, user level threads of a process are non-preemptive at thread level from OS perspective.

Kernel/System Level Thread Kernel level threads are individual units of execution, which the OS treats as separate threads. The OS interrupts the execution of the currently running kernel thread and switches the execution to another kernel thread based on the scheduling policies implemented by the OS. In summary kernel level threads are pre-emptive.

For user level threads, the execution switching (thread context switching) happens only when the currently executing user level thread is voluntarily blocked. Hence, no OS intervention and system calls are involved in the context switching of user level threads. This makes context switching of user level threads very fast. On the other hand, kernel level threads involve lots of kernel overhead and involve system calls for context switching. However, kernel threads maintain a clear layer of abstraction and allow threads to use system calls independently. There are many ways for binding user level threads with system/kernel level threads. The following section gives an overview of various thread binding models.

Many-to-One Model Here many user level threads are mapped to a single kernel thread. In this model, the kernel treats all user level threads as single thread and the execution switching among the user level threads happens when a currently executing user level thread voluntarily blocks itself or relinquishes the CPU. Solaris Green threads and GNU Portable Threads are examples for this. The '*PThread*' example given under the POSIX thread library section is an illustrative example for application with Many-to-One thread model.

One-to-One Model In One-to-One model, each user level thread is bonded to a kernel/system level thread. Windows XP/NT/2000 and Linux threads are examples for One-to-One thread models. The modified '*PThread*' example given under the '*Thread Pre-emption*' section is an illustrative example for application with One-to-One thread model.

Many-to-Many Model In this model many user level threads are allowed to be mapped to many kernel threads. Windows NT/2000 with *ThreadFibre* package is an example for this.

10.3.2.4 Thread v/s Process I hope, by now you got a reasonably good knowledge of *process* and *threads*. Now let us summarise the properties of *process* and *threads*.

Thread	Process
Thread is a single unit of execution and is part of process.	Process is a program in execution and contains one or more threads.
A thread does not have its own data memory and heap memory. It shares the data memory and heap memory with other threads of the same process.	Process has its own code memory, data memory and stack memory.
A thread cannot live independently; it lives within the process.	A process contains at least one thread.
There can be multiple threads in a process. The first thread (main thread) calls the main function and occupies the start of the stack memory of the process.	Threads within a process share the code, data and heap memory. Each thread holds separate memory area for stack (shares the total stack memory of the process).

Threads are very inexpensive to create

Processes are very expensive to create. Involves many OS overhead.

Context switching is inexpensive and fast

Context switching is complex and involves lot of OS overhead and is comparatively slower.

If a thread expires, its stack is reclaimed by the process.

If a process dies, the resources allocated to it are reclaimed by the OS and all the associated threads of the process also dies.

10.4 MULTIPROCESSING AND MULTITASKING

The terms *multiprocessing* and *multitasking* are a little confusing and sounds alike. In the operating system context *multiprocessing* describes the ability to execute multiple processes simultaneously. Systems which are capable of performing multiprocessing, are known as *multiprocessor* systems. *Multiprocessor* systems possess multiple CPUs and can execute multiple processes simultaneously.

The ability of the operating system to have multiple programs in memory, which are ready for execution, is referred as *multiprogramming*. In a uniprocessor system, it is not possible to execute multiple processes simultaneously. However, it is possible for a uniprocessor system to achieve some degree of pseudo parallelism in the execution of multiple processes by switching the execution among different processes. The ability of an operating system to hold multiple processes in memory and switch the processor (CPU) from executing one process to another process is known as *multitasking*. Multitasking creates the illusion of multiple tasks executing in parallel. Multitasking involves the switching of CPU from executing one task to another. In an earlier section '*The Structure of a Process*' of this chapter, we learned that a Process is identical to the physical processor in the sense it has own register set which mirrors the CPU registers, stack and Program Counter (PC). Hence, a '*process*' is considered as a '*Virtual processor*', awaiting its turn to have its properties switched into the physical processor. In a multitasking environment, when task/process switching happens, the virtual processor (task/process) gets its properties converted into that of the physical processor. The switching of the virtual processor to physical processor is controlled by the scheduler of the OS kernel. Whenever a CPU switching happens, the current context of execution should be saved to retrieve it at a later point of time when the CPU executes the process, which is interrupted currently due to execution switching. The context saving and retrieval is essential for resuming a process exactly from the point where it was interrupted due to CPU switching. The act of switching CPU among the processes or changing the current execution context is known as '*Context switching*'. The act of saving the current context which contains the context details (Register details, memory details, system resource usage details, execution details, etc.) for the currently running process at the time of CPU switching is known as '*Context saving*'. The process of retrieving the saved context details for a process, which is going to be executed due to CPU switching, is known as '*Context retrieval*'. Multitasking involves '*Context switching*' (Fig. 10.11), '*Context saving*' and '*Context retrieval*'.

Toss juggling – The skilful object manipulation game is a classic real world example for the multitasking illusion. The juggler uses a number of objects (balls, rings, etc.) and throws them up and catches them. At any point of time, he throws only one ball and catches only one per hand. However, the speed at which he is switching the balls for throwing and catching creates the illusion, he is throwing and catching multiple balls or using more than two hands ☺ simultaneously, to the spectators.

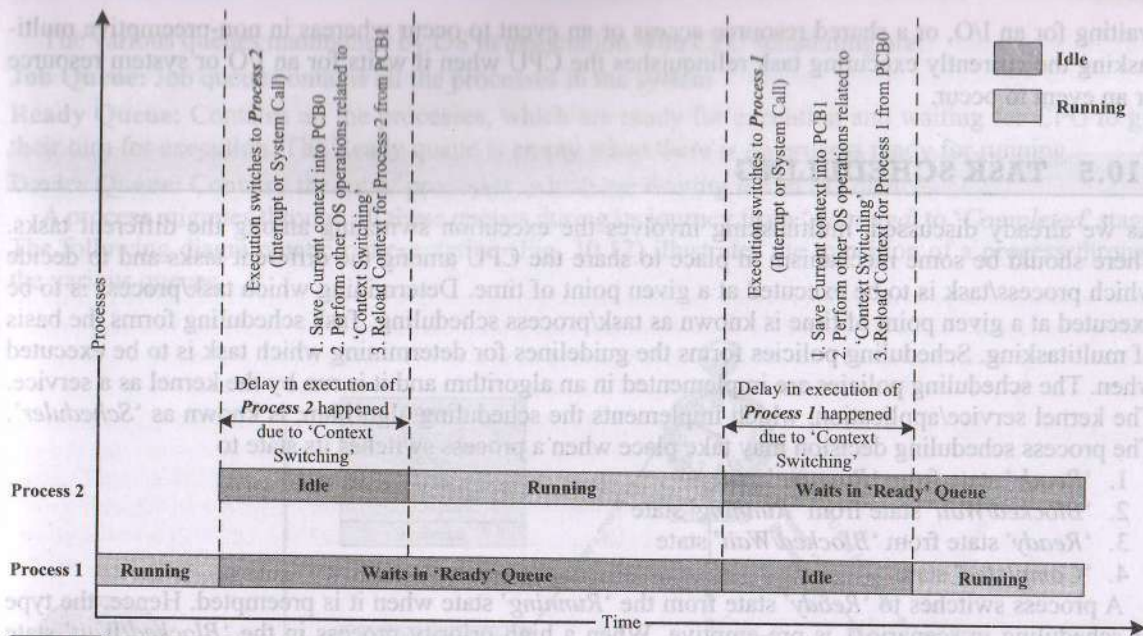


Fig. 10.11 Context switching

10.4.1 Types of Multitasking

As we discussed earlier, multitasking involves the switching of execution among multiple tasks. Depending on how the switching act is implemented, multitasking can be classified into different types. The following section describes the various types of multitasking existing in the Operating System's context.

10.4.1.1 Co-operative Multitasking Co-operative multitasking is the most primitive form of multitasking in which a task/process gets a chance to execute only when the currently executing task/process voluntarily relinquishes the CPU. In this method, any task/process can hold the CPU as much time as it wants. Since this type of implementation involves the mercy of the tasks each other for getting the CPU time for execution, it is known as co-operative multitasking. If the currently executing task is non-cooperative, the other tasks may have to wait for a long time to get the CPU.

10.4.1.2 Preemptive Multitasking Preemptive multitasking ensures that every task/process gets a chance to execute. When and how much time a process gets is dependent on the implementation of the preemptive scheduling. As the name indicates, in preemptive multitasking, the currently running task/process is preempted to give a chance to other tasks/process to execute. The preemption of task may be based on time slots or task/process priority.

10.4.1.3 Non-preemptive Multitasking In non-preemptive multitasking, the process/task, which is currently given the CPU time, is allowed to execute until it terminates (enters the 'Completed' state) or enters the 'Blocked/Wait' state, waiting for an I/O or system resource. The co-operative and non-preemptive multitasking differs in their behaviour when they are in the 'Blocked/Wait' state. In co-operative multitasking, the currently executing process/task need not relinquish the CPU when it enters the 'Blocked/Wait' state,

waiting for an I/O, or a shared resource access or an event to occur whereas in non-preemptive multitasking the currently executing task relinquishes the CPU when it waits for an I/O or system resource or an event to occur.

10.5 TASK SCHEDULING

As we already discussed, multitasking involves the execution switching among the different tasks. There should be some mechanism in place to share the CPU among the different tasks and to decide which process/task is to be executed at a given point of time. Determining which task/process is to be executed at a given point of time is known as task/process scheduling. Task scheduling forms the basis of multitasking. Scheduling policies forms the guidelines for determining which task is to be executed when. The scheduling policies are implemented in an algorithm and it is run by the kernel as a service. The kernel service/application, which implements the scheduling algorithm, is known as '*Scheduler*'. The process scheduling decision may take place when a process switches its state to

1. '*Ready*' state from '*Running*' state
2. '*Blocked/Wait*' state from '*Running*' state
3. '*Ready*' state from '*Blocked/Wait*' state
4. '*Completed*' state

A process switches to '*Ready*' state from the '*Running*' state when it is preempted. Hence, the type of scheduling in scenario 1 is pre-emptive. When a high priority process in the '*Blocked/Wait*' state completes its I/O and switches to the '*Ready*' state, the scheduler picks it for execution if the scheduling policy used is priority based preemptive. This is indicated by scenario 3. In preemptive/non-preemptive multitasking, the process relinquishes the CPU when it enters the '*Blocked/Wait*' state or the '*Completed*' state and switching of the CPU happens at this stage. Scheduling under scenario 2 can be either preemptive or non-preemptive. Scheduling under scenario 4 can be preemptive, non-preemptive or co-operative.

The selection of a scheduling criterion/algorithm should consider the following factors:

CPU Utilisation: The scheduling algorithm should always make the CPU utilisation high. CPU utilisation is a direct measure of how much percentage of the CPU is being utilised.

Throughput: This gives an indication of the number of processes executed per unit of time. The throughput for a good scheduler should always be higher.

Turnaround Time: It is the amount of time taken by a process for completing its execution. It includes the time spent by the process for waiting for the main memory, time spent in the ready queue, time spent on completing the I/O operations, and the time spent in execution. The turnaround time should be a minimal for a good scheduling algorithm.

Waiting Time: It is the amount of time spent by a process in the '*Ready*' queue waiting to get the CPU time for execution. The waiting time should be minimal for a good scheduling algorithm.

Response Time: It is the time elapsed between the submission of a process and the first response. For a good scheduling algorithm, the response time should be as least as possible.

To summarise, a good scheduling algorithm has high CPU utilisation, minimum Turn Around Time (TAT), maximum throughput and least response time.

The Operating System maintains various queues[†] in connection with the CPU scheduling, and a process passes through these queues during the course of its admittance to execution completion.

[†] Queue is a special kind of arrangement of a collection of objects. In the operating system context queue is considered as a buffer.

The various queues maintained by OS in association with CPU scheduling are:

Job Queue: Job queue contains all the processes in the system

Ready Queue: Contains all the processes, which are ready for execution and waiting for CPU to get their turn for execution. The Ready queue is empty when there is no process ready for running.

Device Queue: Contains the set of processes, which are waiting for an I/O device.

A process migrates through all these queues during its journey from 'Admitted' to 'Completed' stage. The following diagrammatic representation (Fig. 10.12) illustrates the transition of a process through the various queues.

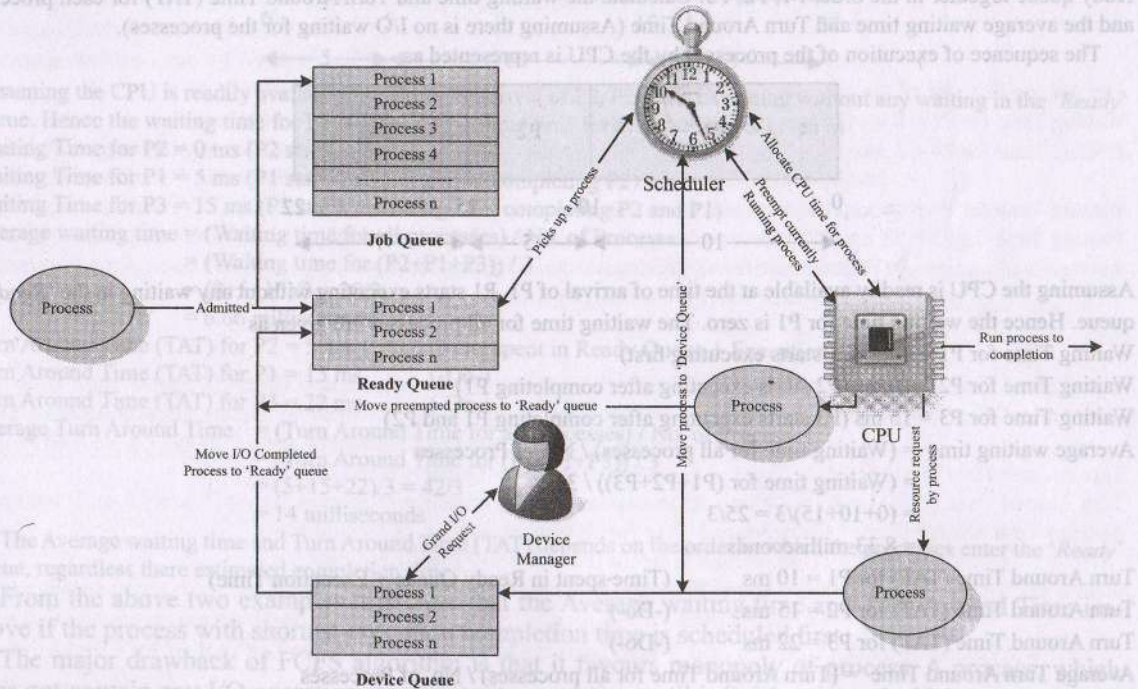


Fig. 10.12 Illustration of process transition through various queues

Based on the scheduling algorithm used, the scheduling can be classified into the following categories.

10.5.1 Non-preemptive Scheduling

Non-preemptive scheduling is employed in systems, which implement non-preemptive multitasking model. In this scheduling type, the currently executing task/process is allowed to run until it terminates or enters the 'Wait' state waiting for an I/O or system resource. The various types of non-preemptive scheduling adopted in task/process scheduling are listed below.

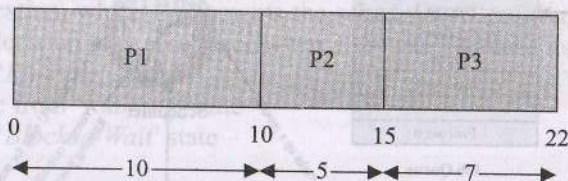
10.5.1.1 First-Come-First-Served (FCFS)/ FIFO Scheduling As the name indicates, the First-Come-First-Served (FCFS) scheduling algorithm allocates CPU time to the processes based on the

order in which they enter the 'Ready' queue. The first entered process is serviced first. It is same as any real world application where queue systems are used; e.g. Ticketing reservation system where people need to stand in a queue and the first person standing in the queue is serviced first. FCFS scheduling is also known as First In First Out (FIFO) where the process which is put first into the 'Ready' queue is serviced first.

Example 1

Three processes with process IDs P1, P2, P3 with estimated completion time 10, 5, 7 milliseconds respectively enters the ready queue together in the order P1, P2, P3. Calculate the waiting time and Turn Around Time (TAT) for each process and the average waiting time and Turn Around Time (Assuming there is no I/O waiting for the processes).

The sequence of execution of the processes by the CPU is represented as



Assuming the CPU is readily available at the time of arrival of P1, P1 starts executing without any waiting in the 'Ready' queue. Hence the waiting time for P1 is zero. The waiting time for all processes are given as

Waiting Time for P1 = 0 ms (P1 starts executing first)

Waiting Time for P2 = 10 ms (P2 starts executing after completing P1)

Waiting Time for P3 = 15 ms (P3 starts executing after completing P1 and P2)

Average waiting time = (Waiting time for all processes) / No. of Processes

$$= (\text{Waiting time for (P1+P2+P3)}) / 3$$

$$= (0+10+15)/3 = 25/3$$

$$= 8.33 \text{ milliseconds}$$

Turn Around Time (TAT) for P1 = 10 ms (Time spent in Ready Queue + Execution Time)

Turn Around Time (TAT) for P2 = 15 ms (-Do-)

Turn Around Time (TAT) for P3 = 22 ms (-Do-)

Average Turn Around Time = (Turn Around Time for all processes) / No. of Processes

$$= (\text{Turn Around Time for (P1+P2+P3)}) / 3$$

$$= (10+15+22)/3 = 47/3$$

$$= 15.66 \text{ milliseconds}$$

Average Turn Around Time (TAT) is the sum of average waiting time and average execution time.

Average Execution Time = (Execution time for all processes)/No. of processes

$$= (\text{Execution time for (P1+P2+P3)})/3$$

$$= (10+5+7)/3 = 22/3$$

$$= 7.33$$

Average Turn Around Time = Average waiting time + Average execution time

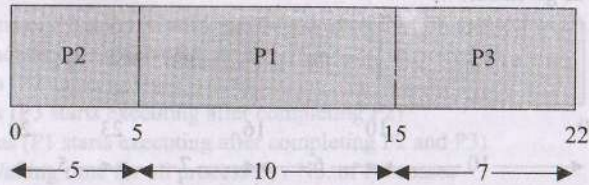
$$= 8.33 + 7.33$$

$$= 15.66 \text{ milliseconds}$$

Example 2

Calculate the waiting time and Turn Around Time (TAT) for each process and the Average waiting time and Turn Around Time (Assuming there is no I/O waiting for the processes) for the above example if the process enters the 'Ready' queue together in the order P2, P1, P3.

The sequence of execution of the processes by the CPU is represented as



Assuming the CPU is readily available at the time of arrival of P2, P2 starts executing without any waiting in the 'Ready' queue. Hence the waiting time for P2 is zero. The waiting time for all processes is given as

Waiting Time for P2 = 0 ms (P2 starts executing first)

Waiting Time for P1 = 5 ms (P1 starts executing after completing P2)

Waiting Time for P3 = 15 ms (P3 starts executing after completing P2 and P1)

Average waiting time = (Waiting time for all processes) / No. of Processes

$$= (\text{Waiting time for } (P2+P1+P3)) / 3$$

$$= (0+5+15)/3 = 20/3$$

$$= 6.66 \text{ milliseconds}$$

Turn Around Time (TAT) for P2 = 5 ms (Time spent in Ready Queue + Execution Time)

Turn Around Time (TAT) for P1 = 15 ms (-Do-)

Turn Around Time (TAT) for P3 = 22 ms (-Do-)

Average Turn Around Time = (Turn Around Time for all processes) / No. of Processes

$$= (\text{Turn Around Time for } (P2+P1+P3)) / 3$$

$$= (5+15+22)/3 = 42/3$$

$$= 14 \text{ milliseconds}$$

The Average waiting time and Turn Around Time (TAT) depends on the order in which the processes enter the 'Ready' queue, regardless their estimated completion time.

From the above two examples it is clear that the Average waiting time and Turn Around Time improve if the process with shortest execution completion time is scheduled first.

The major drawback of FCFS algorithm is that it favours monopoly of process. A process, which does not contain any I/O operation, continues its execution until it finishes its task. If the process contains any I/O operation, the CPU is relinquished by the process. In general, FCFS favours CPU bound processes and I/O bound processes may have to wait until the completion of CPU bound process, if the currently executing process is a CPU bound process. This leads to poor device utilisation. The average waiting time is not minimal for FCFS scheduling algorithm.

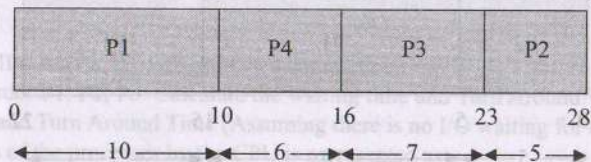
10.5.1.2 Last-Come-First Served (LCFS)/LIFO Scheduling The Last-Come-First Served (LCFS) scheduling algorithm also allocates CPU time to the processes based on the order in which they are entered in the 'Ready' queue. The last entered process is serviced first. LCFS scheduling is also known as Last In First Out (LIFO) where the process, which is put last into the 'Ready' queue, is serviced first.

Example 1

Three processes with process IDs P1, P2, P3 with estimated completion time 10, 5, 7 milliseconds respectively enters the ready queue together in the order P1, P2, P3 (Assume only P1 is present in the 'Ready' queue when the scheduler picks

it up and P2, P3 entered 'Ready' queue after that). Now a new process P4 with estimated completion time 6 ms enters the 'Ready' queue after 5 ms of scheduling P1. Calculate the waiting time and Turn Around Time (TAT) for each process and the Average waiting time and Turn Around Time (Assuming there is no I/O waiting for the processes). Assume all the processes contain only CPU operation and no I/O operations are involved.

Initially there is only P1 available in the Ready queue and the scheduling sequence will be P1, P3, P2. P4 enters the queue during the execution of P1 and becomes the last process entered the 'Ready' queue. Now the order of execution changes to P1, P4, P3, and P2 as given below.



The waiting time for all the processes is given as

Waiting Time for P1 = 0 ms (P1 starts executing first)

Waiting Time for P4 = 5 ms (P4 starts executing after completing P1. But P4 arrived after 5 ms of execution of P1. Hence its waiting time = Execution start time - Arrival Time = $10 - 5 = 5$)

Waiting Time for P3 = 16 ms (P3 starts executing after completing P1 and P4)

Waiting Time for P2 = 23 ms (P2 starts executing after completing P1, P4 and P3)

Average waiting time = (Waiting time for all processes) / No. of Processes

$$= (\text{Waiting time for } (P1+P4+P3+P2)) / 4$$

$$= (0 + 5 + 16 + 23) / 4 = 44 / 4$$

$$= 11 \text{ milliseconds}$$

Turn Around Time (TAT) for P1 = 10 ms (Time spent in Ready Queue + Execution Time)

Turn Around Time (TAT) for P4 = 11 ms (Time spent in Ready Queue + Execution Time = (Execution Start Time - Arrival Time) + Estimated Execution Time = $(10 - 5) + 6 = 5 + 6$)

Turn Around Time (TAT) for P3 = 23 ms (Time spent in Ready Queue + Execution Time)

Turn Around Time (TAT) for P2 = 28 ms (Time spent in Ready Queue + Execution Time)

Average Turn Around Time = (Turn Around Time for all processes) / No. of Processes

$$= (\text{Turn Around Time for } (P1+P4+P3+P2)) / 4$$

$$= (10+11+23+28) / 4 = 72 / 4$$

$$= 18 \text{ milliseconds}$$

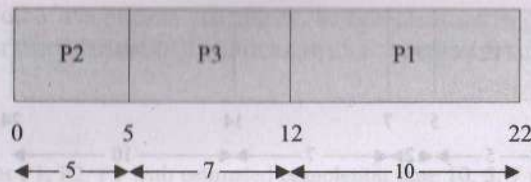
LCFS scheduling is not optimal and it also possesses the same drawback as that of FCFS algorithm.

10.5.1.3 Shortest Job First (SJF) Scheduling Shortest Job First (SJF) scheduling algorithm 'sorts the 'Ready' queue' each time a process relinquishes the CPU (either the process terminates or enters the 'Wait' state waiting for I/O or system resource) to pick the process with shortest (least) estimated completion/run time. In SJF, the process with the shortest estimated run time is scheduled first, followed by the next shortest process, and so on.

Example 1

Three processes with process IDs P1, P2, P3 with estimated completion time 10, 5, 7 milliseconds respectively enters the ready queue together. Calculate the waiting time and Turn Around Time (TAT) for each process and the Average waiting time and Turn Around Time (Assuming there is no I/O waiting for the processes) in SJF algorithm.

The scheduler sorts the 'Ready' queue based on the shortest estimated completion time and schedules the process with the least estimated completion time first and the next least one as second, and so on. The order in which the processes are scheduled for execution is represented as



The estimated execution time of P2 is the least (5 ms) followed by P3 (7 ms) and P1 (10 ms).

The waiting time for all processes are given as

Waiting Time for P2 = 0 ms (P2 starts executing first)

Waiting Time for P3 = 5 ms (P3 starts executing after completing P2)

Waiting Time for P1 = 12 ms (P1 starts executing after completing P2 and P3)

Average waiting time = (Waiting time for all processes) / No. of Processes

$$= (\text{Waiting time for (P2+P3+P1)}) / 3$$

$$= (0+5+12)/3 = 17/3$$

$$= 5.66 \text{ milliseconds}$$

Turn Around Time (TAT) for P2 = 5 ms (Time spent in Ready Queue + Execution Time)

Turn Around Time (TAT) for P3 = 12 ms (-Do-)

Turn Around Time (TAT) for P1 = 22 ms (-Do-)

Average Turn Around Time = (Turn Around Time for all processes) / No. of Processes

$$= (\text{Turn Around Time for (P2+P3+P1)}) / 3$$

$$= (5+12+22)/3 = 39/3$$

$$= 13 \text{ milliseconds}$$

Average Turn Around Time (TAT) is the sum of average waiting time and average execution time.

The average Execution time = (Execution time for all processes)/No. of processes

$$= (\text{Execution time for (P1+P2+P3)})/3$$

$$= (10+5+7)/3 = 22/3 = 7.33$$

Average Turn Around Time = Average Waiting time + Average Execution time

$$= 5.66 + 7.33$$

$$= 13 \text{ milliseconds}$$

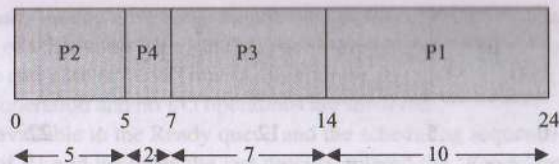
From this example, it is clear that the average waiting time and turn around time is much improved with the SJF scheduling for the same processes when compared to the FCFS algorithm.

Example 2

Calculate the waiting time and Turn Around Time (TAT) for each process and the Average waiting time and Turn Around Time for the above example if a new process P4 with estimated completion time 2 ms enters the 'Ready' queue after 2 ms of execution of P2. Assume all the processes contain only CPU operation and no I/O operations are involved.

At the beginning, there are only three processes (P1, P2 and P3) available in the 'Ready' queue and the SJF scheduler picks up the process with the least execution completion time (In this example P2 with execution completion time 5 ms) for scheduling. The execution sequence diagram for this is same as that of Example 1.

Now process P4 with estimated execution completion time 2 ms enters the 'Ready' queue after 2 ms of start of execution of P2. Since the SJF algorithm is non-preemptive and process P2 does not contain any I/O operations, P2 continues its execution. After 5 ms of scheduling, P2 terminates and now the scheduler again sorts the 'Ready' queue for process with least execution completion time. Since the execution completion time for P4 (2 ms) is less than that of P3 (7 ms), which was supposed to be run after the completion of P2 as per the 'Ready' queue available at the beginning of execution scheduling, P4 is picked up for executing. Due to the arrival of the process P4 with execution time 2 ms, the 'Ready' queue is re-sorted in the order P2, P4, P3, P1. At the beginning it was P2, P3, P1. The execution sequence now changes as per the following diagram



The waiting time for all the processes are given as

Waiting time for P2 = 0 ms (P2 starts executing first)

Waiting time for P4 = 3 ms (P4 starts executing after completing P2. But P4 arrived after 2 ms of execution of P2. Hence its waiting time = Execution start time – Arrival Time = $5 - 2 = 3$)

Waiting time for P3 = 7 ms (P3 starts executing after completing P2 and P4)

Waiting time for P1 = 14 ms (P1 starts executing after completing P2, P4 and P3)

Average waiting time = (Waiting time for all processes) / No. of Processes

$$= (\text{Waiting time for } (P2+P4+P3+P1)) / 4$$

$$= (0 + 3 + 7 + 14) / 4 = 24 / 4$$

$$= 6 \text{ milliseconds}$$

Turn Around Time (TAT) for P2 = 5 ms (Time spent in Ready Queue + Execution Time)

Turn Around Time (TAT) for P4 = 5 ms (Time spent in Ready Queue + Execution Time = (Execution Start Time – Arrival Time) + Estimated Execution Time = $(5 - 2) + 2 = 3 + 2$)

Turn Around Time (TAT) for P3 = 14 ms (Time spent in Ready Queue + Execution Time)

Turn Around Time (TAT) for P1 = 24 ms (Time spent in Ready Queue + Execution Time)

Average Turn Around Time = (Turn Around Time for all Processes) / No. of Processes

$$= (\text{Turn Around Time for } (P2+P4+P3+P1)) / 4$$

$$= (5+5+14+24) / 4 = 48 / 4$$

$$= 12 \text{ milliseconds}$$

The average waiting time for a given set of process is minimal in SJF scheduling and so it is optimal compared to other non-preemptive scheduling like FCFS. The major drawback of SJF algorithm is that a process whose estimated execution completion time is high may not get a chance to execute if more and more processes with least estimated execution time enters the 'Ready' queue before the process with longest estimated execution time started its execution (In non-preemptive SJF). This condition is known as 'Starvation'. Another drawback of SJF is that it is difficult to know in advance the next shortest process in the 'Ready' queue for scheduling since new processes with different estimated execution time keep entering the 'Ready' queue at any point of time.

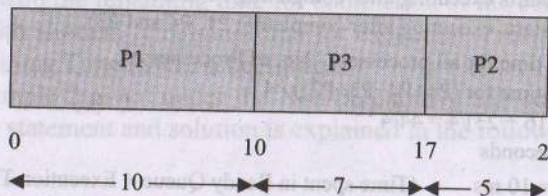
10.5.1.4 Priority Based Scheduling The Turn Around Time (TAT) and waiting time for processes in non-preemptive scheduling varies with the type of scheduling algorithm. Priority based non-preemptive scheduling algorithm ensures that a process with high priority is serviced at the earliest compared to other low priority processes in the 'Ready' queue. The priority of a task/process can be indicated through various mechanisms. The Shortest Job First (SJF) algorithm can be viewed as a priority based scheduling where each task is prioritised in the order of the time required to complete the task. The lower the time required for completing a process the higher is its priority in SJF algorithm. Another way of priority assigning is associating a priority to the task/process at the time of creation of the task/process. The priority is a number ranging from 0 to the maximum priority supported by the OS. The maximum level of priority is OS dependent. For Example, Windows CE supports 256 levels of priority (0 to 255 priority numbers). While creating the process/task, the priority can be assigned to it. The priority number associated with a task/process is the direct indication of its priority. The priority variation from high to low is represented by numbers from 0 to the maximum priority or by numbers from maximum priority to 0. For Windows CE operating system a priority number 0 indicates the highest priority and 255 indicates the lowest priority. This convention need not be universal and it depends on

the kernel level implementation of the priority structure. The non-preemptive priority based scheduler sorts the 'Ready' queue based on priority and picks the process with the highest level of priority for execution.

Example 1

Three processes with process IDs P1, P2, P3 with estimated completion time 10, 5, 7 milliseconds and priorities 0, 3, 2 (0—highest priority, 3—lowest priority) respectively enters the ready queue together. Calculate the waiting time and Turn Around Time (TAT) for each process and the Average waiting time and Turn Around Time (Assuming there is no I/O waiting for the processes) in priority based scheduling algorithm.

The scheduler sorts the 'Ready' queue based on the priority and schedules the process with the highest priority (P1 with priority number 0) first and the next high priority process (P3 with priority number 2) as second, and so on. The order in which the processes are scheduled for execution is represented as



The waiting time for all the processes are given as

Waiting time for P1 = 0 ms (P1 starts executing first)

Waiting time for P3 = 10 ms (P3 starts executing after completing P1)

Waiting time for P2 = 17 ms (P2 starts executing after completing P1 and P3)

Average waiting time = (Waiting time for all processes) / No. of Processes

= (Waiting time for (P1+P3+P2)) / 3

= (0+10+17)/3 = 27/3

= 9 milliseconds

Turn Around Time (TAT) for P1 = 10 ms (Time spent in Ready Queue + Execution Time)

Turn Around Time (TAT) for P3 = 17 ms (-Do-)

Turn Around Time (TAT) for P2 = 22 ms (-Do-)

Average Turn Around Time = (Turn Around Time for all processes) / No. of Processes

= (Turn Around Time for (P1+P3+P2)) / 3

= (10+17+22)/3 = 49/3

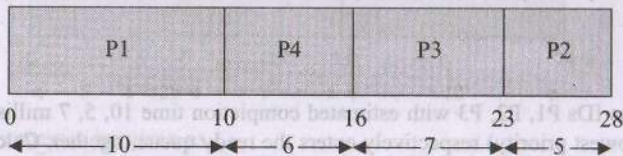
= 16.33 milliseconds

Example 2

Calculate the waiting time and Turn Around Time (TAT) for each process and the Average waiting time and Turn Around Time for the above example if a new process P4 with estimated completion time 6 ms and priority 1 enters the 'Ready' queue after 5 ms of execution of P1. Assume all the processes contain only CPU operation and no I/O operations are involved.

At the beginning, there are only three processes (P1, P2 and P3) available in the 'Ready' queue and the scheduler picks up the process with the highest priority (In this example P1 with priority 0) for scheduling. The execution sequence diagram for this is same as that of Example 1. Now process P4 with estimated execution completion time 6 ms and priority 1 enters the 'Ready' queue after 5 ms of execution of P1. Since the scheduling algorithm is non-preemptive and process P1 does not contain any I/O operations, P1 continues its execution. After 10 ms of scheduling, P1 terminates and now the scheduler again sorts the 'Ready' queue for process with highest priority. Since the priority for P4 (priority 1) is higher than that of P3 (priority 2), which was supposed to be run after the completion of P1 as per the 'Ready' queue available at the beginning of execution scheduling, P4 is picked up for executing. Due to the arrival of the process P4 with

priority 1, the 'Ready' queue is resorted in the order P1, P4, P3, P2. At the beginning it was P1, P3, P2. The execution sequence now changes as per the following diagram



The waiting time for all the processes are given as

Waiting time for P1 = 0 ms (P1 starts executing first)

Waiting time for P4 = 5 ms (P4 starts executing after completing P1. But P4 arrived after 5 ms of execution of P1. Hence its waiting time = Execution start time - Arrival Time = 10 - 5 = 5)

Waiting time for P3 = 16 ms (P3 starts executing after completing P1 and P4)

Waiting time for P2 = 23 ms (P2 starts executing after completing P1, P4 and P3)

Average waiting time = (Waiting time for all processes) / No. of Processes

= (Waiting time for (P1+P4+P3+P2)) / 4

= (0 + 5 + 16 + 23)/4 = 44/4

= 11 milliseconds

Turn Around Time (TAT) for P1 = 10 ms

(Time spent in Ready Queue + Execution Time)

Turn Around Time (TAT) for P4 = 11 ms

(Time spent in Ready Queue + Execution

Time = (Execution Start Time - Arrival Time) + Estimated Execution Time = (10 - 5) + 6 = 5 + 6)

Turn Around Time (TAT) for P3 = 23 ms

(Time spent in Ready Queue + Execution Time)

Turn Around Time (TAT) for P2 = 28 ms

(Time spent in Ready Queue + Execution Time)

Average Turn Around Time = (Turn Around Time for all processes) / No. of Processes

= (Turn Around Time for (P2 + P4 + P3 + P1)) / 4

= (10 + 11 + 23 + 28)/4 = 72/4

= 18 milliseconds

Similar to SJF scheduling algorithm, non-preemptive priority based algorithm also possess the drawback of 'Starvation' where a process whose priority is low may not get a chance to execute if more and more processes with higher priorities enter the 'Ready' queue before the process with lower priority started its execution. 'Starvation' can be effectively tackled in priority based non-preemptive scheduling by dynamically raising the priority of the low priority task/process which is under starvation (waiting in the ready queue for a longer time for getting the CPU time). The technique of gradually raising the priority of processes which are waiting in the 'Ready' queue as time progresses, for preventing 'Starvation', is known as 'Aging'.

10.5.2 Preemptive Scheduling

Preemptive scheduling is employed in systems, which implements preemptive multitasking model. In preemptive scheduling, every task in the 'Ready' queue gets a chance to execute. When and how often each process gets a chance to execute (gets the CPU time) is dependent on the type of preemptive scheduling algorithm used for scheduling the processes. In this kind of scheduling, the scheduler can preempt (stop temporarily) the currently executing task/process and select another task from the 'Ready' queue for execution. When to pre-empt a task and which task is to be picked up from the 'Ready' queue for execution after preempting the current task is purely dependent on the scheduling algorithm. A task which is preempted by the scheduler is moved to the 'Ready' queue. The act of moving a 'Running' process/task into the 'Ready' queue by the scheduler, without the processes requesting for it is known as

'Preemption'. Preemptive scheduling can be implemented in different approaches. The two important approaches adopted in preemptive scheduling are time-based preemption and priority-based preemption. The various types of preemptive scheduling adopted in task/process scheduling are explained below.

10.5.2.1 Preemptive SJF Scheduling/Shortest Remaining Time (SRT) The non-preemptive SJF scheduling algorithm sorts the 'Ready' queue only after completing the execution of the current process or when the process enters 'Wait' state, whereas the preemptive SJF scheduling algorithm sorts the 'Ready' queue when a new process enters the 'Ready' queue and checks whether the execution time of the new process is shorter than the remaining of the total estimated time for the currently executing process. If the execution time of the new process is less, the currently executing process is preempted and the new process is scheduled for execution. Thus preemptive SJF scheduling always compares the execution completion time (It is same as the remaining time for the new process) of a new process entered the 'Ready' queue with the remaining time for completion of the currently executing process and schedules the process with shortest remaining time for execution. Preemptive SJF scheduling is also known as Shortest Remaining Time (SRT) scheduling.

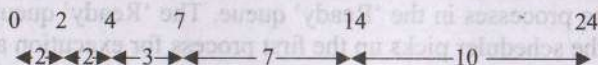
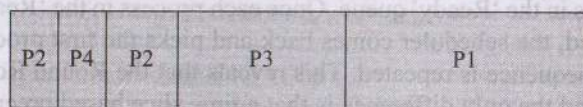
Now let us solve Example 2 given under the Non-preemptive SJF scheduling for preemptive SJF scheduling. The problem statement and solution is explained in the following example.

Example 1

Three processes with process IDs P1, P2, P3 with estimated completion time 10, 5, 7 milliseconds respectively enters the ready queue together. A new process P4 with estimated completion time 2 ms enters the 'Ready' queue after 2 ms. Assume all the processes contain only CPU operation and no I/O operations are involved.

At the beginning, there are only three processes (P1, P2 and P3) available in the 'Ready' queue and the SRT scheduler picks up the process with the shortest remaining time for execution completion (In this example, P2 with remaining time 5 ms) for scheduling. The execution sequence diagram for this is same as that of example 1 under non-preemptive SJF scheduling.

Now process P4 with estimated execution completion time 2 ms enters the 'Ready' queue after 2 ms of start of execution of P2. Since the SRT algorithm is preemptive, the remaining time for completion of process P2 is checked with the remaining time for completion of process P4. The remaining time for completion of P2 is 3 ms which is greater than that of the remaining time for completion of the newly entered process P4 (2 ms). Hence P2 is preempted and P4 is scheduled for execution. P4 continues its execution to finish since there is no new process entered in the 'Ready' queue during its execution. After 2 ms of scheduling P4 terminates and now the scheduler again sorts the 'Ready' queue based on the remaining time for completion of the processes present in the 'Ready' queue. Since the remaining time for P2 (3 ms), which is preempted by P4 is less than that of the remaining time for other processes in the 'Ready' queue, P2 is scheduled for execution. Due to the arrival of the process P4 with execution time 2 ms, the 'Ready' queue is re-sorted in the order P2, P4, P2, P3, P1. At the beginning it was P2, P3, P1. The execution sequence now changes as per the following diagram



The waiting time for all the processes are given as

Waiting time for P2 = 0 ms + (4 - 2) ms = 2 ms (P2 starts executing first and is interrupted by P4 and has to wait till the completion of P4 to get the next CPU slot)

Waiting time for P4 = 0 ms (P4 starts executing by preempting P2 since the execution time for completion of P4 (2 ms) is less than that of the Remaining time for execution completion of P2 (Here it is 3 ms))

Waiting time for P3 = 7 ms (P3 starts executing after completing P4 and P2)

Waiting time for P1 = 14 ms (P1 starts executing after completing P4, P2 and P3)

Average waiting time = (Waiting time for all the processes) / No. of Processes
 = (Waiting time for (P4+P2+P3+P1)) / 4
 = (0 + 2 + 7 + 14) / 4 = 23/4
 = 5.75 milliseconds

Turn Around Time (TAT) for P2 = 7 ms (Time spent in Ready Queue + Execution Time)

Turn Around Time (TAT) for P4 = 2 ms (Time spent in Ready Queue + Execution Time = (Execution Start Time - Arrival Time) + Estimated Execution Time = (2 - 2) + 2)

Turn Around Time (TAT) for P3 = 14 ms (Time spent in Ready Queue + Execution Time)

Turn Around Time (TAT) for P1 = 24 ms (Time spent in Ready Queue + Execution Time)

Average Turn Around Time = (Turn Around Time for all the processes) / No. of Processes
 = (Turn Around Time for (P2+P4+P3+P1)) / 4
 = (7+2+14+24) / 4 = 47/4
 = 11.75 milliseconds

Now let's compare the Average Waiting time and Average Turn Around Time with that of the Average waiting time and Average Turn Around Time for non-preemptive SJF scheduling (Refer to Example 2 given under the section Non-preemptive SJF scheduling)

Average Waiting Time in non-preemptive SJF scheduling = 6 ms

Average Waiting Time in preemptive SJF scheduling = 5.75 ms

Average Turn Around Time in non-preemptive SJF scheduling = 12 ms

Average Turn Around Time in preemptive SJF scheduling = 11.75 ms

This reveals that the Average waiting Time and Turn Around Time (TAT) improves significantly with preemptive SJF scheduling.

10.5.2.2 Round Robin (RR) Scheduling The term *Round Robin* is very popular among the sports and games activities. You might have heard about 'Round Robin' league or 'Knock out' league associated with any football or cricket tournament. In the 'Round Robin' league each team in a group gets an equal chance to play against the rest of the teams in the same group whereas in the 'Knock out' league the losing team in a match moves out of the tournament ☺.

In the process scheduling context also, '*Round Robin*' brings the same message "Equal chance to all". In Round Robin scheduling, each process in the 'Ready' queue is executed for a pre-defined time slot. The execution starts with picking up the first process in the 'Ready' queue (see Fig. 10.13). It is executed for a pre-defined time and when the pre-defined time elapses or the process completes (before the pre-defined time slice), the next process in the 'Ready' queue is selected for execution. This is repeated for all the processes in the 'Ready' queue. Once each process in the 'Ready' queue is executed for the pre-defined time period, the scheduler comes back and picks the first process in the 'Ready' queue again for execution. The sequence is repeated. This reveals that the Round Robin scheduling is similar to the FCFS scheduling and the only difference is that a time slice based preemption is added to switch the execution between the processes in the 'Ready' queue. The 'Ready' queue can be considered as a circular queue in which the scheduler picks up the first process for execution and moves to the next till the end of the queue and then comes back to the beginning of the queue to pick up the first process.

The time slice is provided by the *timer tick* feature of the time management unit of the OS kernel (Refer the Time management section under the subtopic '*The Real-Time kernel*' for more details on Timer tick). Time slice is kernel dependent and it varies in the order of a few microseconds to milliseconds. Certain OS kernels may allow the time slice as user configurable. Round Robin scheduling ensures that

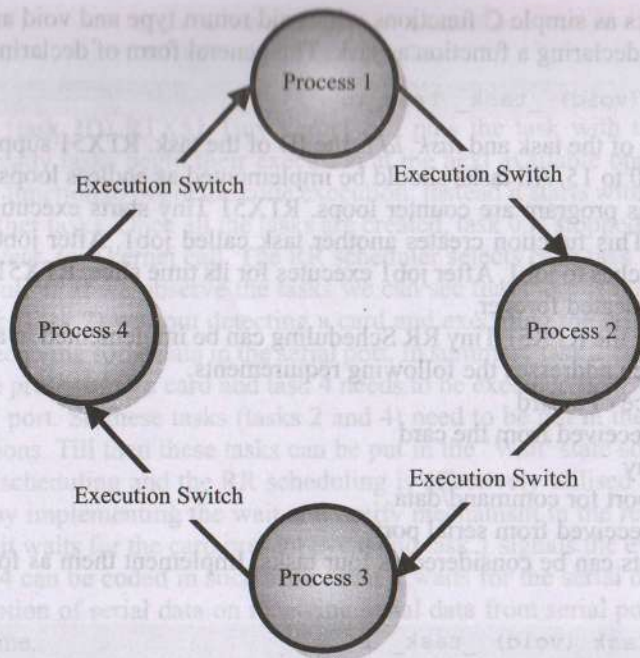


Fig. 10.13 Round Robin Scheduling

every process gets a fixed amount of CPU time for execution. When the process gets its fixed time for execution is determined by the FCFS policy (That is, a process entering the Ready queue first gets its fixed execution time first and so on...). If a process terminates before the elapse of the time slice, the process releases the CPU voluntarily and the next process in the queue is scheduled for execution by the scheduler. The implementation of RR scheduling is kernel dependent. The following code snippet illustrates the RR scheduling implementation for RTX51 Tiny OS, an 8bit OS for 8051 microcontroller from Keil Software (www.keil.com), an ARM® Company.

```

#include <rtx51tny.h>      /* Definitions for RTX51 Tiny */
int counter0;
int counter1;

job0 () _task_ 0 {
    os_create_task (1);    /* Mark task 1 as "ready" */

    while (1) {            /* Endless loop */
        counter0++;        /* Increment counter 0 */
    }
}

job1 () _task_ 1 {
    while (1) {            /* Endless loop */
        counter1++;        /* Increment counter 1 */
    }
}

```


RTX51 defines the tasks as simple C functions with void return type and void argument list. The attribute `_task_` is used for declaring a function as task. The general form of declaring a task is

```
void func (void) _task_ task_id
```

where *func* is the name of the task and *task_id* is the ID of the task. RTX51 supports up to 16 tasks and so *task_id* varies from 0 to 15. All tasks should be implemented as endless loops.

The two tasks in this program are counter loops. RTX51 Tiny starts executing task 0 which is the function named `job0`. This function creates another task called `job1`. After `job0` executes for its time slice, RTX51 Tiny switches to `job1`. After `job1` executes for its time slice, RTX51 Tiny switches back to `job0`. This process is repeated forever.

Now let's check how the RTX51 Tiny RR Scheduling can be implemented in an embedded device (A smart card reader) which addresses the following requirements.

- Check the presence of a card
- Process the data received from the card
- Update the Display
- Check the serial port for command/data
- Process the data received from serial port

These four requirements can be considered as four tasks. Implement them as four RTX51 tasks as explained below.

```
void check_card_task (void) _task_ 1
{
/* This task checks for the presence of a card */
/* Implement the necessary functionality here */
}

void process_card_task (void) _task_ 2
{
/* This task processes the data received from the card */
/* Implement the necessary functionality here */
}

void check_serial_io_task (void) _task_ 3
{
/* This task checks for serial I/O */
/* Implement the necessary functionality here */
}

void process_serial_data_task (void) _task_ 4
{
/* This task processes the data received from the serial port */
/* Implement the necessary functionality here */
}
```

Now the tasks are created. Next step is scheduling the tasks. The following code snippet illustrates the scheduling of tasks.

```
void startup_task (void) _task_ 0
{
os_create_task (1);    /* Create check_card_task Task */
os_create_task (2);    /* Create process_card_task Task */
os_create_task (3);    /* Create serial_io_task Task */
```



```

os_create_task (4);      /* Create serial_data_task Task */
os_delete_task (0);      /* Delete the Startup Task */
)

```

The `os_create_task (task_ID)` RTX51 Tiny kernel call puts the task with task ID `task_ID` in the 'Ready' state. All the ready tasks begin their execution at the next available opportunity. RTX51 Tiny does not have a `main ()` function to begin the code execution; instead it starts with executing task 0. Task 0 is used for creating other tasks. Once all the tasks are created, task 0 is stopped and removed from the task list with the `os_delete_task` kernel call. The RR scheduler selects each task based on the time slice and continues the execution. If we observe the tasks we can see that there is no point in executing the task `process_card_task` (Task 2) without detecting a card and executing the task `process_serial_data_task` (Task 4) without receiving some data in the serial port. In summary task 2 needs to be executed only when task 1 reports the presence of a card and task 4 needs to be executed only when task 3 reports the arrival of data at serial port. So these tasks (tasks 2 and 4) need to be put in the 'Ready' state only on satisfying these conditions. Till then these tasks can be put in the 'Wait' state so that the RR scheduler will not pick them for scheduling and the RR scheduling is effectively utilised among the other tasks. This can be achieved by implementing the wait and notify mechanism in the related tasks. Task 2 can be coded in a way that it waits for the card present event and task 1 signals the event 'card detected'. In a similar fashion Task 4 can be coded in such a way that it waits for the serial data received event and task 3 signals the reception of serial data on receiving serial data from serial port. The following code snippet explains the same.

```

void check_card_task (void) _task_1
{
/* This task checks for the presence of a card */
/* Implement the necessary functionality here */
while (1)
{
//Function for checking the presence of card and card reading
//.....
if (card is present)
//Signal card detected to task 2
os_send_signal (2)
}
}

```

```

void process_card_task (void) _task_2
{
/* This task processes the data received from the card */
/* Implement the necessary functionality here */
while (1)
{
//Function for checking the signaling of card present event
os_wait1(K_SIG);
//Process card data
}
}

```



```

void check_serial_io_task (void) _task_ 3
{
/* This task checks for serial I/O */
/* Implement the necessary functionality here */
while (1)
{
//Function for checking the reception of serial data
//.....
if (data is received)
//Signal serial data reception to task 4
os_send_signal (4)
}
}

void process_serial_data_task (void) _task_ 4
{
/* This task processes the data received from the serial port */
/* Implement the necessary functionality here */
while (1)
{
//Function for checking the signaling of serial data received event
os_wait1(K_SIG);
//Process card data
}
}

```

The `os_send_signal(Task ID)` kernel call sends a signal to task *Task ID*. If the specified task is already waiting for a signal, this function call readies the task for execution but does not start it. The `os_wait1(event)` kernel call halts the current task and waits for an event to occur. The *event* argument specifies the event to wait for and may have only the value `K_SIG` which waits for a signal. RTX51 uses the Timer 0 of 8051 for time slice generation. The time slice can be configured by the user by changing the time slice related parameters in the RTX51 Tiny OS configuration file **CONF_TNY.A51** file which is located in the **(KEIL\C51\RTXTINY2\)** folder. Configuration options in **CONF_TNY.A51** allow users to:

- Specify the Timer Tick Interrupt Register Bank.
- Specify the Timer Tick Interval (in 8051 machine cycles).
- Specify user code to execute in the Timer Tick Interrupt.
- Specify the Round-Robin Timeout.
- Enable or disable Round-Robin Task Switching.
- Specify that your application includes long duration interrupts.
- Specify whether or not code banking is used.
- Define the top of the RTX51 Tiny stack.
- Specify the minimum stack space required.
- Specify code to execute in the event of a stack error.
- Define idle task operations.

The RTX51 kernel provides a set of task management functions for managing the tasks. At any point of time each RTX51 task is exactly in any one of the following state.

Task State	State Description
RUNNING	The task that is currently running is in the RUNNING State. Only one task at a time may be in this state. The <i>os_running_task_id</i> kernel call returns the task number (ID) of the currently executing task.
READY	Tasks which are ready to run are in the READY State. Once the Running task has completed processing, RTX51 Tiny selects and starts the next Ready task. A task may be made ready immediately (even if the task is waiting for a timeout or signal) by setting its ready flag using the <i>os_set_ready</i> or <i>isr_set_ready</i> kernel functions.
WAITING	Tasks which are waiting for an event are in the WAITING State. Once the event occurs, the task is switched to the READY State. The <i>os_wait_function</i> is used for placing a task in the WAITING State.
DELETED	Tasks which have not been started or tasks which have been deleted are in the DELETED State. The <i>os_delete_task</i> routine places a task that has been started (with <i>os_create_task</i>) into the DELETED State.
TIME - OUT	Tasks which were interrupted by a Round-Robin Time-Out are in the TIME-OUT State. This state is equivalent to the READY State for Round-Robin programs.

Refer the documentation available with RTX51 Tiny OS for more information on the various RTX51 task management kernel functions and their usage.

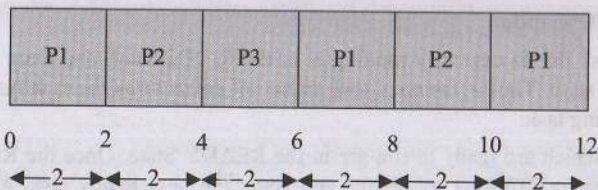
RR scheduling with interrupts is a good choice for the design of comparatively less complex *Real-Time Embedded Systems*. In this approach, the tasks which require less *Real-Time* attention can be scheduled with Round Robin scheduling and the tasks which require *Real-Time* attention can be scheduled through Interrupt Service Routines. RTX51 Tiny supports Interrupts with RR scheduling. For RTX51 the time slice for RR scheduling is provided by the Timer interrupt and if the interrupt is of high priority than that of the timer interrupt and if its service time (ISR) is longer than the timer tick interval, the RTX51 timer interrupt may be interrupted by the ISR and it may be reentered by a subsequent RX51 Tiny timer interrupt. Hence proper care must be taken to limit the ISR time within the timer tick interval or to protect the timer tick interrupt code from reentrancy. Otherwise unexpected results may occur. The limitations of RR with interrupt generic approach are the limited number of interrupts supported by embedded processors and the interrupt latency happening due to the context switching overhead.

RR can also be used as technique for resolving the priority in scheduling among the tasks with same level of priority. We will discuss about how RR scheduling can be used for resolving the priority among equal tasks under the VxWorks kernel in a later chapter.

Example 1

Three processes with process IDs P1, P2, P3 with estimated completion time 6, 4, 2 milliseconds respectively, enters the ready queue together in the order P1, P2, P3. Calculate the waiting time and Turn Around Time (TAT) for each process and the Average waiting time and Turn Around Time (Assuming there is no I/O waiting for the processes) in RR algorithm with Time slice = 2 ms.

The scheduler sorts the 'Ready' queue based on the FCFS policy and picks up the first process P1 from the 'Ready' queue and executes it for the time slice 2 ms. When the time slice is expired, P1 is preempted and P2 is scheduled for execution. The Time slice expires after 2ms of execution of P2. Now P2 is preempted and P3 is picked up for execution. P3 completes its execution within the time slice and the scheduler picks P1 again for execution for the next time slice. This procedure is repeated till all the processes are serviced. The order in which the processes are scheduled for execution is represented as



The waiting time for all the processes are given as

Waiting time for P1 = $0 + (6 - 2) + (10 - 8) = 0 + 4 + 2 = 6$ ms

(P1 starts executing first and waits for two time slices to get execution back and again 1 time slice for getting CPU time)

Waiting time for P2 = $(2 - 0) + (8 - 4) = 2 + 4 = 6$ ms

(P2 starts executing after P1 executes for 1 time slice and waits for two time slices to get the CPU time)

Waiting time for P3 = $(4 - 0) = 4$ ms

(P3 starts executing after completing the first time slices for P1 and P2 and completes its execution in a single time slice)

Average waiting time = (Waiting time for all the processes) / No. of Processes

= (Waiting time for (P1 + P2 + P3)) / 3

= $(6 + 6 + 4) / 3 = 16/3$

= 5.33 milliseconds

Turn Around Time (TAT) for P1 = 12 ms

(Time spent in Ready Queue + Execution Time)

Turn Around Time (TAT) for P2 = 10 ms

(-Do-)

Turn Around Time (TAT) for P3 = 6 ms

(-Do-)

Average Turn Around Time = (Turn Around Time for all the processes) / No. of Processes

= (Turn Around Time for (P1 + P2 + P3)) / 3

= $(12 + 10 + 6) / 3 = 28/3$

= 9.33 milliseconds

Average Turn Around Time (TAT) is the sum of average waiting time and average execution time.

Average Execution time = (Execution time for all the process) / No. of processes

= (Execution time for (P1 + P2 + P3)) / 3

= $(6 + 4 + 2) / 3 = 12/3$

= 4

Average Turn Around Time = Average Waiting time + Average Execution time

= $5.33 + 4$

= 9.33 milliseconds

RR scheduling involves lot of overhead in maintaining the time slice information for every process which is currently being executed.

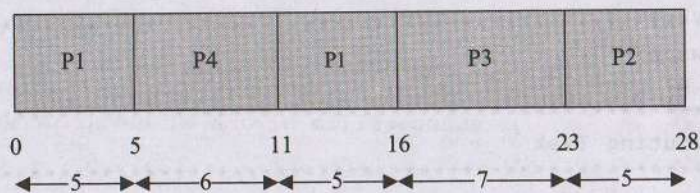
10.5.2.3 Priority Based Scheduling Priority based preemptive scheduling algorithm is same as that of the non-preemptive priority based scheduling except for the switching of execution between tasks. In preemptive scheduling, any high priority process entering the 'Ready' queue is immediately scheduled for execution whereas in the non-preemptive scheduling any high priority process entering the 'Ready' queue is scheduled only after the currently executing process completes its execution or only when it voluntarily relinquishes the CPU. The priority of a task/process in preemptive scheduling is indicated in the same way as that of the mechanism adopted for non-preemptive multitasking. Refer the non-preemptive priority based scheduling discussed in an earlier section of this chapter for more details.

Example 1

Three processes with process IDs P1, P2, P3 with estimated completion time 10, 5, 7 milliseconds and priorities 1, 3, 2 (0—highest priority, 3—lowest priority) respectively enters the ready queue together. A new process P4 with estimated completion time 6 ms and priority 0 enters the 'Ready' queue after 5 ms of start of execution of P1. Assume all the processes contain only CPU operation and no I/O operations are involved.

At the beginning, there are only three processes (P1, P2 and P3) available in the 'Ready' queue and the scheduler picks up the process with the highest priority (In this example P1 with priority 1) for scheduling.

Now process P4 with estimated execution completion time 6 ms and priority 0 enters the 'Ready' queue after 5 ms of start of execution of P1. Since the scheduling algorithm is preemptive, P1 is preempted by P4 and P4 runs to completion. After 6 ms of scheduling, P4 terminates and now the scheduler again sorts the 'Ready' queue for process with highest priority. Since the priority for P1 (priority 1), which is preempted by P4 is higher than that of P3 (priority 2) and P2 ((priority 3), P1 is again picked up for execution by the scheduler. Due to the arrival of the process P4 with priority 0, the 'Ready' queue is resorted in the order P1, P4, P1, P3, P2. At the beginning it was P1, P3, P2. The execution sequence now changes as per the following diagram



The waiting time for all the processes are given as

Waiting time for P1 = $0 + (11 - 5) = 0 + 6 = 6$ ms

(P1 starts executing first and gets preempted by P4 after 5 ms and again gets the CPU time after completion of P4)

Waiting time for P4 = 0 ms

(P4 starts executing immediately on entering the 'Ready' queue, by preempting P1)

Waiting time for P3 = 16 ms (P3 starts executing after completing P1 and P4)

Waiting time for P2 = 23 ms (P2 starts executing after completing P1, P4 and P3)

Average waiting time = (Waiting time for all the processes) / No. of Processes

= (Waiting time for (P1+P4+P3+P2)) / 4

= $(6 + 0 + 16 + 23) / 4 = 45 / 4$

= 11.25 milliseconds

Turn Around Time (TAT) for P1 = 16 ms (Time spent in Ready Queue + Execution Time)

Turn Around Time (TAT) for P4 = 6 ms

(Time spent in Ready Queue + Execution Time = (Execution Start Time - Arrival Time) + Estimated Execution Time = $(5 - 5) + 6 = 0 + 6$)

Turn Around Time (TAT) for P3 = 23 ms (Time spent in Ready Queue + Execution Time)

Turn Around Time (TAT) for P2 = 28 ms (Time spent in Ready Queue + Execution Time)

Average Turn Around Time = (Turn Around Time for all the processes) / No. of Processes

= (Turn Around Time for (P2 + P4 + P3 + P1)) / 4

= $(16 + 6 + 23 + 28) / 4 = 73 / 4$

= 18.25 milliseconds

ention to high priority tasks. Thus priority demands 'Real-Time' behaviour. Most of scheduling algorithm for process scheduling, same drawback of non-preemptive priority is 'Aging' technique. Refer the section Non-tarvation' and 'Aging'.

SCHEDULING: PUTTING

read scheduling. Now let us have a look at tion. Let's examine the following pieces of

seconds

sk,

```

return;
}
WaitForSingleObject(hThread, INFINITE);
return;
}
//*****
//Process 2
//*****
#include <windows.h>
#include <stdio.h>
//*****
//Thread for executing Task
//*****
void Task(void) {
while (1)
{
//Perform some task
//Task execution time is 10 units of execution
//Sleep for 5 units of execution
Sleep(5); //Parameter given is not in milliseconds
//Repeat task
}
}
//*****
//Main Thread.
//*****
void main(void) {
DWORD id;
HANDLE hThread;
//Create thread with above normal priority
//*****
hThread = CreateThread(NULL, 0, Task,
(LPVOID) 0, CREATE_NEW, &id);
if (NULL==hThread)
{//Thread Creation failed. Exit the process
printf("Creating thread failed\n", GetLastError());
return;
}
SetThreadPriority(hThread, THREAD_PRIORITY_NORMAL);
ResumeThread(hThread);
WaitForSingleObject(hThread, INFINITE);
return;
}

```

The first piece of code represents a process


```

    return;
}
WaitForSingleObject(hThread, INFINITE);
return;
}
//*****
//Process 2
//*****
#include <windows.h>
#include <stdio.h>
//*****
//Thread for executing Task
//*****
void Task(void) {
while (1)
{
//Perform some task
//Task execution time is 10 units of execution
//Sleep for 5 units of execution
Sleep(5); //Parameter given is not in milliseconds
//Repeat task
}
}
//*****
//Main Thread.
//*****
void main(void) {
    DWORD id;
    HANDLE hThread;
//Create thread with above normal priority
//*****
    hThread = CreateThread(NULL, 0,
                          (LPTHREAD_START_ROUTINE)Task,
                          (LPVOID) 0, CREATE_SUSPENDED, &id);
    if (NULL==hThread)
    {
//Thread Creation failed. Exit process
printf("Creating thread failed: Error Code =
%d", GetLastError());
return;
}
SetThreadPriority(hThread, THREAD_PRIORITY_ABOVE_NORMAL);
ResumeThread(hThread);
WaitForSingleObject(hThread, INFINITE);
return;
}

```

The first piece of code represents a process (Process 1) with priority normal and it performs a task which requires 7.5 units of execution time. After performing this task, the process sleeps for 17.5 units of execution time and this is repeated forever. The second piece of code represents a process (Process

2) with priority above normal and it performs a task which requires 10 units of execution time. After performing this task, the process sleeps for 5 units of execution time and this is repeated forever. Process 2 is of higher priority compared to process 1, since its priority is above 'Normal'.

Now let us examine what happens if these processes are executed on a Real-Time kernel with pre-emptive priority based scheduling policy. Imagine Process 1 and Process 2 are ready for execution. Both of them enters the 'Ready' queue and the scheduler picks up Process 2 for execution since it is of higher priority (Assuming there is no other process running/ready for execution, when both the processes are 'Ready' for execution) compared to Process 1. Process 2 starts executing and runs until it executes the Sleep instruction (i.e. after 10 units of execution time). When the Sleep instruction is executed, Process 2 enters the wait state. Since Process 1 is waiting for its turn in the 'Ready' queue, the scheduler picks up it for execution, resulting in a context switch. The Process Control Block (PCB) of Process 2 is updated with the values of the Program Counter (PC), stack pointer, etc. at the time of context switch. The estimated task execution time for Process 1 is 7.5 units of execution time and the sleeping time for Process 2 is 5 units of execution time. After 5 units of execution time, Process 2 enters the 'Ready' state and moves to the 'Ready' queue. Since it is of higher priority compared to the running process, the running process (Process 1) is pre-empted and Process 2 is scheduled for execution. Process 1 is moved to the 'Ready' queue, resulting in context switching. The Process Control Block of Process 1 is updated with the current values of the Program Counter (PC), Stack pointer, etc. when the context switch is happened. The Program Counter (PC), Stack pointer, etc. for Process 2 is loaded with the values stored in the Process Control Block (PCB) of Process 2 and Process 2 continues its execution from where it was stopped earlier. Process 2 executes the Sleep instruction after 10 units of execution time and enters the wait state. At this point Process 1 is waiting in the 'Ready' queue and it requires 2.5 units of execution time for completing the task associated with it (The total time for completing the task is 7.5 units of time, out of this it has already completed 5 units of execution when Process 2 was in the wait state). The scheduler schedules Process 1 for execution. The Program Counter (PC), Stack pointer, etc. for Process 1 is loaded with the values stored in the Process Control Block (PCB) of Process 1 and Process 1 continues its execution from where it was stopped earlier. After 2.5 units of execution time, Process 1 executes the Sleep instruction and enters the wait state. Process 2 is already in the wait state and the scheduler finds no other process for scheduling. In order to keep the CPU always busy, the scheduler runs a dummy process (task) called '*IDLE PROCESS (TASK)*'. The '*IDLE PROCESS (TASK)*' executes some dummy task and keeps the CPU engaged. The execution diagram depicted in Fig. 10.24 explains the sequence of operations.

The implementation of the '*IDLE PROCESS (TASK)*' is dependent on the kernel and a typical implementation for a desktop OS may look like. It is simply an endless loop.

```
void Idle_Process (void)
{
    //Simply wait.
    //Do nothing...
    While (1);
}
```

The Real-Time kernels deployed in embedded systems, where operating power is a big constraint (like systems which are battery powered); the '*IDLE TASK*' is used for putting the CPU into *IDLE* mode for saving the power. A typical example is the RTX51 Tiny Real-Time kernel, where the '*IDLE TASK*' sets the 8051 CPU to *IDLE* mode, a power saving mode. In the '*IDLE*' mode, the program execution is

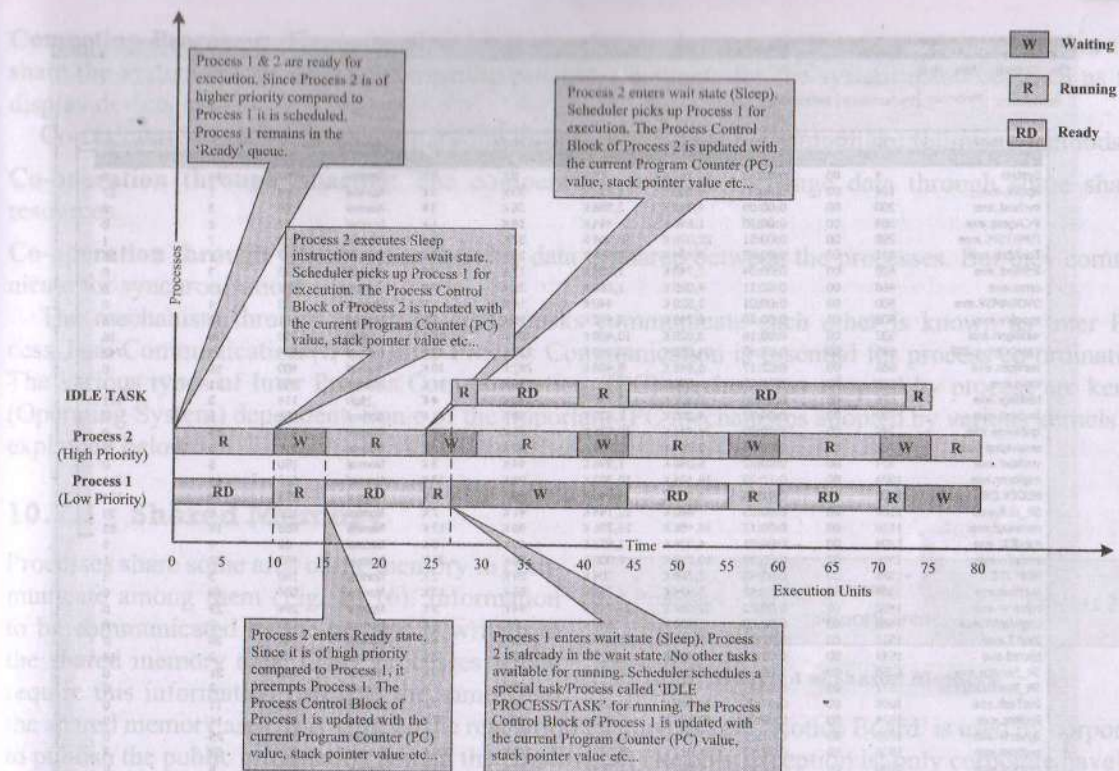


Fig. 10.14 Process scheduling and context switch

halted and all peripherals and the interrupt system continues its operation. Once the CPU is put into the 'IDLE' mode, it comes out of this mode when an Interrupt occurs or when the RTX51 Tiny Timer Tick Interrupt (The timer interrupt used for task scheduling in Round robin scheduling) occurs. It should be noted that the 'IDLE PROCESS (TASK)' execution is not pre-emptive priority scheduling specific, it is applicable to all types of scheduling policies which demand 100% CPU utilisation/CPU power saving.

Back to the desktop OS environment, let's analyse the process, threads and scheduling in the Windows desktop environment. Windows provide a utility called *task manager* for monitoring the different process running on the system and the resources used by each process. A snapshot of the process details returned by the task manager for Windows XP kernel is shown in Fig. 10.15. It should be noted that this snapshot is purely machine dependent and it varies with the number of processes running on the machine.

'Image Name' represents the name of the process. 'PID' represents the Process Identification Number (Process ID). As mentioned in the 'Threads and Process' section, when a process is created an ID is associated to it. CPU usage gives the % of CPU utilised by the process during an interval. 'CPU Time' gives the total CPU time used by a process after its commencement. 'Mem Usage' represents the total main memory, in kilobytes, used by the process. 'VM Size' represents the total virtual memory (paged memory), in kilobytes, used by a process. 'Paged Pool' represents the paged memory, in kilobytes, currently used by the system. 'NP Pool' is the non-paged pool or system memory used by a process. The non-paged memory is not swapped to the secondary storage disk. 'Base Pri' represents the priority of

Windows Task Manager

File Options View Help

Applications Processes Performance Networking

Image Name	PID	CPU	CPU Time	Mem Usage	VM Size	Private Bytes	NP Pool	Base Pri	Handles	Threads	USER Objects
System Idle Process	0	94	240:33:00	16 K	0 K	0 K	0 K	N/A	0	0	0
System	4	00	0:07:53	212 K	28 K	0 K	0 K	Normal	797	64	0
Apoin.exe	164	00	0:00:33	6,500 K	1,732 K	35 K	3 K	Normal	69	1	27
svchost.exe	200	00	0:00:09	4,012 K	2,556 K	36 K	3 K	Normal	89	3	0
IPCAgent.exe	284	00	0:00:00	1,648 K	444 K	18 K	1 K	Normal	37	2	0
lcmrsv.exe	292	00	0:00:51	22,076 K	18,564 K	32 K	4 K	Normal	143	7	1
TosA2dp.exe	312	00	0:00:05	3,404 K	2,668 K	23 K	2 K	Normal	47	1	28
lc98mit.exe	420	00	0:00:04	2,748 K	1,236 K	17 K	3 K	Normal	73	3	0
csrss.exe	468	00	0:02:11	4,028 K	1,868 K	75 K	7 K	High	997	13	0
DVDROMSV.exe	500	00	0:00:01	1,528 K	440 K	14 K	1 K	Normal	33	4	0
spoolsv.exe	508	00	0:00:10	6,748 K	4,432 K	45 K	5 K	Normal	161	10	0
winlogon.exe	532	00	0:00:16	3,828 K	10,468 K	84 K	68 K	High	750	36	16
YAHOO~1.EXE	648	00	0:07:48	54,852 K	53,348 K	128 K	22 K	Normal	3,484	18	742
services.exe	656	00	0:02:17	6,648 K	5,408 K	262 K	10 K	Normal	400	16	0
lsass.exe	668	00	0:00:15	1,596 K	1,960 K	56 K	11 K	Normal	599	21	1
taskmgr.exe	676	05	0:00:30	7,372 K	2,196 K	43 K	4 K	High	114	3	121
svchost.exe	732	00	0:00:02	4,280 K	2,020 K	38 K	11 K	Normal	104	8	0
igateway.exe	740	00	0:00:05	24,444 K	39,976 K	76 K	37 K	Normal	1,301	13	0
smacthlp.exe	752	00	0:00:04	3,764 K	2,248 K	32 K	2 K	Normal	116	3	5
svchost.exe	924	00	0:00:02	5,048 K	1,996 K	44 K	3 K	Normal	150	5	0
explorer.exe	1024	00	0:12:28	16,104 K	18,392 K	98 K	18 K	Normal	616	14	312
MSDEV.EXE	1100	00	0:01:16	4,804 K	13,640 K	68 K	8 K	Normal	253	9	174
SR_GUI.exe	1114	00	0:00:03	880 K	12,144 K	41 K	7 K	Normal	139	8	33
msnmsgr.exe	1120	00	0:00:17	16,456 K	19,736 K	98 K	17 K	Normal	559	15	63
rundll32.exe	1204	00	0:00:01	4,036 K	2,652 K	40 K	3 K	Normal	86	4	5
inetinfo.exe	1316	02	0:05:39	10,212 K	6,000 K	56 K	34 K	Normal	492	27	1
MDM.EXE	1376	00	0:03:45	2,768 K	704 K	24 K	2 K	Normal	130	3	3
svchost.exe	1384	00	0:00:43	5,924 K	2,792 K	56 K	13 K	Normal	564	10	0
sqlservr.exe	1432	00	0:00:22	22,088 K	29,748 K	44 K	9 K	Normal	294	29	0
LogWatch.exe	1488	00	0:00:00	1,496 K	664 K	9 K	2 K	Normal	23	2	0
InoT.exe	1512	00	0:01:23	23,468 K	19,804 K	26 K	4 K	Normal	146	20	0
hcmd.exe	1544	00	0:00:02	2,428 K	672 K	22 K	2 K	Normal	87	2	14
SR_Service.exe	1556	00	0:06:22	2,848 K	17,296 K	58 K	19 K	Normal	363	21	

Fig. 10.15 Windows XP task manager for monitoring process and resource usage

the process. As mentioned in an earlier section, a process may contain multiple threads. The '*Threads*' section gives the number of threads present in a process. '*Handles*' reflects the number of object handles owned by the process. This value is the reflection of the object handles present in the process's object table. '*User Objects*' reflects the number of objects active in the user mode for a process. Use '*Ctrl*' + '*Alt*' + '*Del*' key for accessing the task manager and select the '*View*' → '*Select Columns*' option to select the different monitoring parameters for a process.

10.7 TASK COMMUNICATION

In a multitasking system, multiple tasks/processes run concurrently (in pseudo parallelism) and each process may or may not interact between. Based on the degree of interaction, the processes running on an OS are classified as

Co-operating Processes: In the co-operating interaction model one process requires the inputs from other processes to complete its execution.

Competing Processes: The competing processes do not share anything among themselves but they share the system resources. The competing processes compete for the system resources such as file, display device, etc.

Co-operating processes exchange information and communicate through the following methods.

Co-operation through Sharing: The co-operating process exchange data through some shared resources.

Co-operation through Communication: No data is shared between the processes. But they communicate for synchronisation.

The mechanism through which processes/tasks communicate each other is known as Inter Process/Task Communication (IPC). Inter Process Communication is essential for process co-ordination. The various types of Inter Process Communication (IPC) mechanisms adopted by process are kernel (Operating System) dependent. Some of the important IPC mechanisms adopted by various kernels are explained below.

10.7.1 Shared Memory

Processes share some area of the memory to communicate among them (Fig. 10.16). Information to be communicated by the process is written to the shared memory area. Other processes which require this information can read the same from the shared memory area. It is same as the real world example where 'Notice Board' is used by corporate to publish the public information among the employees (The only exception is; only corporate have the right to modify the information published on the Notice board and employees are given 'Read' only access, meaning it is only a one way channel).

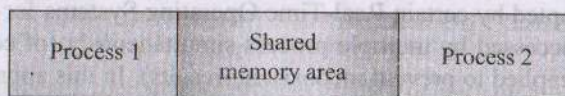


Fig. 10.16 Concept of Shared Memory

The implementation of shared memory concept is kernel dependent. Different mechanisms are adopted by different kernels for implementing this. A few among them are:

10.7.1.1 Pipes 'Pipe' is a section of the shared memory used by processes for communicating. Pipes follow the client-server[†] architecture. A process which creates a pipe is known as a pipe server and a process which connects to a pipe is known as pipe client. A pipe can be considered as a conduit for information flow and has two conceptual ends. It can be unidirectional, allowing information flow in one direction or bidirectional allowing bi-directional information flow. A unidirectional pipe allows the process connecting at one end of the pipe to write to the pipe and the process connected at the other end of the pipe to read the data, whereas a bi-directional pipe allows both reading and writing at one end. The unidirectional pipe can be visualised as

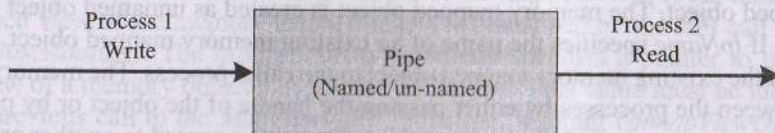


Fig. 10.17 Concept of Pipe for IPC

[†]Client Server is a software architecture containing a client application and a server application. The application which sends request is known as client and the application which receives the request process it and sends a response back to the client is known as server. A server is capable of receiving request from multiple clients.

The implementation of 'Pipes' is also OS dependent. Microsoft® Windows Desktop Operating Systems support two types of 'Pipes' for Inter Process Communication. They are:

Anonymous Pipes: The anonymous pipes are unnamed, unidirectional pipes used for data transfer between two processes.

Named Pipes: Named pipe is a named, unidirectional or bi-directional pipe for data exchange between processes. Like anonymous pipes, the process which creates the named pipe is known as pipe server. A process which connects to the named pipe is known as pipe client. With named pipes, any process can act as both client and server allowing point-to-point communication. Named pipes can be used for communicating between processes running on the same machine or between processes running on different machines connected to a network.

Please refer to the Online Learning Centre for details on the Pipe implementation under Windows Operating Systems.

Under VxWorks kernel, *pipe* is a special implementation of message queues. We will discuss the same in a latter chapter.

10.7.1.2 Memory Mapped Objects Memory mapped object is a shared memory technique adopted by certain Real-Time Operating Systems for allocating a shared block of memory which can be accessed by multiple process simultaneously (of course certain synchronisation techniques should be applied to prevent inconsistent results). In this approach a mapping object is created and physical storage for it is reserved and committed. A process can map the entire committed physical area or a block of it to its virtual address space. All read and write operation to this virtual address space by a process is directed to its committed physical area. Any process which wants to share data with other processes can map the physical memory area of the mapped object to its virtual memory space and use it for sharing the data.

Windows CE 5.0 RTOS uses the memory mapped object based shared memory technique for Inter Process Communication (Fig. 10.18). The *CreateFileMapping (HANDLE hFile, LPSECURITY_ATTRIBUTES lpFileMappingAttributes, DWORD flProtect, DWORD dwMaximumSizeHigh, DWORD dwMaximumSizeLow, LPCTSTR lpName)* system call is used for sharing the memory. This API call is used for creating a mapping from a file. In order to create the mapping from the system paging memory, the handle parameter should be passed as *INVALID_HANDLE_VALUE (-1)*. The *lpFileMappingAttributes* parameter represents the security attributes and it must be *NULL*. The *flProtect* parameter represents the read write access for the shared memory area. A value of *PAGE_READONLY* makes the shared memory read only whereas the value *PAGE_READWRITE* gives read-write access to the shared memory. The parameter *dwMaximumSizeHigh* specifies the higher order 32 bits of the maximum size of the memory mapped object and *dwMaximumSizeLow* specifies the lower order 32 bits of the maximum size of the memory mapped object. The parameter *lpName* points to a null terminated string specifying the name of the memory mapped object. The memory mapped object is created as unnamed object if the parameter *lpName* is *NULL*. If *lpName* specifies the name of an existing memory mapped object, the function returns the handle of the existing memory mapped object to the caller process. The memory mapped object can be shared between the processes by either passing the handle of the object or by passing its name. If the handle of the memory mapped object created by a process is passed to another process for shared access, there is a possibility of closing the handle by the process which created the handle while it is in use by another process. This will throw OS level exceptions. If the name of the memory object is passed for shared access among processes, processes can use this name for creating a shared memory object which will open the shared memory object already existing with the given name. The OS will maintain a

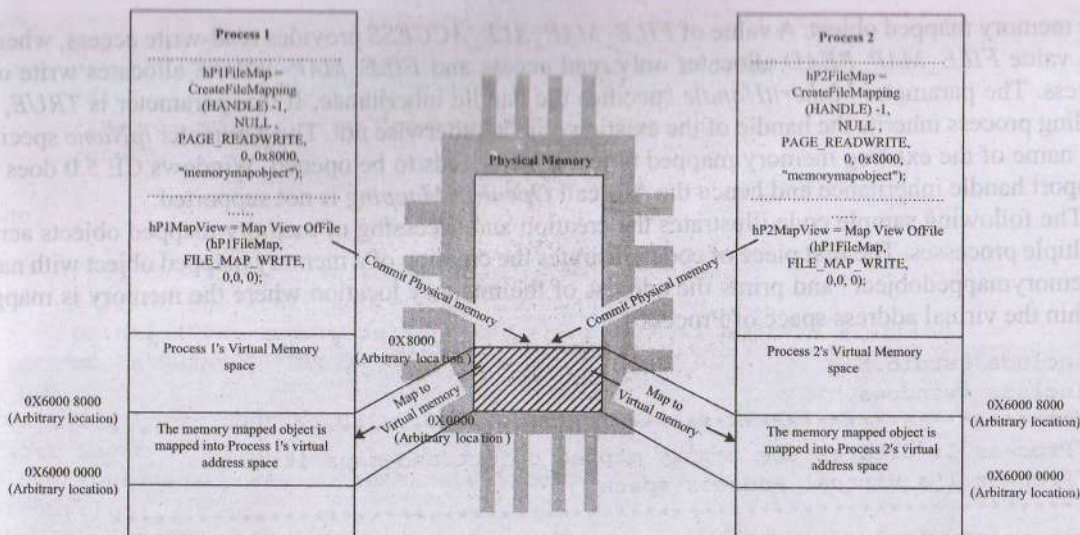


Fig. 10.18 Concept of memory mapped object

usage count for the named object and it is incremented each time when a process creates/opens a memory mapped object with existing name. This will prevent the destruction of a shared memory object by one process while it is being accessed by another process. Hence passing the name of the memory mapped object is strongly recommended for memory mapped object based inter process communication. The *MapViewOfFile* (*HANDLE hFileMappingObject*, *DWORD dwDesiredAccess*, *DWORD dwFileOffsetHigh*, *DWORD dwFileOffsetLow*, *DWORD dwNumberOfBytesToMap*) system call maps a view of the memory mapped object to the address space of the calling process. The parameter *hFileMappingObject* specifies the handle to an existing memory mapped object. The *dwDesiredAccess* parameter represents the read write access for the mapped view area. A value of *FILE_MAP_WRITE* makes the view access read-write, provided the memory mapped object *hFileMappingObject* is created with read-write access, whereas the value *FILE_MAP_READ* gives read only access to the shared memory, provided the memory mapped object *hFileMappingObject* is created with read-write/read only access. The parameter *dwFileOffsetHigh* specifies the higher order 32 bits and *dwFileOffsetLow* specifies the lower order 32 bits of the memory offset where mapping is to begin from the memory mapped object. A value of '0' for both of these maps the view from the beginning memory area of the memory object. *dwNumberOfBytesToMap* specifies the number of bytes of the memory object to map. If *dwNumberOfBytesToMap* is zero, the entire memory area owned by the memory mapped object is mapped. On successful execution, *MapViewOfFile* call returns the starting address of the mapped view. If the function fails it returns *NULL*. A mapped view of the memory mapped object is unmapped by the API call *UnmapViewOfFile* (*LPCVOID lpBaseAddress*). The *lpBaseAddress* parameter specifies a pointer to the base address of the mapped view of a memory object that is to be unmapped. This value must be identical to the value returned by a previous call to the *MapViewOfFile* function. Calling *UnmapViewOfFile* cleans up the committed physical storage in a process's virtual address space. In other words, it frees the virtual address space of the mapping object. Under Windows NT/XP OS, a process can open an existing memory mapped object by calling the API *OpenFileMapping* (*DWORD dwDesiredAccess*, *BOOL bInheritHandle*, *LPCTSTR lpName*). The parameter *dwDesiredAccess* specifies the read write access permissions for

the memory mapped object. A value of `FILE_MAP_ALL_ACCESS` provides read-write access, whereas the value `FILE_MAP_READ` allocates only read access and `FILE_MAP_WRITE` allocates write only access. The parameter `blInheritHandle` specifies the handle inheritance. If this parameter is `TRUE`, the calling process inherits the handle of the existing object, otherwise not. The parameter `lpName` specifies the name of the existing memory mapped object which needs to be opened. Windows CE 5.0 does not support handle inheritance and hence the API call `OpenFileMapping` is not supported.

The following sample code illustrates the creation and accessing of memory mapped objects across multiple processes. The first piece of code illustrates the creation of a memory mapped object with name "memorymappedobject" and prints the address of the memory location where the memory is mapped within the virtual address space of Process 1.

```
#include <stdio.h>
#include <windows.h>
//*****
//Process 1: Creates the memory mapped object and maps it to
//Process 1's Virtual Address space
//*****
void main() {
    //Define the handle to Memory mapped Object
    HANDLE hFileMap;
    //Define the handle to the view of Memory mapped Object
    LPTSTR hMapView;
    printf("//*****\n");
    printf("                Process 1\n");
    printf("//*****\n");
    //Create an 8 KB memory mapped object
    hFileMap = CreateFileMapping( (HANDLE) -1,
        NULL,                // default security attributes
        PAGE_READWRITE, // Read-Write Access
        0, //Higher order 32 bits of the memory mapping object
        0x2000, //Lower order 32 bits of the memory mapping object
        TEXT("memorymappedobject")); // Memory mapped object name
    if (NULL == hFileMap)
    {
        printf ("Memory mapped Object Creation Failed: Error Code: %d\n", GetLastError
        ());
        //Memory mapped Object Creation failed. Return
        return;
    }

    //Map the memory mapped object to Process 1's address space
    hMapView= (LPTSTR) MapViewOfFile(hFileMap,
        FILE_MAP_WRITE,
        0,                //Map the entire view
        0,
        0);
    if (NULL == hMapView)
```



```

printf ("Mapping of Memory mapped view Failed: Error Code:
%d\n", GetLastError ());
//Memory mapped view Creation failed. Return
return;
}
else
{
    //Successfully created the memory mapped view.
    //Print the start address of the mapped view
    printf ("The memory is mapped to the virtual address starting
    at 0x%08x\n", hMapView);
}
//Wait for user input to exit. Run Process 2 before providing
//user input
printf ("Press any key to terminate Process 1");
getchar();
//Unmap the view
UnmapViewOfFile(hMapView);
//Close memory mapped object handle
CloseHandle(hFileMap);
return;
}

```

The piece of code given below corresponds to Process 2. It illustrates the accessing of an existing memory mapped object (memory mapped object with name "memorymappedobject" created by Process 1) and prints the address of the memory location where the memory is mapped within the virtual address space of Process 2. To demonstrate the application, the program corresponding to Process 1 should be executed first and the program corresponding to Process 2 should be executed following Process 1.

```

#include <stdio.h>
#include <windows.h>
//*****
//Process 2: Opens the memory mapped object created by Process 1
//Maps the object to Process 2's virtual address space.
//*****
void main() {
    //Define the handle for the Memory mapped Object
    HANDLE hChildFileMap;
    //Define the handle for the view of Memory mapped Object
    LPTSTR hChildMapView;
    printf("//*****\n");
    printf("                Process 2\n");
    printf("//*****\n");
    //Create an 8 KB memory mapped object
    hChildFileMap = CreateFileMapping( INVALID_HANDLE_VALUE,
    NULL,                // default security attributes
    PAGE_READWRITE, // Read-Write Access
    0, //Higher order 32 bits of the memory mapping object

```

Message passing is an (a)synchronous information exchange mechanism used for Inter Process Thread


```

0x2000, //Lower order 32 bits of the memory mapping object
TEXT("memorymappedobject")); // Memory mapped object name
if (NULL == hChildFileMap || INVALID_HANDLE_VALUE == hChildFileMap)
{
printf ("Memory mapped Object Creation Failed: Error Code: %d\n",
GetLastError ());
//Memory mapped Object Creation failed. Return
return;
}
else
{
if (ERROR_ALREADY_EXISTS == GetLastError ())
{
//A memory mapped object with given name exists already.
printf ("The named memory mapped object is already
existing\n");
}
//Map the memory mapped object to Process 2's address space

hChildMapView=(LPTSTR)MapViewOfFile(hChildFileMap,
FILE_MAP_WRITE, //Read-Write access
0, //Map the entire view
0,
0);
if (NULL == hChildMapView)
{
printf ("Mapping of Memory mapped view Failed: Error Code: %d\n", GetLastError ());
//Memory mapped view Creation failed. Return
return;
}
else
{
//Successfully created the memory mapped view.
//Print the start address of the mapped view
printf ("The memory mapped view starts at memory location 0x%08x\n", hChild-
MapView);
}

//Unmap the view
UnmapViewOfFile(hChildMapView);
//Close memory mapped object handle
CloseHandle(hChildFileMap);
return;
}

```


The output of the above programs when run in the sequence, Process 1 is executed first and Process 2 is executed while Process 1 waits for the user input from the keyboard, is given in Fig. 10.19.

The Memory mapped object is mapped to the Virtual address 0x00340000 of both the Processes

```

C:\Program Files\Microsoft Visual Studio\MyProjects\consume\Debug\consume.exe
*****
Process 1
*****
The memory is mapped to the virtual address starting at 0x00340000
Press any key to terminate Process 1

C:\Program Files\Microsoft Visual Studio\MyProjects\consume\Debug\consume.exe
*****
Process 2
*****
The named memory mapped object is already existing
The memory mapped view starts at memory location 0x00340000
Press any key to continue
  
```

Fig. 10.19 Output of Win32 memory mapped object illustration program

This reveals that using memory mapped objects with same name across multiple processes running on the same system maps the object to the same virtual address space of the processes.

Reading and writing to a memory mapped area is same as any read write operation using pointers. The pointer returned by the API call *MapViewOfFile* can be used for this. The exercise of Read and Write operation is left to the readers. Proper care should be taken to avoid any conflicts that may arise due to the simultaneous read/write access of the shared memory area by multiple processes. This can be handled by applying various synchronisation techniques like events, mutex, semaphore, etc.

For using a memory mapped object across multiple threads of a process, it is not required for all the threads of the process to create/open the memory mapped object and map it to the thread's virtual address space. Since the thread's address space is part of the process's virtual address space, which contains the thread, only one thread, preferably the parent thread (main thread) is required to create the memory mapped object and map it to the process's virtual address space. The thread which creates the memory mapped object can share the pointer to the mapped memory area as global pointer and other threads of the process can use this pointer for reading and writing to the mapped memory area. If one thread of a process tries to create a memory mapped object with the same name as that of an existing mapping object, which is created by another thread of the same process, a new view of the mapping object is created at a different virtual address of the process. This is same as one process trying to create two views of the same memory mapped object☺.

10.7.2 Message Passing

Message passing is an (a)synchronous information exchange mechanism used for Inter Process/Thread

Communication. The major difference between shared memory and message passing technique is that, through shared memory lots of data can be shared whereas only limited amount of info/data is passed through message passing. Also message passing is relatively fast and free from the synchronisation overheads compared to shared memory. Based on the message passing operation between the processes, message passing is classified into

10.7.2.1 Message Queue Usually the process which wants to talk to another process posts the message to a First-In-First-Out (FIFO) queue called 'Message queue', which stores the messages temporarily in a system defined memory object, to pass it to the desired process (Fig. 10.20). Messages are sent and received through *send* (*Name of the process to which the message is to be sent, message*) and *receive* (*Name of the process from which the message is to be received, message*) methods. The messages are exchanged through a message queue. The implementation of the message queue, *send* and *receive* methods are OS kernel dependent. The Windows XP OS kernel maintains a single system message queue and one process/thread (Process and threads are used interchangeably here, since thread is the basic unit of process in windows) specific message queue. A thread which wants to communicate with another thread posts the message to the system message queue. The kernel picks up the message from the system message queue one at a time and examines the message for finding the destination thread and then posts the message to the message queue of the corresponding thread. For posting a message to a thread's message queue, the kernel fills a message structure *MSG* and copies it to the message queue of the thread. The message structure *MSG* contains the handle of the process/thread for which the message is intended, the message parameters, the time at which the message is posted, etc. A thread can simply post a message to another thread and can continue its operation or it may wait for a response from the thread to which the message is posted. The messaging mechanism is classified into synchronous and asynchronous based on the behaviour of the message posting thread. In asynchronous messaging, the message posting thread just posts the message to the queue and it will not wait for an acceptance (return) from the thread to which the message is posted, whereas in synchronous messaging, the thread which posts a message enters waiting state and waits for the message result from the thread to which the message is posted. The thread which invoked the send message becomes blocked and the scheduler will not pick it up for scheduling. The *PostMessage* (*HWND hWnd, UINT Msg, WPARAM wParam, LPARAM*

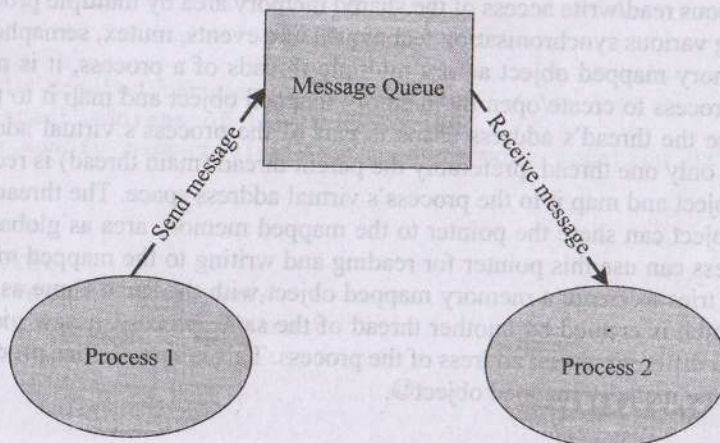


Fig. 10.20 Concept of message queue based indirect messaging for IPC

lParam) or *PostThreadMessage* (*DWORD idThread*, *UINT Msg*, *WPARAM wParam*, *LPARAM lParam*) API is used by a thread in Windows for posting a message to its own message queue or to the message queue of another thread. The *PostMessage* API does not always guarantee the posting of messages to message queue. The *PostMessage* API will not post a message to the message queue when the message queue is full. Hence it is recommended to check the return value of *PostMessage* API to confirm the posting of message. The *SendMessage* (*HWND hWnd*, *UINT Msg*, *WPARAM wParam*, *LPARAM lParam*) API call sends a message to the thread specified by the handle *hWnd* and waits for the called thread to process the message. The thread which calls the *SendMessage* API enters waiting state and waits for the message result from the thread to which the message is posted. The thread which invoked the *SendMessage* API call becomes blocked and the scheduler will not pick it up for scheduling.

The Windows CE operating system supports a special Point-to-Point Message queue implementation. The OS maintains a First In First Out (FIFO) buffer for storing the messages and each process can access this buffer for reading and writing messages. The OS also maintains a special queue, with single message storing capacity, for storing high priority messages (Alert messages). The creation and usage of message queues under Windows CE OS is explained below.

The *CreateMsgQueue*(*LPCWSTR lpzName*, *LMSGQUEUEOPTIONS lpOptions*) API call creates a message queue or opens a named message queue and returns a read only or write only handle to the message queue. A process can use this handle for reading or writing a message from/to of the message queue pointed by the handle. The parameter *lpzName* specifies the name of the message queue. If this parameter is *NULL*, an unnamed message queue is created. Processes can use the handle returned by the API call if the message queue is created without any name. If the message queue is created as named message queue, other processes can use the name of the message queue for opening the named message queue created by a process. Calling the *CreateMsgQueue* API with an existing named message queue as parameter returns a handle to the existing message queue. Under the Desktop Windows Operating Systems (Windows 9x/XP/NT/2K), each object type (viz. mutex, semaphores, events, memory maps, watchdog timers and message queues) share the same namespace and the same name is not allowed for creating any of this. Windows CE kernel maintains separate namespace for each and supports the same name across different objects. The *lpOptions* parameter points to a *MSGQUEUEOPTIONS* structure that sets the properties of the message queue. The member details of the *MSGQUEUEOPTIONS* structure is explained below.

```
typedef MSGQUEUEOPTIONS_OS{
    DWORD dwSize;
    DWORD dwFlags;
    DWORD dwMaxMessages;
    DWORD cbMaxMessage;
    BOOL bReadAccess;
} MSGQUEUEOPTIONS, FAR* LMSGQUEUEOPTIONS, *PMSGQUEUEOPTIONS;
```

The members of the structure are listed below.

Member	Description
<i>dwSize</i>	Specifies the size of the structure in bytes
<i>dwFlags</i>	Describes the behaviour of the message queue. Set to <i>MSGQUEUE_NOPRECOMMIT</i> to allocate message buffers on demand and to free the message buffers after they are read, or set to <i>MSGQUEUE_ALLOW_BROKEN</i> to enable a read or write operation to complete even if there is no corresponding writer or reader present.

<code>dwMaxMessages</code>	Specifies the maximum number of messages to queue at any point of time. Set this value to zero to specify no limit on the number of messages to queue at any point of time.
<code>cbMaxMessage</code>	Specifies the maximum number of bytes in each message. This value must be greater than zero.
<code>bReadAccess</code>	Specifies the Read Write access to the message queue. Set to TRUE to request read access to the queue. Set to FALSE to request write access to the queue.

On successful execution, the *CreateMsgQueue* API call returns a 'Read Only' or 'Write Only' handle to the specified queue based on the *bReadAccess* member of the *MSGQUEUEOPTIONS* structure *lpOptions*. If the queue with specified name already exists, a new handle, which points to the existing queue, is created and a following call to *GetLastError* returns *ERROR_ALREADY_EXISTS*. If the function fails it returns *NULL*. A single call to the *CreateMsgQueue* creates the queue for either 'read' or 'write' access. The *CreateMsgQueue* API should be called twice with the *bReadAccess* member of the *MSGQUEUEOPTIONS* structure *lpOptions* set to TRUE and FALSE respectively in successive calls for obtaining 'Read only' and 'Write only' handles to the specified message queue. The handle returning by *CreateMsgQueue* API call is an event handle and, if it is a 'Read Only' access handle, it is signalled by the message queue if a new message is placed in the queue. The signal is reset on reading the message by *ReadMsgQueue* API call. A 'Write Only' access handle to the message queue is signalled when the queue is no longer full, i.e. when there is room for accommodating new messages. Processes can monitor the handles with the help of the wait functions, viz. *WaitForSingleObject* or *WaitForMultipleObjects*, supported by Windows CE. The *OpenMsgQueue(HANDLE hSrcProc, HANDLE hMsgQ, LPMSGQUEUEOPTIONS lpOptions)* API call opens an existing named or unnamed message queue. The parameter *hSrcProc* specifies the process handle of the process that owns the message queue and *hMsgQ* specifies the handle of the existing message queue (Handle to the message queue returned by the *CreateMsgQueue* function). As in the case of *CreateMsgQueue*, the *lpOptions* parameter points to a *MSGQUEUEOPTIONS* structure that sets the properties of the message queue. On successful execution the *OpenMsgQueue* API call returns a handle to the message queue and *NULL* if it fails. Normally the *OpenMsgQueue* API is used for opening an unnamed message queue. The *WriteMsgQueue(HANDLE hMsgQ, LPVOID lpBuffer, DWORD cbDataSize, DWORD dwTimeout, DWORD dwFlags)* API call is used for writing a single message to the message queue pointed by the handle *hMsgQ*. *lpBuffer* points to a buffer that contains the message to be written to the message queue. The parameter *cbDataSize* specifies the number of bytes stored in the buffer pointed by *lpBuffer*, which forms a message. The parameter *dwTimeout* specifies the timeout interval in milliseconds for the message writing operation. A value of zero specifies the write operation to return immediately without blocking if the write operation cannot succeed. If the parameter is set to *INFINITE*, the write operation will block until it succeeds or the message queue signals the 'write only' handle indicating the availability of space for posting a message. The *dwFlags* parameter sets the priority of the message. If it is set to *MSGQUEUE_MSGALERT*, the message is posted to the queue as high priority or alert message. The Alert message is always placed in the front of the message queue. This function returns *TRUE* if it succeeds and *FALSE* otherwise.

The *ReadMsgQueue(HANDLE hMsgQ, LPVOID lpBuffer, DWORD cbBufferSize, LPDWORD lpNumberOfBytesRead, DWORD dwTimeout, DWORD* pdwFlags)* API reads a single message from the message queue. The parameter *hMsgQ* specifies a handle to the message queue from which the message needs to be read. *lpBuffer* points to a buffer for storing the message read from the queue. The parameter *cbBufferSize* specifies the size of the buffer pointed by *lpBuffer*, in bytes. *lpNumberOfBytesRead* specifies the number of bytes stored in the buffer. This is same as the number of bytes present in

the message which is read from the message queue. *dwTimeout* specifies the timeout interval in milliseconds for the message reading operation. The timeout values and their meaning are same as that of the write message timeout parameter. The *dwFlags* parameter indicates the priority of the message. If the message read from the message queue is a high priority message (alert message), *dwFlags* is set to *MSGQUEUE_MSGALERT*. The function returns *TRUE* if it succeeds and *FALSE* otherwise. The *GetMsgQueueInfo* (*HANDLE hMsgQ*, *LPMMSGQUEUEINFO lpInfo*) API call returns the information about a message queue specified by the handle *hMsgQ*. The message information is returned in a *MSGQUEUEINFO* structure pointed by *lpInfo*. The details of the *MSGQUEUEINFO* structure is explained below.

```
typedef MSGQUEUEINFO{
    DWORD dwSize;
    DWORD dwFlags;
    DWORD dwMaxMessages;
    DWORD cbMaxMessage;
    DWORD dwCurrentMessages;
    DWORD dwMaxQueueMessages;
    WORD wNumReaders;
    WORD wNumWriters;
} MSGQUEUEINFO, *PMSGQUEUEINFO, FAR* LPMMSGQUEUEINFO;
```

The member variable details are listed below.

Member	Description
DwSize	Specifies the size of the buffer passed in.
dwFlags	Describes the behaviour of the message queue. It retrieves the <i>MSGQUEUEOPTIONS</i> . <i>dwFlags</i> passed when the message queue is created with <i>CreateMsgQueue</i> API call.
dwMaxMessages	Specifies the maximum number of messages to queue at any point of time. This reflects the <i>MSGQUEUEOPTIONS</i> . <i>dwMaxMessages</i> value passed when the message queue is created with <i>CreateMsgQueue</i> API call.
cbMaxMessage	Specifies the maximum number of bytes in each message. This reflects the <i>MSGQUEUEOPTIONS</i> . <i>cbMaxMessage</i> value passed when the message queue is created with <i>CreateMsgQueue</i> API call.
dwCurrentMessages	Specifies the number of messages currently existing in the specified message queue.
dwMaxQueueMessages	Specifies maximum number of messages that have ever been in the queue at one time.
wNumReaders	Specifies the number of readers (processes which opened the message queue for reading) subscribed to the message queue for reading.
wNumWriters	Specifies the number of writers (processes which opened the message queue for writing) subscribed to the message queue for writing.

The *GetMsgQueueInfo* API call returns *TRUE* if it succeeds and *FALSE* otherwise. The *CloseMsgQueue*(*HANDLE hMsgQ*) API call closes a message queue specified by the handle *hMsgQ*. If a process holds a 'read only' and 'write only' handle to the message queue, both should be closed for closing the message queue.

'Message queue' is the primary inter-task communication mechanism under VxWorks kernel. Message queues support two-way communication of messages of variable length. The two-way messaging

between tasks can be implemented using one message queue for incoming messages and another one for outgoing messages. Messaging mechanism can be used for task-to task and task to Interrupt Service Routine (ISR) communication. We will discuss about the VxWorks' message queue implementation in a separate chapter.

10.7.2.2 Mailbox Mailbox is an alternate form of 'Message queues' and it is used in certain Real-Time Operating Systems for IPC. Mailbox technique for IPC in RTOS is usually used for one way messaging. The task/thread which wants to send a message to other tasks/threads creates a mailbox for posting the messages. The threads which are interested in receiving the messages posted to the mailbox by the mailbox creator thread can subscribe to the mailbox. The thread which creates the mailbox is known as 'mailbox server' and the threads which subscribe to the mailbox are known as 'mailbox clients'. The mailbox server posts messages to the mailbox and notifies it to the clients which are subscribed to the mailbox. The clients read the message from the mailbox on receiving the notification. The mailbox creation, subscription, message reading and writing are achieved through OS kernel provided API calls. Mailbox and message queues are same in functionality. The only difference is in the number of messages supported by them. Both of them are used for passing data in the form of message(s) from a task to another task(s). Mailbox is used for exchanging a single message between two tasks or between an Interrupt Service Routine (ISR) and a task. Mailbox associates a pointer pointing to the mailbox and a wait list to hold the tasks waiting for a message to appear in the mailbox. The implementation of mailbox is OS kernel dependent. The MicroC/OS-II implements mailbox as a mechanism for inter-task communication. We will discuss about the mailbox based IPC implementation under MicroC/OS-II in a latter chapter. Figure 10.21 given below illustrates the mailbox based IPC technique.

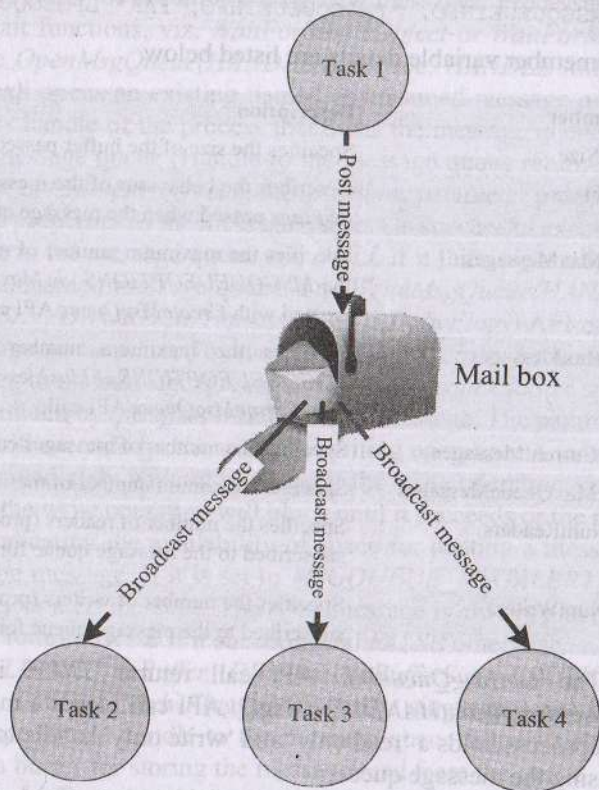


Fig. 10.21

Concept of Mailbox based indirect messaging for IPC

10.7.2.3 Signalling Signalling is a primitive way of communication between processes/threads. Signals are used for asynchronous notifications where one process/thread fires a signal, indicating the occurrence of a scenario which the other process(es)/thread(s) is waiting. Signals are not queued and they do not carry any data. The communication mechanisms used in RTX51 Tiny OS is an example for Signalling. The `os_send_signal` kernel call under RTX 51 sends a signal from one task to a specified task. Similarly the `os_wait_signal` kernel call waits for a specified signal. Refer to the topic 'Round Robin Scheduling' under the section 'Priority based scheduling' for more details on Signalling in RTX51 Tiny OS. The VxWorks RTOS kernel also implements 'signals' for inter process

communication. Whenever a specified signal occurs it is handled in a signal handler associated with the signal. We will discuss about the signal based IPC mechanism for VxWorks' kernel in a later chapter.

10.7.3 Remote Procedure Call (RPC) and Sockets

Remote Procedure Call or RPC (Fig. 10.22) is the Inter Process Communication (IPC) mechanism used by a process to call a procedure of another process running on the same CPU or on a different CPU which is interconnected in a network. In the object oriented language terminology RPC is also known as *Remote Invocation* or *Remote Method Invocation (RMI)*. RPC is mainly used for distributed applications like client-server applications. With RPC it is possible to communicate over a heterogeneous network (i.e. Network where Client and server applications are running on different Operating systems). The CPU/process containing the procedure which needs to be invoked remotely is known as server. The CPU/process which initiates an RPC request is known as client.

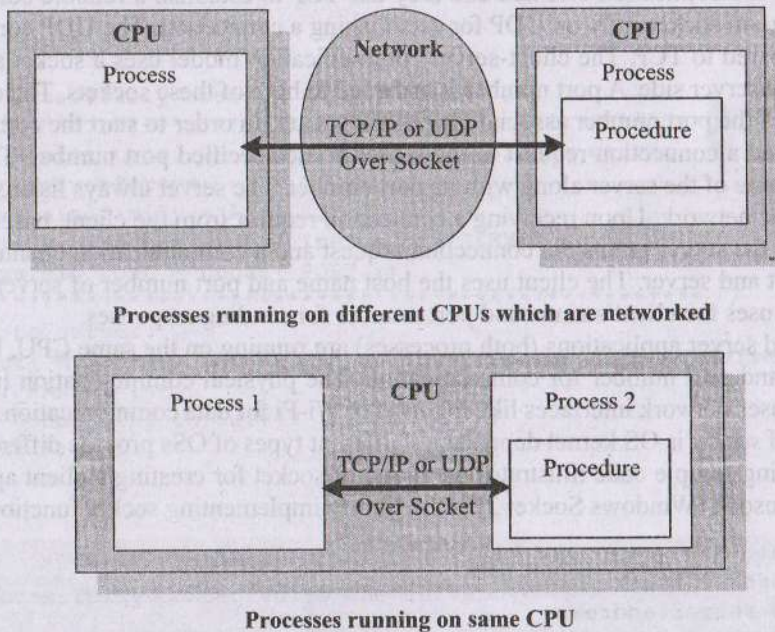


Fig. 10.22 Concept of Remote Procedure Call (RPC) for IPC

It is possible to implement RPC communication with different invocation interfaces. In order to make the RPC communication compatible across all platforms it should stick on to certain standard formats. Interface Definition Language (IDL) defines the interfaces for RPC. Microsoft Interface Definition Language (MIDL) is the IDL implementation from Microsoft for all Microsoft platforms. The RPC communication can be either Synchronous (Blocking) or Asynchronous (Non-blocking). In the Synchronous communication, the process which calls the remote procedure is blocked until it receives a response back from the other process. In asynchronous RPC calls, the calling process continues its execution while the remote process performs the execution of the procedure. The result from the remote procedure is returned back to the caller through mechanisms like callback functions.

On security front, RPC employs authentication mechanisms to protect the systems against vulnerabilities. The client applications (processes) should authenticate themselves with the server for getting access. Authentication mechanisms like IDs, public key cryptography (like DES, 3DES), etc. are used by the client for authentication. Without authentication, any client can access the remote procedure. This may lead to potential security risks.

Sockets are used for RPC communication. *Socket is a logical endpoint in a two-way communication link between two applications running on a network. A port number is associated with a socket so that the network layer of the communication channel can deliver the data to the designated application.* Sockets are of different types, namely, Internet sockets (INET), UNIX sockets, etc. The INET socket works on internet communication protocol. TCP/IP, UDP, etc. are the communication protocols used by INET sockets. INET sockets are classified into:

1. Stream sockets
2. Datagram sockets

Stream sockets are connection oriented and they use TCP to establish a reliable connection. On the other hand, *Datagram sockets* rely on UDP for establishing a connection. The UDP connection is unreliable when compared to TCP. The client-server communication model uses a socket at the client side and a socket at the server side. A port number is assigned to both of these sockets. The client and server should be aware of the port number associated with the socket. In order to start the communication, the client needs to send a connection request to the server at the specified port number. The client should be aware of the name of the server along with its port number. The server always listens to the specified port number on the network. Upon receiving a connection request from the client, based on the success of authentication, the server grants the connection request and a communication channel is established between the client and server. The client uses the host name and port number of server for sending requests and server uses the client's name and port number for sending responses.

If the client and server applications (both processes) are running on the same CPU, both can use the same host name and port number for communication. The physical communication link between the client and server uses network interfaces like Ethernet or Wi-Fi for data communication. The underlying implementation of *socket* is OS kernel dependent. Different types of OSs provide different socket interfaces. The following sample code illustrates the usage of socket for creating a client application under Windows OS. Winsock (Windows Socket 2) is the library implementing socket functions for Win32.

```
#include <stdio.h>
#include "winsock2.h"
//Specify the server address
#define SERVER        "172.168.0.1"
//Specify the server port
#define PORT 5000
int buflength = 100;
char *sendbuf = "Hi from Client";
char buffer[buflength];
void main() {
    //*****
    // Initialize Winsock
    WSADATA wsaData;
    if (WSAStartup(MAKEWORD(2,2), &wsaData) == NO_ERROR)
        printf("Winsock Initialisation succeeded...\n");
```



```

else
{
    printf("Winsock Initialisation failed...\n");
    return;
}
//*****
// Create a SOCKET for connecting to server
SOCKET MySocket;
MySocket = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
if (MySocket == INVALID_SOCKET)
{
    printf("Socket Creation failed...\n");
    WSACleanup();
    return;
}
else
{
    printf("Successfully created the socket...\n");
    //*****
    // Set the Socket type, IP address and port of the server
    sockaddr_in ServerParams;
    ServerParams.sin_family = AF_INET;
    ServerParams.sin_addr.s_addr = inet_addr( SERVER );
    ServerParams.sin_port = htons( PORT );
    //*****
    // Connect to server.
    if ( connect( MySocket, (SOCKADDR*)&ServerParams, sizeof(ServerParams) )
        == SOCKET_ERROR ) {
        printf( "Connecting to Server failed...\n" );
        WSACleanup();
        return;
    }
    else
    {
        printf("Successfully Connected to the server...\n");
        //*****
        // Send command to server
        if (send(MySocket, sendbuf, (int)strlen(sendbuf), 0 )
            == SOCKET_ERROR) {
            printf("Sending data to server failed...\n");
            closesocket(MySocket);
            WSACleanup();
            return;
        }
    }
}
else
{
    printf("Successfully sent command to server...\n");
}

```



```

//*****
// Receive a data packet
if (recv(MySocket, recvbuf, recvbuflen, 0) > 0 )
    printf("Successfully Received a byte...\n
    The received byte is %s\n", recvbuf);
else
    printf("No response from server...\n");
//*****
//Close Socket
closesocket(MySocket);
WSACleanup();
return;
}
}
}
}
}
}

```

The above application tries to connect to a server machine with IP address 172.168.0.1 and port number 5000. Change the values of *SERVER* and *PORT* to connect to a machine with different IP address and port number. If the connection is success, it sends the data "Hi from Client" to the server and waits for a response from the server and finally terminates the connection.

Under Windows, the socket function library *Winsock* should be initiated before using the socket related functions. The function *WSAStartup* performs this initiation. The *socket()* function call creates a socket. The socket type, connection type and protocols for communication are the parameters for socket creation. Here the socket type is *INET (AF_INET)* and connection type is stream socket (*SOCK_STREAM*). The protocol selected for communication is *TCP/IP (IPPROTO_TCP)*. After creating the socket it is connected to a server. For connecting to server, the server address and port number should be indicated in the connection request. The *sockaddr_in* structure specifies the socket type, IP address and port of the server to be connected to. The *connect()* function connects the socket with a specified server. If the server grants the connection request, the *connect()* function returns success. The *send()* function is used for sending data to a server. It takes the socket name and data to be sent as parameters. Data from server is received using the function call *recv()*. It takes the socket name and details of buffer to hold the received data as parameters. The TCP/IP network stack expects network byte order (Big Endian: Higher order byte of the data is stored in lower memory address location) for data. The function *htons()* converts the byte order of an unsigned short integer to the network order. The *closesocket()* function closes the socket connection. On the server side, the server creates a socket using the function *socket()* and binds the socket with a port using the *bind()* function. It listens to the port bonded to the socket for any incoming connection request. The function *listen()* performs this. Upon receiving a connection request, the server accepts it. The function *accept()* performs the accepting operation. Now the connectivity is established. Server can receive and transmit data using the function calls *recv()* and *send()* respectively. The implementation of the server application is left to the readers as an exercise.

10.8 TASK SYNCHRONISATION

In a multitasking environment, multiple processes run concurrently (in pseudo parallelism) and share the system resources. Apart from this, each process has its own boundary wall and they communicate with

each other with different IPC mechanisms including shared memory and variables. Imagine a situation where two processes try to access display hardware connected to the system or two processes try to access a shared memory area where one process tries to write to a memory location when the other process is trying to read from this. What could be the result in these scenarios? Obviously unexpected results. How these issues can be addressed? The solution is, make each process aware of the access of a shared resource either directly or indirectly. The act of making processes aware of the access of shared resources by each process to avoid conflicts is known as '*Task/Process Synchronisation*'. Various synchronisation issues may arise in a multitasking environment if processes are not synchronised properly. The following sections describe the major task communication synchronisation issues observed in multitasking and the commonly adopted synchronisation techniques to overcome these issues.

10.8.1 Task Communication/Synchronisation Issues

10.8.1.1 Racing Let us have a look at the following piece of code:

```
#include <windows.h>
#include <stdio.h>
//*****
//counter is an integer variable and Buffer is a byte array shared
//between two processes Process A and Process B
char Buffer[10] = {1,2,3,4,5,6,7,8,9,10};
short int counter = 0;
//*****
// Process A
void Process_A (void) {
int i;
for (i =0; i<5; i++)
{
if (Buffer[i] > 0)
counter++;
}
}
//*****
// Process B
void Process_B(void) {
int j;
for (j =5; j<10; j++)
{
if (Buffer[j] > 0)
counter++;
}
}
//*****
//Main Thread.
int main() {
DWORD id;
```



```

CreateThread(NULL,0,
            (LPTHREAD_START_ROUTINE)Process_A,
            (LPVOID)0,0,&id);
CreateThread(NULL,0,
            (LPTHREAD_START_ROUTINE)Process_B,
            (LPVOID)0,0,&id);
Sleep(100000);
return 0;
}

```

From a programmer perspective the value of *counter* will be 10 at the end of execution of processes A & B. But 'it need not be always' in a real world execution of this piece of code under a multitasking kernel. The results depend on the process scheduling policies adopted by the OS kernel. Now let's dig into the piece of code illustrated above. The program statement *counter++*; looks like a single statement from a high level programming language ('C' language) perspective. The low level implementation of this statement is dependent on the underlying processor instruction set and the (cross) compiler in use. The low level implementation of the high level program statement *counter++*; under Windows XP operating system running on an *Intel Centrino Duo* processor is given below. The code snippet is compiled with Microsoft Visual Studio 6.0 compiler.

```

mov eax,dword ptr [ebp-4];Load counter in Accumulator
add eax,1 ; Increment Accumulator by 1
mov dword ptr [ebp-4],eax ;Store counter with Accumulator

```

At the processor instruction level, the value of the variable *counter* is loaded to the Accumulator register (*EAX* register). The memory variable *counter* is represented using a pointer. The base pointer register (*EBP* register) is used for pointing to the memory variable *counter*. After loading the contents of the variable *counter* to the Accumulator, the Accumulator content is incremented by one using the add instruction. Finally the content of Accumulator is loaded to the memory location which represents the variable counter. Both the processes Process A and Process B contain the program statement *counter++*; Translating this into the machine instruction.

Process A	Process B
mov eax,dword ptr [ebp-4]	mov eax,dword ptr [ebp-4]
add eax,1	add eax,1
mov dword ptr [ebp-4],eax	mov dword ptr [ebp-4],eax

Imagine a situation where a process switching (context switching) happens from Process A to Process B when Process A is executing the *counter++*; statement. Process A accomplishes the *counter++*; statement through three different low level instructions. Now imagine that the process switching happened at the point where Process A executed the low level instruction, 'mov eax,dword ptr [ebp-4]' and is about to execute the next instruction 'add eax,1'. The scenario is illustrated in Fig. 10.23.

Process B increments the shared variable 'counter' in the middle of the operation where Process A tries to increment it. When Process A gets the CPU time for execution, it starts from the point where it got interrupted (If Process B is also using the same registers *eax* and *ebp* for executing *counter++*;

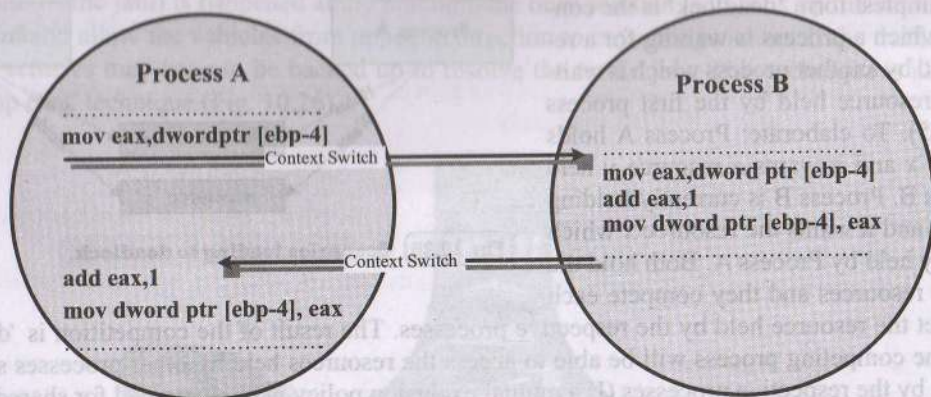


Fig. 10.23 Race condition

instruction, the original content of these registers will be saved as part of the context saving and it will be retrieved back as part of context retrieval, when process A gets the CPU for execution. Hence the content of *eax* and *ebp* remains intact irrespective of context switching). Though the variable *counter* is incremented by Process B, Process A is unaware of it and it increments the variable with the old value. This leads to the loss of one increment for the variable *counter*. This problem occurs due to non-atomic[†] operation on variables. This issue wouldn't have been occurred if the underlying actions corresponding to the program statement *counter++*; is finished in a single CPU execution cycle. The best way to avoid this situation is make the access and modification of shared variables mutually exclusive; meaning when one process accesses a shared variable, prevent the other processes from accessing it. We will discuss this technique in more detail under the topic 'Task Synchronisation techniques' in a later section of this chapter.

To summarise, *Racing* or *Race condition* is the situation in which multiple processes compete (race) each other to access and manipulate shared data concurrently. In a Race condition the final value of the shared data depends on the process which acted on the data finally.

10.8.1.2 Deadlock A race condition produces incorrect results whereas a deadlock condition creates a situation where none of the processes are able to make any progress in their execution, resulting in a set of deadlocked processes. A situation very similar to our traffic jam issues in a junction as illustrated in Fig. 10.24.

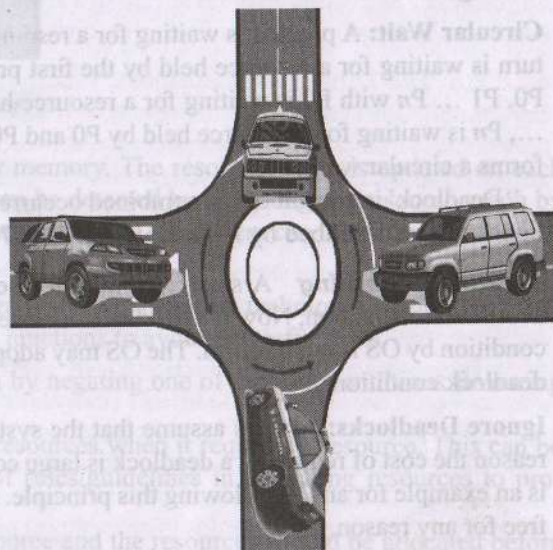


Fig. 10.24 Deadlock visualisation

[†] Atomic Operation: Operations which are non-interruptible.

In its simplest form 'deadlock' is the condition in which a process is waiting for a resource held by another process which is waiting for a resource held by the first process (Fig. 10.25). To elaborate: Process A holds a resource x and it wants a resource y held by Process B. Process B is currently holding resource y and it wants the resource x which is currently held by Process A. Both hold the respective resources and they compete each

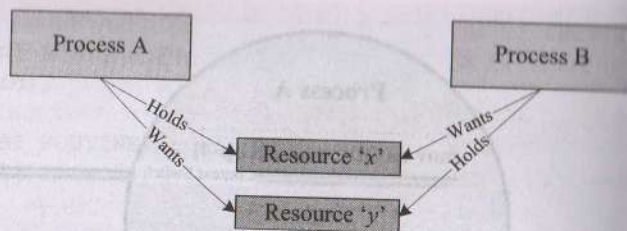


Fig. 10.25 Scenarios leading to deadlock

other to get the resource held by the respective processes. The result of the competition is 'deadlock'. None of the competing process will be able to access the resources held by other processes since they are locked by the respective processes (If a mutual exclusion policy is implemented for shared resource access, the resource is locked by the process which is currently accessing it).

The different conditions favouring a deadlock situation are listed below.

Mutual Exclusion: The criteria that only one process can hold a resource at a time. Meaning processes should access shared resources with mutual exclusion. Typical example is the accessing of display hardware in an embedded device.

Hold and Wait: The condition in which a process holds a shared resource by acquiring the lock controlling the shared access and waiting for additional resources held by other processes.

No Resource Preemption: The criteria that operating system cannot take back a resource from a process which is currently holding it and the resource can only be released voluntarily by the process holding it.

Circular Wait: A process is waiting for a resource which is currently held by another process which in turn is waiting for a resource held by the first process. In general, there exists a set of waiting process $P_0, P_1 \dots P_n$ with P_0 is waiting for a resource held by P_1 and P_1 is waiting for a resource held by $P_0, \dots, P_n$ is waiting for a resource held by P_0 and P_0 is waiting for a resource held by P_n and so on... This forms a circular wait queue.

'Deadlock' is a result of the combined occurrence of these four conditions listed above. These conditions are first described by E. G. Coffman in 1971 and it is popularly known as *Coffman conditions*.

Deadlock Handling A smart OS may foresee the deadlock condition and will act proactively to avoid such a situation. Now if a deadlock occurred, how the OS responds to it? The reaction to deadlock condition by OS is nonuniform. The OS may adopt any of the following techniques to detect and prevent deadlock conditions.

Ignore Deadlocks: Always assume that the system design is deadlock free. This is acceptable for the reason the cost of removing a deadlock is large compared to the chance of happening a deadlock. UNIX is an example for an OS following this principle. A life critical system cannot pretend that it is deadlock free for any reason.

Detect and Recover: This approach suggests the detection of a deadlock situation and recovery from it. This is similar to the deadlock condition that may arise at a traffic junction. When the vehicles from different directions compete to cross the junction, deadlock (traffic jam) condition is resulted. Once a

deadlock (traffic jam) is happened at the junction, the only solution is to back up the vehicles from one direction and allow the vehicles from opposite direction to cross the junction. If the traffic is too high, lots of vehicles may have to be backed up to resolve the traffic jam. This technique is also known as 'back up cars' technique (Fig. 10.26).

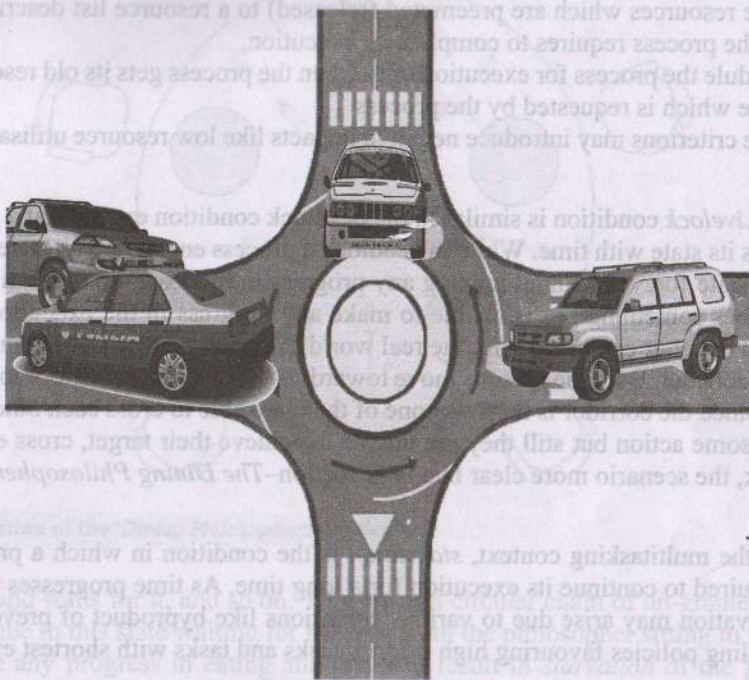


Fig. 10.26 'Back up cars' technique for deadlock recovery

Operating systems keep a resource graph in their memory. The resource graph is updated on each resource request and release. A deadlock condition can be detected by analysing the resource graph by graph analyser algorithms. Once a deadlock condition is detected, the system can terminate a process or preempt the resource to break the deadlocking cycle.

Avoid Deadlocks: Deadlock is avoided by the careful resource allocation techniques by the Operating System. It is similar to the traffic light mechanism at junctions to avoid the traffic jams.

Prevent Deadlocks: Prevent the deadlock condition by negating one of the four conditions favouring the deadlock situation.

- Ensure that a process does not hold any other resources when it requests a resource. This can be achieved by implementing the following set of rules/guidelines in allocating resources to processes.
 1. A process must request all its required resource and the resources should be allocated before the process begins its execution.
 2. Grant resource allocation requests from processes only if the process does not hold a resource currently.

- Ensure that resource preemption (resource releasing) is possible at operating system level. This can be achieved by implementing the following set of rules/guidelines in resources allocation and releasing.
 1. Release all the resources currently held by a process if a request made by the process for a new resource is not able to fulfil immediately.
 2. Add the resources which are preempted (released) to a resource list describing the resources which the process requires to complete its execution.
 3. Reschedule the process for execution only when the process gets its old resources and the new resource which is requested by the process.

Imposing these criterions may introduce negative impacts like low resource utilisation and starvation of processes.

Livelock The *Livelock* condition is similar to the deadlock condition except that a process in livelock condition changes its state with time. While in deadlock a process enters in wait state for a resource and continues in that state forever without making any progress in the execution, in a livelock condition a process always does something but is unable to make any progress in the execution completion. The livelock condition is better explained with the real world example, two people attempting to cross each other in a narrow corridor. Both the persons move towards each side of the corridor to allow the opposite person to cross. Since the corridor is narrow, none of them are able to cross each other. Here both of the persons perform some action but still they are unable to achieve their target, cross each other. We will make the livelock, the scenario more clear in a later section—*The Dining Philosophers' Problem*, of this chapter.

Starvation In the multitasking context, *starvation* is the condition in which a process does not get the resources required to continue its execution for a long time. As time progresses the process starves on resource. Starvation may arise due to various conditions like byproduct of preventive measures of deadlock, scheduling policies favouring high priority tasks and tasks with shortest execution time, etc.

10.8.1.3 The Dining Philosophers' Problem The '*Dining philosophers' problem*' is an interesting example for synchronisation issues in resource utilisation. The terms '*dining*', '*philosophers*', etc. may sound awkward in the operating system context, but it is the best way to explain technical things abstractly using non-technical terms. Now coming to the problem definition:

Five philosophers (It can be '*n*'. The number 5 is taken for illustration) are sitting around a round table, involved in eating and brainstorming (Fig. 10.37). At any point of time each philosopher will be in any one of the three states: eating, hungry or brainstorming. (While eating the philosopher is not involved in brainstorming and while brainstorming the philosopher is not involved in eating). For eating, each philosopher requires 2 forks. There are only 5 forks available on the dining table ('*n*' for '*n*' number of philosophers) and they are arranged in a fashion one fork in between two philosophers. The philosopher can only use the forks on his/her immediate left and right that too in the order pickup the left fork first and then the right fork. Analyse the situation and explain the possible outcomes of this scenario.

Let's analyse the various scenarios that may occur in this situation.

Scenario 1: All the philosophers involve in brainstorming together and try to eat together. Each philosopher picks up the left fork and is unable to proceed since two forks are required for eating the spaghetti present in the plate. Philosopher 1 thinks that Philosopher 2 sitting to the right of him/her will put the fork down and waits for it. Philosopher 2 thinks that Philosopher 3 sitting to the right of him/her will

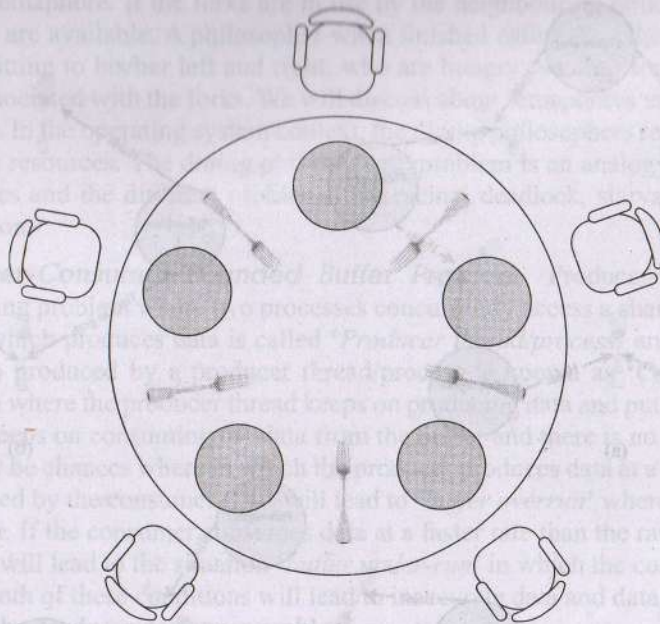


Fig. 10.27 Visualisation of the 'Dining Philosophers problem'

put the fork down and waits for it, and so on. This forms a circular chain of un-granted requests. If the philosophers continue in this state waiting for the fork from the philosopher sitting to the right of each, they will not make any progress in eating and this will result in *starvation* of the philosophers and *deadlock*.

Scenario 2: All the philosophers start brainstorming together. One of the philosophers is hungry and he/she picks up the left fork. When the philosopher is about to pick up the right fork, the philosopher sitting to his right also become hungry and tries to grab the left fork which is the right fork of his neighbouring philosopher who is trying to lift it, resulting in a 'Race condition'.

Scenario 3: All the philosophers involve in brainstorming together and try to eat together. Each philosopher picks up the left fork and is unable to proceed, since two forks are required for eating the spaghetti present in the plate. Each of them anticipates that the adjacently sitting philosopher will put his/her fork down and waits for a fixed duration and after this puts the fork down. Each of them again tries to lift the fork after a fixed duration of time. Since all philosophers are trying to lift the fork at the same time, none of them will be able to grab two forks. This condition leads to *livelock* and *starvation* of philosophers, where each philosopher tries to do something, but they are unable to make any progress in achieving the target.

Figure 10.28 illustrates these scenarios.

Solution: We need to find out alternative solutions to avoid the *deadlock*, *livelock*, *racing* and *starvation* condition that may arise due to the concurrent access of forks by philosophers. This situation can be handled in many ways by allocating the forks in different allocation techniques including Round Robin allocation, FIFO allocation, etc. But the requirement is that the solution should be optimal, avoiding

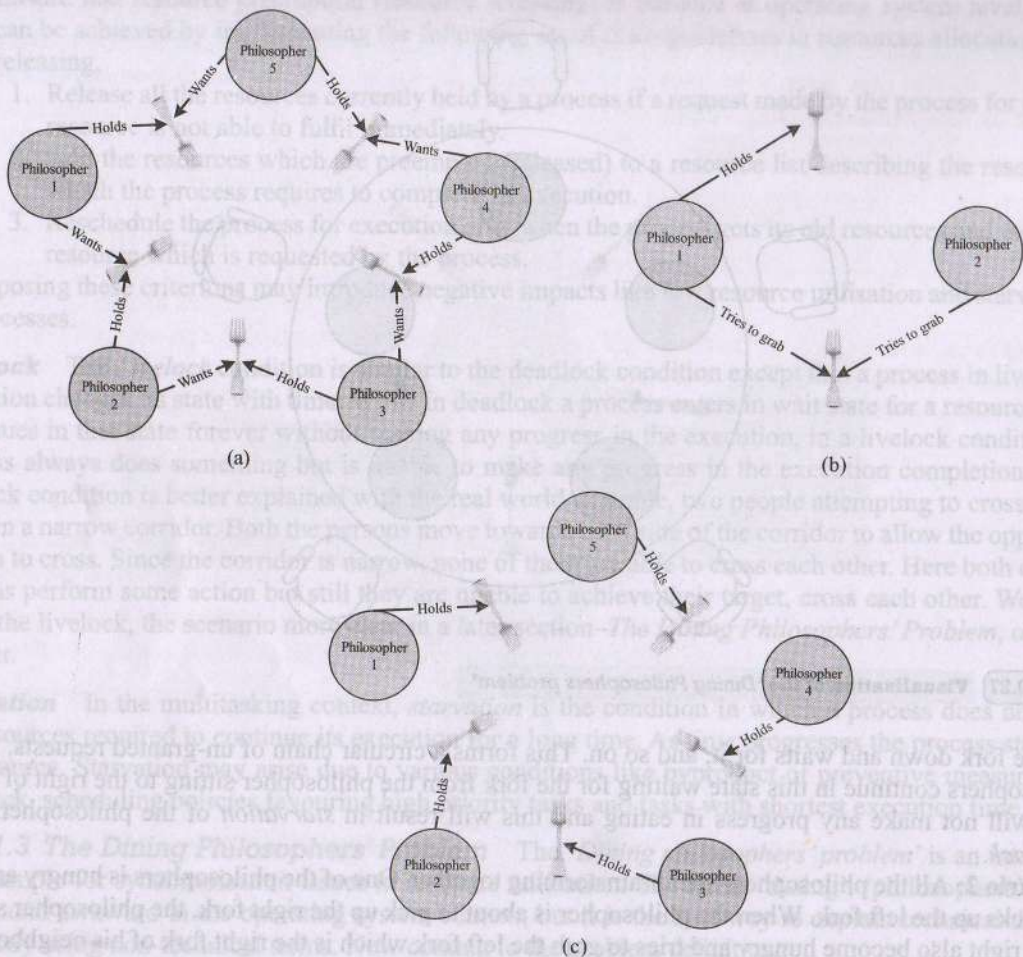


Fig. 10.28 The 'Real Problems' in the 'Dining Philosophers problem' (a) Starvation and Deadlock (b) Racing (c) Livelock and Starvation

deadlock and starvation of the philosophers and allowing maximum number of philosophers to eat at a time. One solution that we could think of is:

- Imposing rules in accessing the forks by philosophers, like: The philosophers should put down the fork he/she already have in hand (left fork) after waiting for a fixed duration for the second fork (right fork) and should wait for a fixed time before making the next attempt.

This solution works fine to some extent, but, if all the philosophers try to lift the forks at the same time, a *livelock* situation is resulted.

Another solution which gives maximum concurrency that can be thought of is each philosopher acquires a semaphore (mutex) before picking up any fork. When a philosopher feels hungry he/she checks whether the philosopher sitting to the left and right of him is already using the fork, by checking the state

of the associated semaphore. If the forks are in use by the neighbouring philosophers, the philosopher waits till the forks are available. A philosopher when finished eating puts the forks down and informs the philosophers sitting to his/her left and right, who are hungry (waiting for the forks), by signalling the semaphores associated with the forks. We will discuss about semaphores and mutexes at a latter section of this chapter. In the operating system context, the dining philosophers represent the processes and forks represent the resources. The dining philosophers' problem is an analogy of processes competing for shared resources and the different problems like racing, deadlock, starvation and livelock arising from the competition.

10.8.1.4 Producer-Consumer/Bounded Buffer Problem Producer-Consumer problem is a common data sharing problem where two processes concurrently access a shared buffer with fixed size. A thread/process which produces data is called '*Producer thread/process*' and a thread/process which consumes the data produced by a producer thread/process is known as '*Consumer thread/process*'. Imagine a situation where the producer thread keeps on producing data and puts it into the buffer and the consumer thread keeps on consuming the data from the buffer and there is no synchronisation between the two. There may be chances where in which the producer produces data at a faster rate than the rate at which it is consumed by the consumer. This will lead to '*buffer overrun*' where the producer tries to put data to a full buffer. If the consumer consumes data at a faster rate than the rate at which it is produced by the producer, it will lead to the situation '*buffer under-run*' in which the consumer tries to read from an empty buffer. Both of these conditions will lead to inaccurate data and data loss. The following code snippet illustrates the producer-consumer problem

```
#include <windows.h>
#include <stdio.h>
#define N 20 //Define buffer size as 20
int buffer[N]; //Shared buffer for producer & consumer
//*****
//Producer thread
void producer_thread(void) {
    int x;
    while(true) {
        for(x=0;x<N;x++)
        {
            //Fill buffer with random data
            buffer[x]= rand()%1000;
            printf("Produced : Buffer [%d] = %4d\n", x,
            buffer[x]);
            Sleep(25);
        }
    }
}
//*****
//Consumer thread
void consumer_thread(void) {
    int y=0,value;
    while(true) {
        for(y=0;y<N;y++)
```



```

    {
        value=buffer[y];
        printf("Consumed : Buffer [%d] = %4d\n", y, value);
        Sleep(20);
    }
}
//*****
//Main Thread
int main()
{
    DWORD thread_id;
    //Create Producer thread
    CreateThread(NULL,0,
                (LPTHREAD_START_ROUTINE)producer_thread,
                NULL,0,&thread_id);
    //Create Consumer thread
    CreateThread(NULL,0,
                (LPTHREAD_START_ROUTINE)consumer_thread,
                NULL,0,&thread_id);
    //Wait for some time and exit
    Sleep(500);
    return 0;
}

```

Here the 'producer thread' produces random numbers and puts it in a buffer of size 20. If the 'producer thread' fills the buffer fully it re-starts the filling of the buffer from the bottom. The 'consumer thread' consumes the data produced by the 'producer thread'. For consuming the data, the 'consumer thread' reads the buffer which is shared with the 'producer thread'. Once the 'consumer thread' consumes all the data, it starts consuming the data from the bottom of the buffer. These two threads run independently and are scheduled for execution based on the scheduling policies adopted by the OS. The different situations that may arise based on the scheduling of the 'producer thread' and 'consumer thread' is listed below.

1. 'Producer thread' is scheduled more frequently than the 'consumer thread': There are chances for overwriting the data in the buffer by the 'producer thread'. This leads to inaccurate data.
2. 'Consumer thread' is scheduled more frequently than the 'producer thread': There are chances for reading the old data in the buffer again by the 'consumer thread'. This will also lead to inaccurate data.

The output of the above program when executed on a Windows XP machine is shown in Fig. 10.29.

The output shows that the consumer thread runs faster than the producer thread and most often leads to buffer under-run and thereby inaccurate data.

Note

It should be noted that the scheduling of the threads '*producer_thread*' and '*consumer_thread*' is OS kernel scheduling policy dependent and you may not get the same output all the time when you run this piece of code in Windows XP.

The producer-consumer problem can be rectified in various methods. One simple solution is the '*sleep and wake-up*'. The '*sleep and wake-up*' can be implemented in various process synchronisation techniques like semaphores, mutex, monitors, etc. We will discuss it in a latter section of this chapter.


```

C:\Program Files\Microsoft Visual Studio\MyProjects\consume\Debug\consume.exe
Produced : Buffer 10 = 41
Consumed : Buffer 10 = 0
Produced : Buffer 11 = 467
Consumed : Buffer 11 = 0
Produced : Buffer 12 = 334
Consumed : Buffer 12 = 0
Produced : Buffer 13 = 500
Consumed : Buffer 13 = 0
Produced : Buffer 14 = 169
Consumed : Buffer 14 = 0
Produced : Buffer 15 = 724
Consumed : Buffer 15 = 0
Produced : Buffer 16 = 478
Consumed : Buffer 16 = 0
Produced : Buffer 17 = 358
Consumed : Buffer 17 = 0
Produced : Buffer 18 = 962
Consumed : Buffer 18 = 0
Produced : Buffer 19 = 464
Consumed : Buffer 19 = 0
Produced : Buffer 20 = 705
Consumed : Buffer 20 = 0
Produced : Buffer 21 = 145
Consumed : Buffer 21 = 0
Produced : Buffer 22 = 281
Consumed : Buffer 22 = 0
Produced : Buffer 23 = 827
Consumed : Buffer 23 = 0
Produced : Buffer 24 = 961

```

Fig. 10.29 Output of Win32 program illustrating producer consumer problem

10.8.1.5 Readers-Writers Problem The Readers-Writers problem is a common issue observed in processes competing for limited shared resources. The Readers-Writers problem is characterised by multiple processes trying to read and write shared data concurrently. A typical real-world example for the Readers-Writers problem is the banking system where one process tries to read the account information like available balance and the other process tries to update the available balance for that account. This may result in inconsistent results. If multiple processes try to read a shared data concurrently it may not create any impacts, whereas when multiple processes try to write and read concurrently it will definitely create inconsistent results. Proper synchronisation techniques should be applied to avoid the readers-writers problem. We will discuss about the various synchronisation techniques in a later section of this chapter.

10.8.1.6 Priority Inversion Priority inversion is the byproduct of the combination of blocking based (lock based) process synchronisation and pre-emptive priority scheduling. 'Priority inversion' is the condition in which a high priority task needs to wait for a low priority task to release a resource which is shared between the high priority task and the low priority task, and a medium priority task which doesn't require the shared resource continue its execution by preempting the low priority task (Fig. 10.30). Priority based preemptive scheduling technique ensures that a high priority task is always executed first, whereas the lock based process synchronisation mechanism (like mutex, semaphore, etc.) ensures that a process will not access a shared resource, which is currently in use by another process. The synchronisation technique is only interested in avoiding conflicts that may arise due to the concurrent access of the shared resources and not at all bothered about the priority of the process which tries to access the shared resource. In fact, the priority based preemption and lock based synchronisation are the two contradicting OS primitives. Priority inversion is better explained with the following scenario:

Let Process A, Process B and Process C be three processes with priorities High, Medium and Low respectively. Process A and Process C share a variable 'X' and the access to this variable is synchronised

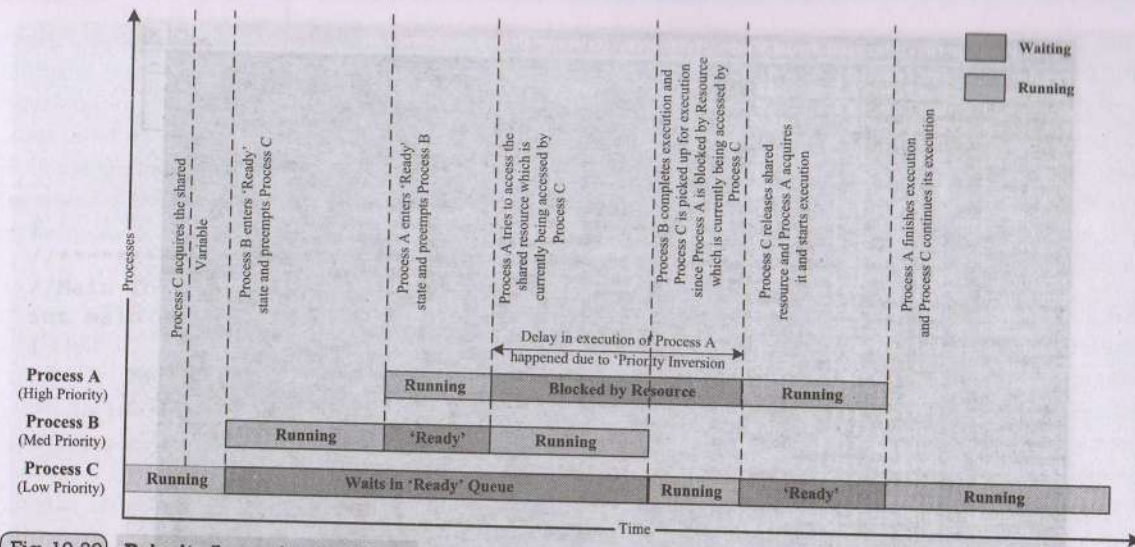
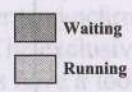


Fig. 10.30 Priority Inversion Problem

through a mutual exclusion mechanism like *Binary Semaphore S*. Imagine a situation where Process C is ready and is picked up for execution by the scheduler and 'Process C' tries to access the shared variable 'X'. 'Process C' acquires the 'Semaphore S' to indicate the other processes that it is accessing the shared variable 'X'. Immediately after 'Process C' acquires the 'Semaphore S', 'Process B' enters the 'Ready' state. Since 'Process B' is of higher priority compared to 'Process C', 'Process C' is preempted and 'Process B' starts executing. Now imagine 'Process A' enters the 'Ready' state at this stage. Since 'Process A' is of higher priority than 'Process B', 'Process B' is preempted and 'Process A' is scheduled for execution. 'Process A' involves accessing of shared variable 'X' which is currently being accessed by 'Process C'. Since 'Process C' acquired the semaphore for signalling the access of the shared variable 'X', 'Process A' will not be able to access it. Thus 'Process A' is put into blocked state (This condition is called Pending on resource). Now 'Process B' gets the CPU and it continues its execution until it relinquishes the CPU voluntarily or enters a wait state or preempted by another high priority task. The highest priority process 'Process A' has to wait till 'Process C' gets a chance to execute and release the semaphore. This produces unwanted delay in the execution of the high priority task which is supposed to be executed immediately when it was 'Ready'.

Priority inversion may be sporadic in nature but can lead to potential damages as a result of missing critical deadlines. Literally speaking, priority inversion 'inverts' the priority of a high priority task with that of a low priority task. Proper workaround mechanism should be adopted for handling the priority inversion problem. The commonly adopted priority inversion workarounds are:

Priority Inheritance: A low-priority task that is currently accessing (by holding the lock) a shared resource requested by a high-priority task temporarily '*inherits*' the priority of that high-priority task, from the moment the high-priority task raises the request. Boosting the priority of the low priority task to that of the priority of the task which requested the shared resource holding by the low priority task eliminates the preemption of the low priority task by other tasks whose priority are below that of the task requested the shared resource and thereby reduces the delay in waiting to get the resource requested by the high priority task. The priority of the low priority task which is temporarily boosted to high is

10^{-10} g^{-1} 

© Cambridge University Press

run-time. Implementation of 'priority ceiling' workaround in the priority inversion problem discussed for Process A, Process B and Process C example will change the execution sequence as shown in Fig. 10.32.

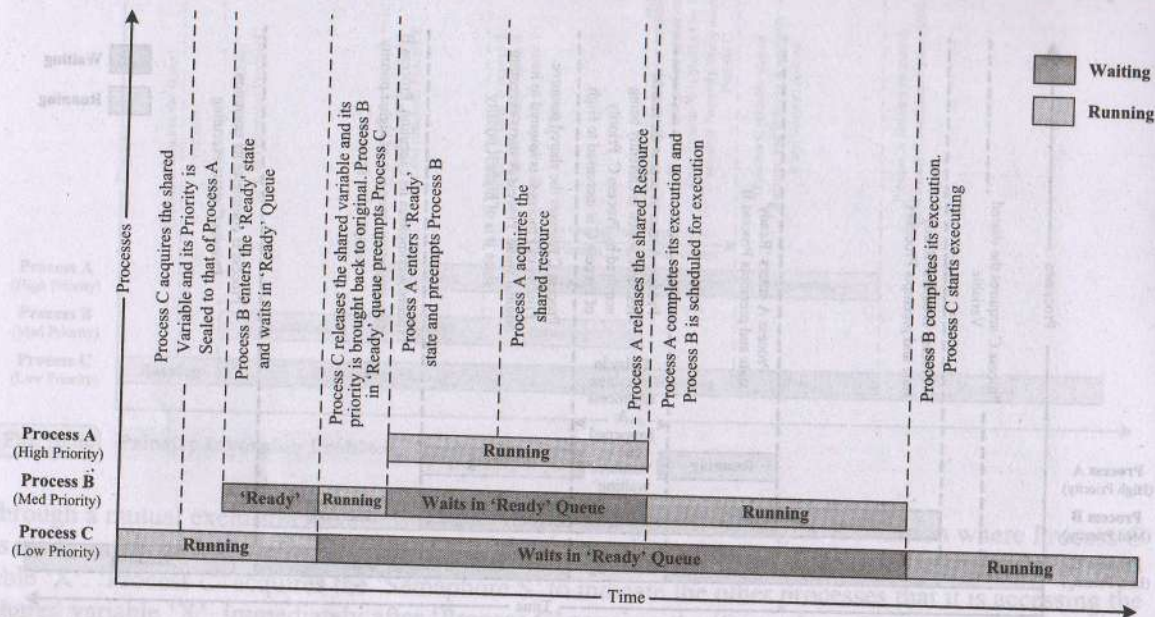


Fig. 10.32 Handling Priority Inversion Problem with Priority Ceiling

The biggest drawback of 'Priority Ceiling' is that it may produce *hidden priority inversion*. With 'Priority Ceiling' technique, the priority of a task is always elevated no matter another task wants the shared resources. This unnecessary priority elevation always boosts the priority of a low priority task to that of the highest priority tasks among which the resource is shared and other tasks with priorities higher than that of the low priority task is not allowed to preempt the low priority task when it is accessing a shared resource. This always gives the low priority task the luxury of running at high priority when accessing shared resources 😊.

10.8.2 Task Synchronisation Techniques

So far we discussed about the various task/process synchronisation issues encountered in multitasking systems due to concurrent resource access. Now let's have a discussion on the various techniques used for synchronisation in concurrent access in multitasking. Process/Task synchronisation is essential for

1. Avoiding conflicts in resource access (racing, deadlock, starvation, livelock, etc.) in a multitasking environment.
2. Ensuring proper sequence of operation across processes. The producer consumer problem is a typical example for processes requiring proper sequence of operation. In producer consumer problem, accessing the shared buffer by different processes is not the issue, the issue is the writing process should write to the shared buffer only if the buffer is not full and the consumer thread

should not read from the buffer if it is empty. Hence proper synchronisation should be provided to implement this sequence of operations.

3. Communicating between processes.

The code memory area which holds the program instructions (piece of code) for accessing a shared resource (like shared memory, shared variables, etc.) is known as '*critical section*'. In order to synchronise the access to shared resources, the access to the critical section should be exclusive. The exclusive access to critical section of code is provided through mutual exclusion mechanism. Let us have a look at how mutual exclusion is important in concurrent access. Consider two processes *Process A* and *Process B* running on a multitasking system. *Process A* is currently running and it enters its critical section. Before *Process A* completes its operation in the critical section, the scheduler preempts *Process A* and schedules *Process B* for execution (*Process B* is of higher priority compared to *Process A*). *Process B* also contains the access to the critical section which is already in use by *Process A*. If *Process B* continues its execution and enters the critical section which is already in use by *Process A*, a racing condition will be resulted. A mutual exclusion policy enforces mutually exclusive access of critical sections.

Mutual exclusions can be enforced in different ways. Mutual exclusion blocks a process. Based on the behaviour of the blocked process, mutual exclusion methods can be classified into two categories. In the following section we will discuss them in detail.

10.8.2.1 Mutual Exclusion through Busy Waiting/Spin Lock '*Busy waiting*' is the simplest method for enforcing mutual exclusion. The following code snippet illustrates how '*Busy waiting*' enforces mutual exclusion.

```
//Inside parent thread/main thread corresponding to a process
bool bFlag; //Global declaration of lock Variable.
bFlag= FALSE; //Initialise the lock to indicate it is available.
//.....
//Inside the child threads/threads of a process
while(bFlag == TRUE); //Check the lock for availability
bFlag=TRUE; //Lock is available. Acquire the lock
//Rest of the source code dealing with shared resource access
```

The '*Busy waiting*' technique uses a lock variable for implementing mutual exclusion. Each process/thread checks this lock variable before entering the critical section. The lock is set to '*1*' by a process/thread if the process/thread is already in its critical section; otherwise the lock is set to '*0*'. The major challenge in implementing the lock variable based synchronisation is the non-availability of a single atomic instruction[†] which combines the reading, comparing and setting of the lock variable. Most often the three different operations related to the locks, viz. the operation of Reading the lock variable, checking its present value and setting it are achieved with multiple low level instructions. The low level implementation of these operations are dependent on the underlying processor instruction set and the (cross) compiler in use. The low level implementation of the '*Busy waiting*' code snippet, which we discussed earlier, under Windows XP operating system running on an *Intel Centrino Duo* processor is given below. The code snippet is compiled with Microsoft Visual Studio 6.0 compiler.

```
--- D:\Examples\counter.cpp -----
1:  #include <stdio.h>
2:  #include <windows.h>
```

[†] Atomic Instruction: Instruction whose execution is uninterruptible.


```

3:
4:   int main()
5:   {
//Code memory      Opcode      Operand
00401010      push      ebp
00401011      mov       ebp,esp
00401013      sub       esp,44h
00401016      push      ebx
00401017      push      esi
00401018      push      edi
00401019      lea       edi,[ebp-44h]
0040101C      mov       ecx,11h
00401021      mov       eax,0CCCCCCCch
00401026      rep stos   dword ptr [edi]
6:   //Inside parent thread/ main thread corresponding to a process
7:   bool bFlag; //Global declaration of lock Variable.
8:   bFlag= FALSE; //Initialise the lock to indicate it is //available.
00401028      mov       byte ptr [ebp-4],0
9:   //.....
10:  //Inside the child threads/ threads of a process
11:  while(bFlag == TRUE); //Check the lock for availability
0040102C      mov       eax,dword ptr [ebp-4]
0040102F      and       eax,0FFh
00401034      cmp       eax,1
00401037      jne       main+2Bh (0040103b)
00401039      jmp       main+1Ch (0040102c)
12:  bFlag=TRUE; //Lock is available. Acquire the lock
0040103B      mov       byte ptr [ebp-4],1

```

The assembly language instructions reveals that the two high level instructions (*while(bFlag==false);* and *bFlag=true;*), corresponding to the operation of reading the lock variable, checking its present value and setting it is implemented in the processor level using six low level instructions. Imagine a situation where 'Process 1' read the lock variable and tested it and found that the lock is available and it is about to set the lock for acquiring the critical section (Fig. 10.33). But just before 'Process 1' sets the lock variable, 'Process 2' preempts 'Process 1' and starts executing. 'Process 2' contains a critical section code and it tests the lock variable for its availability. Since 'Process 1' was unable to set the lock variable, its state is still '0' and 'Process 2' sets it and acquires the critical section. Now the scheduler preempts 'Process 2' and schedules 'Process 1' before 'Process 2' leaves the critical section. Remember, 'Process 1' was preempted at a point just before setting the lock variable ('Process 1' has already tested the lock variable just before it is preempted and found that the lock is available). Now 'Process 1' sets the lock variable and enters the critical section☺. It violates the mutual exclusion policy and may produce unpredictable results.

The above issue can be effectively tackled by combining the actions of reading the lock variable, testing its state and setting the lock into a single step. This can be achieved with the combined hardware and software support. Most of the processors support a single instruction '*Test and Set Lock (TSL)*' for testing and setting the lock variable. The '*Test and Set Lock (TSL)*' instruction call copies the value of the lock variable and sets it to a nonzero value. It should be noted that the implementation and usage of

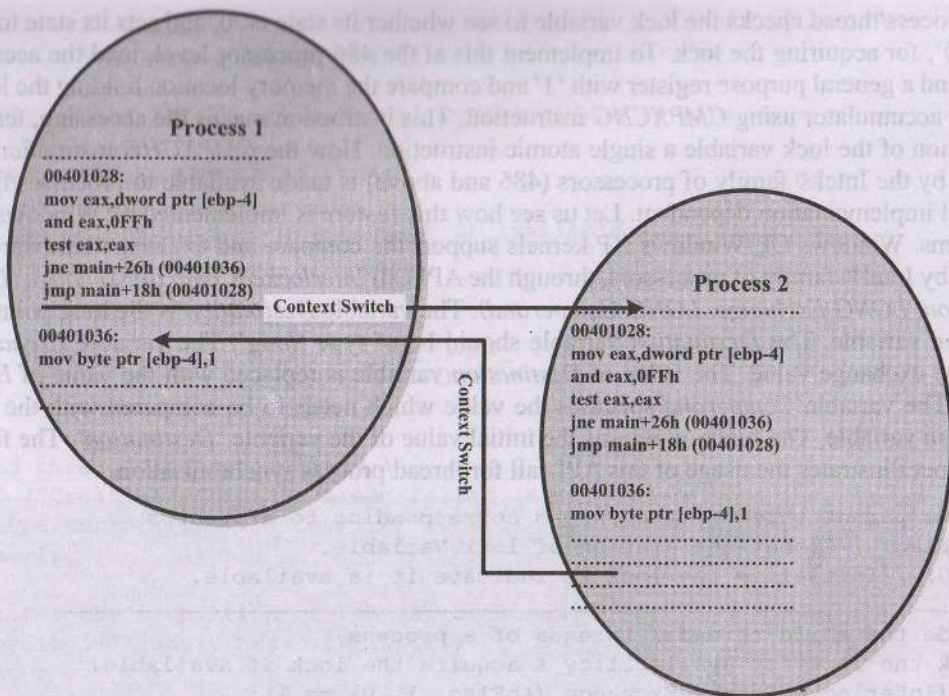


Fig. 10.33 Issue with locks

'Test and Set Lock (TSL)' instruction is processor architecture dependent. The *Intel 486* and the above family of processors support the 'Test and Set Lock (TSL)' instruction with a special instruction *CMPXCHG*—Compare and Exchange. The usage of *CMPXCHG* instruction is given below.

```
CMPXCHG dest,src
```

This instruction compares the Accumulator (*EAX* register) with 'dest'. If the Accumulator and 'dest' contents are equal, 'dest' is loaded with 'src'. If not, the Accumulator is loaded with 'dest'. Executing this instruction changes the six status bits of the Program Control and Status register *EFLAGS*. The destination ('dest') can be a register or a memory location. The source ('src') is always a register. From a programmer's perspective the operation of *CMPXCHG* instruction can be viewed as:

```

if (accumulator == destination)
{
    ZF = 1; //Set the Zero Flag of EFLAGS Register
    destination = source;
}
else
{
    ZF = 0; //Reset the Zero Flag of EFLAGS Register
    accumulator = destination;
}

```


The process/thread checks the lock variable to see whether its state is '0' and sets its state to '1' if its state is '0', for acquiring the lock. To implement this at the 486 processor level, load the accumulator with '0' and a general purpose register with '1' and compare the memory location holding the lock variable with accumulator using *CMPXCHG* instruction. This instruction makes the accessing, testing and modification of the lock variable a single atomic instruction. How the *CMPXCHG* instruction support provided by the Intel® family of processors (486 and above) is made available to processes/threads is OS kernel implementation dependent. Let us see how this feature is implemented by Windows Operating systems. Windows CE/Windows XP kernels support the compare and exchange hardware feature provided by Intel® family of processors, through the API call *InterlockedCompareExchange* (*LPLONG Destination, LONG Exchange, LONG Comperand*). The variable *Destination* is the long pointer to the destination variable. The *Destination* variable should be of type 'long'. The variable *Exchange* represents the exchange value. The value of *Destination* variable is replaced with the value of *Exchange* variable. The variable *Comperand* specifies the value which needs to be compared with the value of *Destination* variable. The function returns the initial value of the variable '*Destination*'. The following code snippet illustrates the usage of this API call for thread/process synchronisation.

```
//Inside parent thread/ main thread corresponding to a process
long bFlag; //Global declaration of lock Variable.
bFlag=0; //Initialise the lock to indicate it is available.
//.....
//Inside the child threads/ threads of a process
//Check the lock for availability & acquire the lock if available.
while (InterlockedCompareExchange (&bFlag, 1, 0) == 1);

//Rest of the source code dealing with shared resource access
```

The *InterlockedCompareExchange* function is implemented as '*Compiler intrinsic function*'. The 'code for *Compiler intrinsic functions*' are inserted inline while compiling the code. This avoids the function call overhead and makes use of the built-in knowledge of the optimisation technique for intrinsic functions. The compiler can be instructed to use the intrinsic implementation for a function using the compiler directive *#pragma intrinsic (intrinsic-function-name)*. A sample implementation of the *InterlockedCompareExchange* interlocked intrinsic function for Windows XP OS is given below.

```
#include "stdafx.h"
#include <intrin.h>
#include <windows.h>
long bFlag; //Global declaration of lock Variable.
//Declare InterlockedCompareExchange as intrinsic function
#pragma intrinsic( InterlockedCompareExchange)
void child_thread(void)
{
//Inside the child thread of a process
//Check the lock for availability & acquire the lock if available.
//The lock can be set by any other threads
while (_InterlockedCompareExchange (&bFlag, 1, 0) == 1);
//Rest of the source code dealing with shared resource access
//.....
return;
}
```



```

//.....
int _tmain(int argc, _TCHAR* argv[])
{
//Inside parent thread/ main thread corresponding to a process
DWORD thread_id;
//Define handle to the child thread
HANDLE tThread;
//Initialize the lock to indicate it is available.
bFlag =0;
//Create child thread
tThread = CreateThread (NULL,0,
(LPTHREAD_START_ROUTINE) child_thread,
NULL, 0, &thread_id);

if(NULL== tThread)
{
//Child thread creation failed.
printf ("Creation of Child thread failed. Error Code =
%d",GetLastError());
return -1;
}
//Wait for the completion of the child thread.
WaitForSingleObject(tThread,INFINITE);
return 0;
}

```

Note: Visual Studio 2005 or a later version of the compiler, which supports interlocked intrinsic functions, is required for compiling this application. The assembly code generated for the intrinsic interlocked function `while ( InterlockedCompareExchange (&bFlag, 1, 0) == 1);` when compiled using Visual Studio 2005 compiler, on Windows XP platform with Service Pack 2 running on an Intel® Centrino® Duo processor is given below. It clearly depicts the usage of the `cmpxchg` instruction

```

//Inside the child threads/ threads of a process
//Check the lock for availability & acquire the lock if available.
//The lock can be set by any other threads
while ( _InterlockedCompareExchange (&bFlag, 1, 0) == 1);
004113DE      mov     ecx,1
004113E3      mov     edx,offset bFlag (417178h)
004113E8      xor     eax,eax
004113EA      lock    cmpxchg dword ptr [edx],ecx
004113EE      cmp     eax,1
004113F1      jne     child_thread+35h (4113F5h)
004113F3      jmp     child_thread+1Eh (4113DEh)
//Rest of the source code dealing with shared resource access
//.....

```

The Intel 486 and above family of processors provide hardware level support for atomic execution of increment and decrement operations also. The `XADD` low level instruction implements atomic execution of increment and decrement operations. Windows CE/XP kernel makes these features available to the users through a set of *Interlocked* function API calls. The API call *InterlockedIncrement* (`LPLONG`

lpAddend) increments the value of the variable pointed by *lpAddend* and the API *InterlockedDecrement* (*LPLONG lpAddend*) decrements the value of the variable pointed by *lpAddend*.

The lock based mutual exclusion implementation always checks the state of a lock and waits till the lock is available. This keeps the processes/threads always busy and forces the processes/threads to wait for the availability of the lock for proceeding further. Hence this synchronisation mechanism is popularly known as '*Busy waiting*'. The '*Busy waiting*' technique can also be visualised as a lock around which the process/thread spins, checking for its availability. Spin locks are useful in handling scenarios where the processes/threads are likely to be blocked for a shorter period of time on waiting the lock, as they avoid OS overheads on context saving and process re-scheduling. Another drawback of Spin lock based synchronisation is that if the lock is being held for a long time by a process and if it is preempted by the OS, the other threads waiting for this lock may have to spin a longer time for getting it. The '*Busy waiting*' mechanism keeps the process/threads always active, performing a task which is not useful and leads to the wastage of processor time and high power consumption.

The interlocked operations are the most efficient synchronisation primitives when compared to the classic lock based synchronisation mechanism. Interlocked function based synchronisation technique brings the following value adds.

- The interlocked operation is free from waiting. Unlike the mutex, semaphore and critical section synchronisation objects which may require waiting on the object, if they are not available at the time of request, the interlocked function simply performs the operation and returns immediately. This avoids the blocking of the thread which calls the interlocked function.
- The interlocked function call is directly converted to a processor specific instruction and there is no user mode to kernel mode transition as in the case of mutex, semaphore and critical section objects. This avoids the user mode to kernel mode transition delay and thereby increases the overall performance.

The types of interlocked operations supported by an OS are underlying processor hardware dependent and so they are limited in functionality. Normally the bit manipulation (Boolean) operations are not supported by interlocked functions. Also the interlocked operations are limited to integer or pointer variables only. This limits the possibility of extending the interlocked functions to variables of other types. Under windows operating systems, each process has its own virtual address space and so the interlocked functions can only be used for synchronising the access to a variable that is shared by multiple threads of a process (Multiple threads of a process share the same address space) (Intra Process Synchronisation). The interlocked functions can be extended for synchronising the access of the variables shared across multiple processes if the variable is kept in shared memory.

10.8.2.2 Mutual Exclusion through Sleep & Wakeup The '*Busy waiting*' mutual exclusion enforcement mechanism used by processes makes the CPU always busy by checking the lock to see whether they can proceed. This results in the wastage of CPU time and leads to high power consumption. This is not affordable in embedded systems powered on battery, since it affects the battery backup time of the device. An alternative to '*busy waiting*' is the '*Sleep & Wakeup*' mechanism. When a process is not allowed to access the critical section, which is currently being locked by another process, the process undergoes '*Sleep*' and enters the '*blocked*' state. The process which is blocked on waiting for access to the critical section is awakened by the process which currently owns the critical section. The process which owns the critical section sends a wakeup message to the process, which is sleeping as a result of waiting for the access to the critical section, when the process leaves the critical section. The '*Sleep & Wakeup*' policy for mutual exclusion can be implemented in different ways. Implementation of

this policy is OS kernel dependent. The following section describes the important techniques for 'Sleep & Wakeup' policy implementation for mutual exclusion by Windows XP/CE OS kernels.

Semaphore Semaphore is a sleep and wakeup based mutual exclusion implementation for shared resource access. Semaphore is a system resource and the process which wants to access the shared resource can first acquire this system object to indicate the other processes which wants the shared resource that the shared resource is currently acquired by it. The resources which are shared among a process can be either for exclusive use by a process or for using by a number of processes at a time. The display device of an embedded system is a typical example for the shared resource which needs exclusive access by a process. The Hard disk (secondary storage) of a system is a typical example for sharing the resource among a limited number of multiple processes. Various processes can access the different sectors of the hard-disk concurrently. Based on the implementation of the sharing limitation of the shared resource, semaphores are classified into two; namely '*Binary Semaphore*' and '*Counting Semaphore*'. The binary semaphore provides exclusive access to shared resource by allocating the resource to a single process at a time and not allowing the other processes to access it when it is being owned by a process. The implementation of binary semaphore is OS kernel dependent. Under certain OS kernel it is referred as *mutex*. Unlike a binary semaphore, the '*Counting Semaphore*' limits the access of resources by a fixed number of processes/threads. '*Counting Semaphore*' maintains a count between zero and a

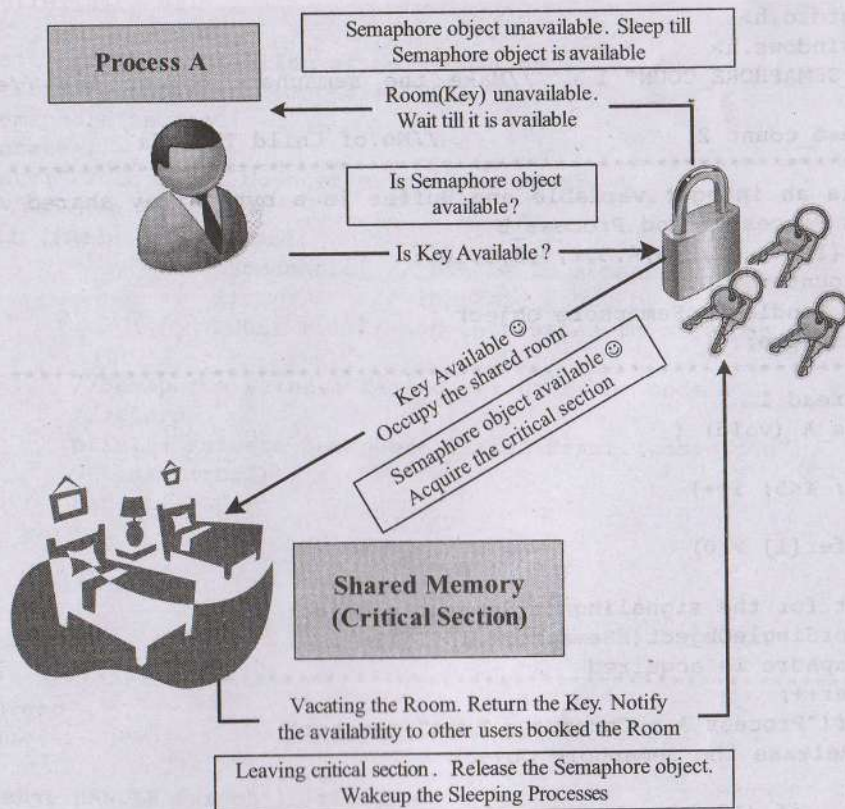


Fig. 10.34 The Concept of Counting Semaphore

value. It limits the usage of the resource to the maximum value of the count supported by it. The state of the counting semaphore object is set to 'signalled' when the count of the object is greater than zero. The count associated with a '*Semaphore object*' is decremented by one when a process/thread acquires it and the count is incremented by one when a process/thread releases the '*Semaphore object*'. The state of the '*Semaphore object*' is set to non-signalled when the semaphore is acquired by the maximum number of processes/threads that the semaphore can support (i.e. when the count associated with the '*Semaphore object*' becomes zero). A real world example for the counting semaphore concept is the dormitory system for accommodation (Fig. 10.34). A dormitory contains a fixed number of beds (say 5) and at any point of time it can be shared by the maximum number of users supported by the dormitory. If a person wants to avail the dormitory facility, he/she can contact the dormitory caretaker for checking the availability. If beds are available in the dorm the caretaker will hand over the keys to the user. If beds are not available currently, the user can register his/her name to get notifications when a slot is available. Those who are availing the dormitory shares the dorm facilities like TV, telephone, toilet, etc. When a dorm user vacates, he/she gives the keys back to the caretaker. The caretaker informs the users, who booked in advance, about the dorm availability.

The creation and usage of '*counting semaphore object*' is OS kernel dependent. Let us have a look at how we can implement semaphore based synchronisation for the '*Racing*' problem we discussed in the beginning, under the Windows kernel. The following code snippet explains the same.

```
#include <stdio.h>
#include <windows.h>
#define MAX_SEMAPHORE_COUNT 1    //Make the semaphore object for //exclusive
use
#define thread_count 2           //No.of Child Threads
//*****
//counter is an integer variable and Buffer is a byte array shared //between
two threads Process_A and Process_B
char Buffer[10] = {1,2,3,4,5,6,7,8,9,10};
short int counter = 0;
//Define the handle to Semaphore object
HANDLE hSemaphore;
//*****
// Child Thread 1
void Process_A (void) {
int i;
for (i =0; i<5; i++)
{
if (Buffer[i] > 0)
{
//Wait for the signaling of Semaphore object
WaitForSingleObject(hSemaphore,INFINITE);
//Semaphore is acquired
counter++;
printf("Process A : Counter = %d\n",counter);
//Release the Semaphore Object

if (!ReleaseSemaphore(
hSemaphore, // handle to semaphore
```



```

        1,          // increase count by one
        NULL))     // not interested in previous count
    {
        //Semaphore Release failed. Print Error code &
        return.
        printf("Release Semaphore Failed with Error Code:
        %d\n", GetLastError());
        return;
    }
}

return;
}

//*****
// Child Thread 2
void Process_B(void) {
    int j;
    for (j =5; j<10; j++)
    {
        if (Buffer[j] > 0)
        {
            //Wait for the signalling of Semaphore object
            WaitForSingleObject(hSemaphore,INFINITE);
            //Semaphore is acquired
            counter++;
            printf("Process B : Counter = %d\n",counter);
            //Release Semaphore
            if (!ReleaseSemaphore(
                hSemaphore, // handle to semaphore
                1,          // increase count by one
                NULL) )     // not interested in previous count
            {
                //Semaphore Release failed. Print Error code &
                //return.
                printf("Release Semaphore Failed Error Code: %d\n",
                GetLastError());
                return;
            }
        }
    }

    return;
}

//*****
// Main Thread
void main() {

    //Define HANDLE for child threads
    HANDLE child_threads[thread_count];

```



```

DWORD thread_id;
int i;

//Create Semaphore object
hSemaphore = CreateSemaphore(
    NULL,           // default security attributes
    MAX_SEMAPHORE_COUNT, // initial count. Create as signaled
    MAX_SEMAPHORE_COUNT, // maximum count
    "Semaphore"); // Semaphore object with name "Semaphore"

if (NULL == hSemaphore)
{
    printf ("Semaphore Object Creation Failed: Error Code: %d",
        GetLastError ());
    //Semaphore Object Creation failed. Return
    return;
}

//Create Child thread 1
child_threads[0]= CreateThread(NULL,0,
    (LPTHREAD_START_ROUTINE)Process_A,
    (LPVOID)0,0,&thread_id);

//Create Child thread 2
child_threads[1]= CreateThread(NULL,0,
    (LPTHREAD_START_ROUTINE)Process_B,
    (LPVOID)0,0,&thread_id);

//Check the success of creation of child threads
for (i=0;i<thread_count; i++)
{
    if(NULL==child_threads[i])
    {
        //Child thread creation failed.
        printf ("Child thread Creation failed with Error Code: %d",
            GetLastError ());
        return;
    }
}

// Wait for the termination of child threads
WaitForMultipleObjects(thread_count, child_threads, TRUE,
INFINITE);

//Close handles of child threads
for( i=0; i < thread_count; i++ )
    CloseHandle(child_threads[i]);

//Close Semaphore object handle
CloseHandle(hSemaphore);
return;
}

```

Please refer to the Online Learning Centre for details on the various Win32 APIs used in the program for counting semaphore creation, acquiring, signalling, and releasing. The VxWorks and MicroC/OS-II

Real-Time kernels also implements the Counting semaphore based task synchronisation/shared resource access. We will discuss them in detail in a later chapter.

Counting Semaphores are similar to *Binary Semaphores* in operation. The only difference between *Counting Semaphore* and *Binary Semaphore* is that *Binary Semaphore* can only be used for exclusive access, whereas *Counting Semaphores* can be used for both exclusive access (by restricting the maximum count value associated with the semaphore object to one (1) at the time of creation of the semaphore object) and limited access (by restricting the maximum count value associated with the semaphore object to the limited number at the time of creation of the semaphore object).

Binary Semaphore (Mutex) Binary Semaphore (Mutex) is a synchronisation object provided by OS for process/thread synchronisation. Any process/thread can create a '*mutex object*' and other processes/threads of the system can use this '*mutex object*' for synchronising the access to critical sections. Only one process/thread can own the '*mutex object*' at a time. The state of a mutex object is set to signalled when it is not owned by any process/thread, and set to non-signalled when it is owned by any process/thread. A real world example for the mutex concept is the hotel accommodation system (lodging system) Fig. 10.35. The rooms in a hotel are shared for the public. Any user who pays and follows the norms of the hotel can avail the rooms for accommodation. A person wants to avail the hotel room facility can contact the hotel reception for checking the room availability (see Fig. 10.35). If room is available the receptionist will handover the room key to the user. If room is not available currently, the user can book

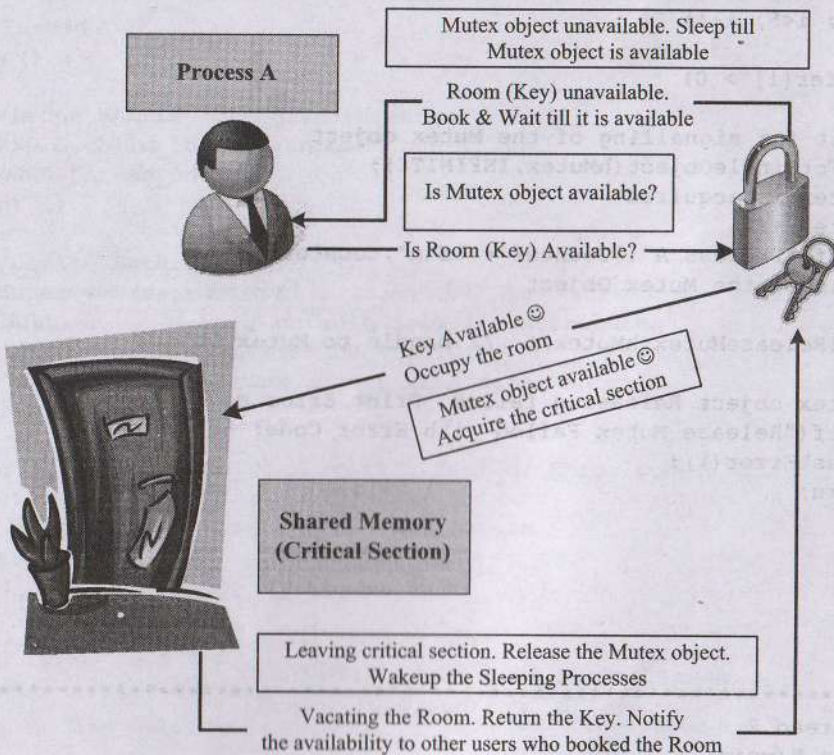


Fig. 10.35

The Concept of Binary Semaphore (Mutex)

the room to get notifications when a room is available. When a person gets a room he/she is granted the exclusive access to the room facilities like TV, telephone, toilet, etc. When a user vacates the room, he/she gives the keys back to the receptionist. The receptionist informs the users, who booked in advance, about the room's availability.

Let's see how we can implement mutual exclusion with mutex object in the 'Racing' problem example given under the section 'Racing', under Windows kernel.

```
#include <stdio.h>
#include <windows.h>
#define thread_count 2 //No.of Child Threads
//*****
//counter is an integer variable and Buffer is a byte array shared
//between two
//threads Process_A and Process_B
char Buffer[10] = {1,2,3,4,5,6,7,8,9,10};
short int counter = 0;
//Define the handle to Mutex Object
HANDLE hMutex;
//*****
// Child Thread 1
void Process_A (void) {
int i;
for (i =0; i<5; i++)
{
if (Buffer[i] > 0)
{
//Wait for signalling of the Mutex object
WaitForSingleObject(hMutex,INFINITE);
//Mutex is acquired
counter++;
printf("Process A : Counter = %d\n",counter);
//Release the Mutex Object

if (!ReleaseMutex(hMutex)) // handle to Mutex Object
{
//Mutex object Releasing failed. Print Error code & return.
printf("Release Mutex Failed with Error Code: %d\n",
GetLastError());
return;
}

}

}
return;
}
//*****
// Child Thread 2
void Process_B(void) {
int j;
```



```

for (j =5; j<10; j++)
{
    if (Buffer[j] > 0)
    {
        //Wait for signalling of the Mutex object
        WaitForSingleObject(hMutex, INFINITE);
        //Mutex object is acquired
        counter++;
        printf("Process B : Counter = %d\n", counter);
        //Release Mutex object
        if (!ReleaseMutex(hMutex)) // handle to Mutex Object
        {
            //Mutex object Release failed. Print Error code & return.
            printf("Release Mutex Failed with Error Code: %d\n",
                GetLastError());
            return;
        }
    }
}

return;
}

//*****
// Main Thread
void main() {

    //Define HANDLE for child threads
    HANDLE child_threads[thread_count];
    DWORD thread_id;
    int i;

    //Create Mutex object
    hMutex = CreateMutex(
        NULL, // default security attributes
        FALSE, // Not initial ownership
        "Mutex"); // Mutex object with name "Mutex"
    if (NULL == hMutex)
    {
        printf ("Mutex Object Creation Failed: Error Code: %d",
            GetLastError ());
        //Mutex Object Creation failed. Return
        return;
    }

    //Create Child thread 1
    child_threads[0]= CreateThread(NULL, 0,
        (LPTHREAD_START_ROUTINE)Process_A,
        (LPVOID)0, 0, &thread_id);
}

```



```

//Create Child thread 2
child_threads[1]=      CreateThread(NULL,0,
                        (LPTHREAD_START_ROUTINE)Process_B,
                        (LPVOID)0,0,&thread_id);

//Check the success of creation of child threads
for (i=0;i<thread_count; i++)
{
    if(NULL==child_threads[i])
    {
        //Child thread creation failed.
        printf ("Child thread Creation failed with Error Code: %d",
        GetLastError ());
        return;
    }
}

// Wait for the termination of child threads

WaitForMultipleObjects(thread_count, child_threads, TRUE,
INFINITE);

//Close child thread handles
for( i=0; i < thread_count; i++ )
    CloseHandle(child_threads[i]);
//Close Mutex object handle
CloseHandle(hMutex);
return;
}

```

Please refer to the Online Learning Centre for details on the various Win32 APIs used in the program for mutex creation, acquiring, signalling, and releasing.

The mutual exclusion semaphore is a special implementation of the binary semaphore by certain real-time operating systems like VxWorks and MicroC/OS-II to prevent priority inversion problems in shared resource access. The mutual exclusion semaphore has an option to set the priority of a task owning it to the highest priority of the task which is being pended while attempting to acquire the semaphore which is already in use by a low priority task. This ensures that the low priority task which is currently holding the semaphore, when a high priority task is waiting for it, is not pre-empted by a medium priority task. This is the mechanism supported by the mutual exclusion semaphore to prevent priority inversion.

VxWorks kernel also supports binary semaphores for synchronising shared resource access. We will discuss about it in detail in a later chapter.

Critical Section Objects In Windows CE, the '*Critical Section object*' is same as the '*mutex object*' except that '*Critical Section object*' can only be used by the threads of a single process (Intra process). The piece of code which needs to be made as '*Critical Section*' is placed at the '*Critical Section*' area by the process. The memory area which is to be used as the '*Critical Section*' is allocated by the process. The process creates a '*Critical Section*' area by creating a variable of type *CRITICAL_SECTION*. The '*Critical Section*' must be initialised before the threads of a process can use it for getting exclusive access. The *InitializeCriticalSection(LPCRITICAL_SECTION lpCriticalSection)* API initialises the critical section pointed by the pointer *lpCriticalSection* to the critical section. Once the critical section

is initialized, all threads in the process can use it. Threads can use the API call *EnterCriticalSection* (*LPCriticalSection lpCriticalSection*) for getting the exclusive ownership of the critical section pointed by the pointer *lpCriticalSection*. Calling the *EnterCriticalSection()* API blocks the execution of the caller thread if the critical section is already in use by other threads and the thread waits for the critical section object. Threads which are blocked by the *EnterCriticalSection()* call, waiting on a critical section are added to a wait queue and are woken when the critical section is available to the requested thread. The API call *TryEnterCriticalSection(LPCriticalSection lpCriticalSection)* attempts to enter the critical section pointed by the pointer *lpCriticalSection* without blocking the caller thread. If the critical section is not in use by any other thread, the calling thread gets the ownership of the critical section. If the critical section is already in use by another thread, the *TryEnterCriticalSection()* call indicates it to the caller thread by a specific return value and the thread resumes its execution. A thread can release the exclusive ownership of a critical section by calling the API *LeaveCriticalSection(LPCriticalSection lpCriticalSection)*. The threads of a process can use the API *DeleteCriticalSection(LPCriticalSection lpCriticalSection)* to release all resources used by a critical section object which was created by the process with the *CRITICAL_SECTION* variable.

Now let's have a look at the 'Racing' problem we discussed under the section 'Racing'. The racing condition can be eliminated by using a critical section object for synchronisation. The following code snippet illustrates the same.

```
#include <stdio.h>
#include <windows.h>
/*****
//counter is an integer variable and Buffer is a byte array shared
//between two threads
char Buffer[10] = {1,2,3,4,5,6,7,8,9,10};
short int counter = 0;
//Define the critical section
CRITICAL_SECTION CS;
/*****
// Child Thread 1
void Process_A (void) {
int i;
for (i =0; i<5; i++)
{
if (Buffer[i] > 0)
{
//Use critical section object for synchronisation
EnterCriticalSection(&CS);
counter++;
LeaveCriticalSection(&CS);
}
printf("Process A : Counter = %d\n",counter);
}
}
/*****
// Child Thread 2
void Process_B(void) {
```



```

int j;
for (j =5; j<10; j++)
{
    if (Buffer[j] > 0)
    {
        //Use critical section object for synchronisation
        EnterCriticalSection(&CS);
        counter++;
        LeaveCriticalSection(&CS);
    }
    printf("Process B : Counter = %d\n",counter);
}
}
//*****
// Main Thread

int main() {

    DWORD id;

    //Initialize critical section object
    InitializeCriticalSection(&CS);
    CreateThread(NULL,0,
                (LPTHREAD_START_ROUTINE) Process_A,
                (LPVOID)0,0,&id);
    CreateThread(NULL,0,
                (LPTHREAD_START_ROUTINE) Process_B,
                (LPVOID)0,0,&id);

    Sleep(100000);
    return 0;
}

```

Here the shared resource is the shared variable '*counter*'. The concurrent access to this variable by the threads '*Process_A*' and '*Process_B*' may create race condition and may produce incorrect results. The critical section object '*CS*' holds the piece of code corresponding to the access of the shared variable '*counter*' by each threads. This ensures that the memory area containing the low level instructions corresponding to the high level instruction '*counter++*' is accessed exclusively by threads '*Process_A*' and '*Process_B*' and avoids a race condition. The output of the above piece of code when executed on Windows XP machine is given in Fig. 10.36.

The final value of '*counter*' is obtained as 10, which is the expected result for this piece of code. If you observe this output window you can see that the text is not outputted to the o/p window in the expected manner. The *printf()* library routine used in this sample code is re-entrant and it can be pre-empted while in execution. That is why the outputting of text happened in a non expected way.

Note

It should be noted that the scheduling of the threads '*Process_A*' and '*Process_B*' is OS kernel scheduling policy dependent and you may not get the same output all the time when you run this piece of code under Windows XP.


```

C:\Program Files\Microsoft Visual Studio\MyProjects\
Process A : Counter = 1
Process A : Counter = 1
Process B : Counter = 2
Process A : Counter = 4
Process B : Counter = 4
Process B : Counter = 5
Process A : Counter = 6
Process B : Counter = 7
Process B : Counter = 9
Process A : Counter = 10

```

Fig. 10.36 Output of the Win32 application resolving racing condition through critical section object

The critical section object makes the piece of code residing inside it non-reentrant. Now let's try the above piece of code by putting the *printf()* library routine in the critical section object.

```

#include <stdio.h>
#include <windows.h>
//*****
//counter is an integer variable and Buffer is a byte array shared
//between two threads Process A and Process B
char Buffer[10] = {1,2,3,4,5,6,7,8,9,10};
short int counter = 0;
//Define the critical section
CRITICAL_SECTION CS;
//*****
// Child Thread 1
void Process_A (void) {
    int i;
    for (i = 0; i < 5; i++)
    {
        if (Buffer[i] > 0)
        {
            //Use critical section object for synchronisation
            EnterCriticalSection(&CS);
            counter++;
            printf("Process A : Counter = %d\n", counter);
            LeaveCriticalSection(&CS);
        }
    }
}

```



```

//*****
// Child Thread 2
void Process_B(void) {

int j;
    for (j = 5; j<10; j++)
    {
        if (Buffer[j] > 0)
        {
            //Use critical section object for synchronisation
            EnterCriticalSection(&CS);
            counter++;
            printf("Process B : Counter = %d\n",counter);
            LeaveCriticalSection(&CS);
        }
    }
}
//*****
// Main Thread

int main() {

    DWORD id;

    //Initialize critical section object
    InitializeCriticalSection(&CS);
    CreateThread(NULL,0,
                (LPTHREAD_START_ROUTINE)Process_A,
                (LPVOID)0,0,&id);
    CreateThread(NULL,0,
                (LPTHREAD_START_ROUTINE)Process_B,
                (LPVOID)0,0,&id);

    Sleep(100000);
    return 0;
}

```

The output of the above piece of code when executed on a Windows XP machine is given below.

```

C:\Program Files\Microsoft Visual Studio...
Process A : Counter = 1
Process A : Counter = 2
Process B : Counter = 3
Process A : Counter = 4
Process B : Counter = 5
Process A : Counter = 6
Process B : Counter = 7
Process A : Counter = 8
Process B : Counter = 9
Process B : Counter = 10

```

Fig. 10.37 Output of the Win32 application resolving racing condition through critical section object

Note

It should be noted that the scheduling of the threads '*Process_A*' and '*Process_B*' is OS kernel scheduling policy dependent and you may not get the same output all the time when you run this piece of code in Windows XP. The output of the above program when executed at three different instances of time is given shown in Fig. 10.38.

```

C:\Program Files\Microsoft Visual Studio\MyProjects\...
Process B : Counter = 1
Process B : Counter = 2
Process B : Counter = 3
Process B : Counter = 4
Process B : Counter = 5
Process A : Counter = 6
Process A : Counter = 7
Process A : Counter = 8
Process A : Counter = 9
Process A : Counter = 10

C:\Program Files\Microsoft Visual Studio\MyProjects\...
Process B : Counter = 1
Process A : Counter = 2
Process B : Counter = 3
Process B : Counter = 4
Process B : Counter = 5
Process B : Counter = 6
Process A : Counter = 7
Process A : Counter = 8
Process A : Counter = 9
Process A : Counter = 10

C:\Program Files\Microsoft Visual Studio\MyProjects\...
Process A : Counter = 1
Process A : Counter = 2
Process A : Counter = 3
Process A : Counter = 4
Process A : Counter = 5
Process B : Counter = 6
Process B : Counter = 7
Process B : Counter = 8
Process B : Counter = 9
Process B : Counter = 10
  
```

Fig. 10.38 Illustration of scheduler behaviour under Windows XP kernel

Events Event object is a synchronisation technique which uses the notification mechanism for synchronisation. In concurrent execution we may come across situations which demand the processes to wait for a particular sequence for its operations. A typical example of this is the producer consumer threads, where the consumer thread should wait for the consumer thread to produce the data and producer thread should wait for the consumer thread to consume the data before producing fresh data. If this sequence is not followed it will end up in producer-consumer problem. Notification mechanism is used for handling this scenario. Event objects are used for implementing notification mechanisms.

A thread/process can wait for an event and another thread/process can set this event for processing by the waiting thread/process. The creation and handling of event objects for notification is OS kernel dependent. Please refer to the Online Learning Centre for information on the usage of 'Events' under Windows Kernel for process/thread synchronisation.

The MicroC/OS-II kernel also uses 'events' for task synchronisation. We will discuss it in a later chapter.

10.9 DEVICE DRIVERS

Device driver is a piece of software that acts as a bridge between the operating system and the hardware. In an operating system based product architecture, the user applications talk to the Operating System kernel for all necessary information exchange including communication with the hardware peripherals. The architecture of the OS kernel will not allow direct device access from the user application. All the device related access should flow through the OS kernel and the OS kernel routes it to the concerned hardware peripheral. OS provides interfaces in the form of Application Programming Interfaces (APIs) for accessing the hardware. The device driver abstracts the hardware from user applications. The topology of user applications and hardware interaction in an RTOS based system is depicted in Fig. 10.39.

Device drivers are responsible for initiating and managing the communication with the hardware peripherals. They are responsible for establishing the connectivity, initialising the hardware (setting up various registers of the hardware device) and transferring data. An embedded product may contain different types of hardware components like Wi-Fi module, File systems, Storage device interface, etc. The initialisation of these devices and the protocols required for communicating with these devices may be different. All these requirements are implemented in drivers and a single driver will not be able to satisfy all these. Hence each hardware (more specifically each class of hardware) requires a unique driver component.

Certain drivers come as part of the OS kernel and certain drivers need to be installed on the fly. For example, the program storage memory for an embedded product, say NAND Flash memory requires a NAND Flash driver to read and write data from/to it. This driver should come as part of the OS kernel image. Certainly the OS will not contain the drivers for all devices and peripherals under the Sun. It contains only the necessary drivers to communicate with the onboard devices (Hardware devices which are part of the platform) and for certain set of devices supporting standard protocols and device class (Say USB Mass storage device or HID devices like Mouse/keyboard). If an external device, whose driver software is not available with OS kernel image, is connected to the embedded device (Say a medical device with custom USB class implementation is connected to the USB port of the embedded product), the OS prompts the user to install its driver manually. Device drivers which are part of the OS image are known as 'Built-in drivers' or 'On-board drivers'. These drivers are loaded by the OS at the

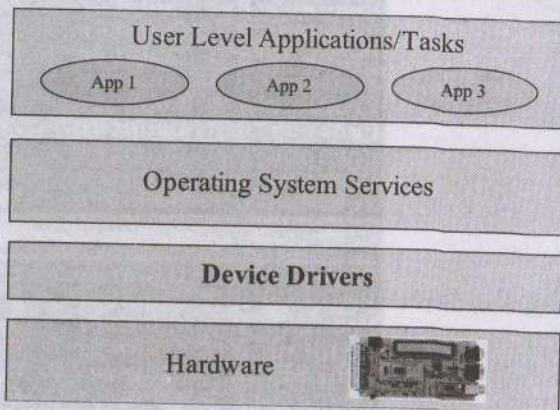


Fig. 10.39 Role of Device driver in Embedded OS based products

time of booting the device and are always kept in the RAM. Drivers which need to be installed for accessing a device are known as 'Installable drivers'. These drivers are loaded by the OS on a need basis. Whenever the device is connected, the OS loads the corresponding driver to memory. When the device is removed, the driver is unloaded from memory. The Operating system maintains a record of the drivers corresponding to each hardware.

The implementation of driver is OS dependent. There is no universal implementation for a driver. How the driver communicates with the kernel is dependent on the OS structure and implementation. Different Operating Systems follow different implementations.

It is very essential to know the hardware interfacing details like the memory address assigned to the device, the Interrupt used, etc. of on-board peripherals for writing a driver for that peripheral. It varies on the hardware design of the product. Some Real-Time operating systems like 'Windows CE' support a layered architecture for the driver which separates out the low level implementation from the OS specific interface. The low level implementation part is generally known as Platform Dependent Device (PDD) layer. The OS specific interface part is known as Model Device Driver (MDD) or Logical Device Driver (LDD). For a standard driver, for a specific operating system, the MDD/LDD always remains the same and only the PDD part needs to be modified according to the target hardware for a particular class of devices.

Most of the time, the hardware developer provides the implementation for all on board devices for a specific OS along with the platform. The drivers are normally shipped in the form of *Board Support Package*. The *Board Support Package* contains low level driver implementations for the onboard peripherals and OEM Adaptation Layer (OAL) for accessing the various chip level functionalities and a bootloader for loading the operating system. The OAL facilitates communication between the Operating System (OS) and the target device and includes code to handle interrupts, timers, power management, bus abstraction, generic I/O control codes (IOCTLs), etc. The driver files are usually in the form of a dll file. Drivers can run on either user space or kernel space. Drivers which run in user space are known as *user mode drivers* and the drivers which run in kernel space are known as *kernel mode drivers*. User mode drivers are safer than kernel mode drivers. If an error or exception occurs in a user mode driver, it won't affect the services of the kernel. On the other hand, if an exception occurs in the kernel mode driver, it may lead to the kernel crash. The way how a device driver is written and how the interrupts are handled in it are operating system and target hardware specific. However regardless of the OS types, a device driver implements the following:

1. Device (Hardware) Initialisation and Interrupt configuration
2. Interrupt handling and processing
3. Client interfacing (Interfacing with user applications)

The Device (Hardware) initialisation part of the driver deals with configuring the different registers of the device (target hardware). For example configuring the I/O port line of the processor as Input or output line and setting its associated registers for building a General Purpose IO (GPIO) driver. The interrupt configuration part deals with configuring the interrupts that needs to be associated with the hardware. In the case of the GPIO driver, if the intention is to generate an interrupt when the Input line is asserted, we need to configure the interrupt associated with the I/O port by modifying its associated registers. The basic Interrupt configuration involves the following.

1. Set the interrupt type (Edge Triggered (Rising/Falling) or Level Triggered (Low or High)), enable the interrupts and set the interrupt priorities.
2. Bind the Interrupt with an Interrupt Request (IRQ). The processor identifies an interrupt through IRQ. These IRQs are generated by the Interrupt Controller. In order to identify an interrupt the interrupt needs to be bonded to an IRQ.

3. Register an Interrupt Service Routine (ISR) with an Interrupt Request (IRQ). ISR is the handler for an interrupt. In order to service an interrupt, an ISR should be associated with an IRQ. Registering an ISR with an IRQ takes care of it.

With these the interrupt configuration is complete. If an interrupt occurs, depending on its priority, it is serviced and the corresponding ISR is invoked. The processing part of an interrupt is handled in an ISR. The whole interrupt processing can be done by the ISR itself or by invoking an Interrupt Service Thread (IST). The IST performs interrupt processing on behalf of the ISR. To make the ISR compact and short, it is always advised to use an IST for interrupt processing. The intention of an interrupt is to send or receive command or data to and from the hardware device and make the received data available to user programs for application specific processing. Since interrupt processing happens at kernel level, user applications may not have direct access to the drivers to pass and receive data. Hence it is the responsibility of the Interrupt processing routine or thread to inform the user applications that an interrupt is occurred and data is available for further processing. The client interfacing part of the device driver takes care of this. The client interfacing implementation makes use of the Inter Process communication mechanisms supported by the embedded OS for communicating and synchronising with user applications and drivers. For example, to inform a user application that an interrupt is occurred and the data received from the device is placed in a shared buffer, the client interfacing code can signal (or set) an event. The user application creates the event, registers it and waits for the driver to signal it. The driver can share the received data through shared memory techniques. IOCTLs, shared buffers, etc. can be used for data sharing. The story line is incomplete without performing an interrupt done (Interrupt processing completed) functionality in the driver. Whenever an interrupt is asserted, while vectoring to its corresponding ISR, all interrupts of equal and low priorities are disabled. They are re-enable only on executing the interrupt done function (Same as the Return from Interrupt RETI instruction execution for 8051) by the driver. The interrupt done function can be invoked at the end of corresponding ISR or IST.

We will discuss more about device driver development in a dedicated book coming under this book series.

10.10 HOW TO CHOOSE AN RTOS

The decision of choosing an RTOS for an embedded design is very crucial. A lot of factors needs to be analysed carefully before making a decision on the selection of an RTOS. These factors can be either functional or non-functional. The following section gives a brief introduction to the important functional and non-functional requirements that needs to be analysed in the selection of an RTOS for an embedded design.

10.10.1 Functional Requirements

Processor Support It is not necessary that all RTOS's support all kinds of processor architecture. It is essential to ensure the processor support by the RTOS.

Memory Requirements The OS requires ROM memory for holding the OS files and it is normally stored in a non-volatile memory like FLASH. OS also requires working memory RAM for loading the OS services. Since embedded systems are memory constrained, it is essential to evaluate the minimal ROM and RAM requirements for the OS under consideration.

Real-time Capabilities It is not mandatory that the operating system for all embedded systems need to be Real-time and all embedded Operating systems are 'Real-time' in behaviour. The task/process

After Sales For a commercial embedded RTOS, after sales in the form of e-mail, on-call services, etc. for bug fixes, critical patch updates and support for production issues, etc. should be analysed thoroughly.



Summary

- ✓ The *Operating System* is responsible for making the system convenient to use, organise and manage system resources efficiently and properly.
- ✓ Process/Task management, Primary memory management, File system management, I/O system (Device) management, Secondary Storage Management, protection implementation, Time management, Interrupt handling, etc. are the important services handled by the OS kernel.
- ✓ The core of the operating system is known as *kernel*. Depending on the implementation of the different kernel services, the kernel is classified as *Monolithic* and *Micro*. *User Space* is the memory area in which user applications are confined to run, whereas *kernel space* is the memory area reserved for kernel applications.
- ✓ Operating systems with a generalised kernel are known as *General Purpose Operating Systems (GPOS)*, whereas operating systems with a specialised kernel with deterministic timing behaviour are known as *Real-Time Operating Systems (RTOS)*.
- ✓ In the operating system context a task/process is a program, or part of it, in execution. The process holds a set of registers, process status, a Program Counter (PC) to point to the next executable instruction of the process, a stack for holding the local variables associated with the process and the code corresponding to the process.
- ✓ The different states through which a process traverses through during its journey from the newly created state to finished state is known as *Process Life Cycle*.
- ✓ Process management deals with the creation of a process, setting up the memory space for the process, loading the process's code into the memory space, allocating system resources, setting up a Process Control Block (PCB) for the process and process termination/deletion.
- ✓ A thread is the primitive that can execute code. It is a single sequential flow of control within a process. A process may contain multiple threads. The act of concurrent execution of multiple threads under an operating system is known as *multithreading*.
- ✓ Thread standards are the different standards available for thread creation and management. POSIX, Win32, Java, etc. are the commonly used thread creation and management libraries.
- ✓ The ability of a system to execute multiple processes simultaneously is known as *multiprocessing*, whereas the ability of an operating system to hold multiple processes in memory and switch the processor (CPU) from executing one process to another process is known as *multitasking*. Multitasking involves *Context Switching*, *Context Saving* and *Context Retrieval*.
- ✓ *Co-operative* multitasking, *Preemptive* multitasking and *Non-preemptive* multitasking are the three important types of multitasking which exist in the Operating system context.
- ✓ CPU utilisation, Throughput, Turn Around Time (TAT), Waiting Time and Response Time are the important criterions that need to be considered for the selection of a scheduling algorithm for task scheduling.
- ✓ Job queue, Ready queue and Device queue are the important queues maintained by an operating system in association with CPU scheduling.
- ✓ First Come First Served (FCFS)/First in First Out (FIFO), Last Come First Served (LCFS)/Last in First Out (LIFO), Shortest Job First (SJF), priority based scheduling, etc. are examples for Non-preemptive scheduling, whereas Preemptive SJF Scheduling/Shortest Remaining Time (SRT), Round Robin (RR) scheduling and priority based scheduling are examples for preemptive scheduling.
- ✓ Processes in a multitasking system falls into either *Co-operating* or *Competing*. The co-operating processes share data for communicating among the processes through *Inter Process Communication (IPC)*, whereas competing